COMPUTER NETWORKING • TRANSPORT LAYER

TCP / UDP 与可靠传输
在不可靠的网络上造可靠的路

一本面向秋招的传输层深度学习书。从"浏览器打开一个网页时网卡上到底发生了什么"讲起，依次讲清 UDP 与 TCP 的报文结构和工作原理、三次握手与四次挥手的每一个状态、SYN Flood 攻防与各种异常，最后在 UDP 之上亲手搭出一套可靠传输协议，并拆解 KCP 和 QUIC 的设计取舍。关键结论都配有本机实测的 C++ 程序输出或可复现的仿真数据。

5 部分 · 12 章 · 1 篇速记附录　2 个实测 / 仿真程序　30+ 张原理插图　2 个交互实验

网页 · 视频 · 在线游戏 场景贯穿

CHAPTER 00

阅读指南：一条主线串起传输层

先说清楚这本书在讲什么、按什么顺序讲、遇到哪种标记该怎么读。

一条主线：可靠性是一组机制，而不是一个开关

传输层的知识点乍看很散：端口、校验和、序号、三次握手、滑动窗口、拥塞控制、TIME_WAIT、KCP、QUIC……面试时也常被拆成一道道独立的题来问。但它们其实都在回答同一个问题：

> 🧭 **全书主线**
>
> **IP 网络只承诺"尽力而为"：包会丢、会乱序、会重复、会出错、会因为拥塞而变慢。要想在它上面提供"可靠"的服务，就要拿出一组机制去对付这些问题，而每个机制都有代价（延迟、带宽、内存、复杂度）。**
>
> UDP 几乎什么机制都不加，把选择权交给应用；TCP 把一整套机制固化在操作系统内核里，给所有应用同一种"可靠有序字节流"；可靠 UDP（KCP、QUIC、游戏引擎的网络层）则在用户态*按需挑选、重新调参*这些机制。

读的时候遇到新知识点，可以先问自己两句：*它是在对付哪一种网络问题？它付出了什么代价？*能答上这两句，这个知识点就不再是孤立的一道题，而是知识网里的一个节点。

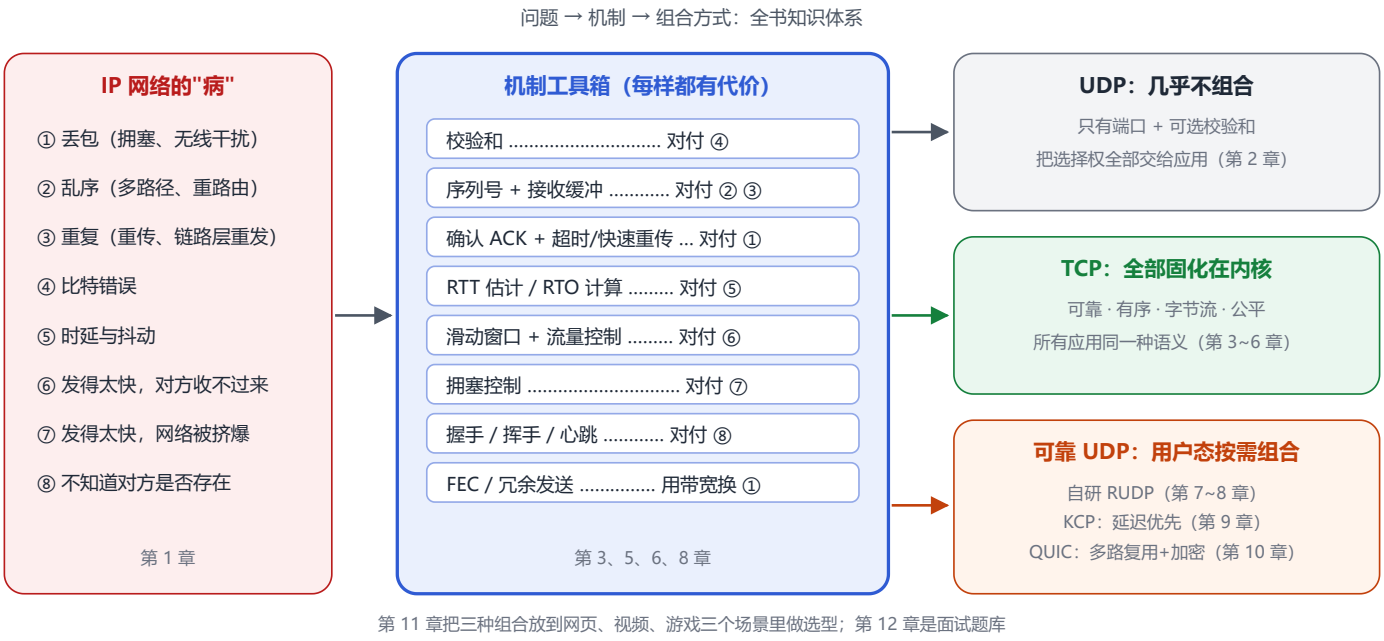


图 0-1 全书知识体系：左边是网络的"病"，中间是治病的"药"，右边是三种不同的"药方"

全书路线图

部分	章节	核心问题	面试权重
一 · 网络基础	1 从一次网页请求说起	分层、封装、端口、IP 尽力而为、MTU/MSS	**铺垫**
二 · TCP 与 UDP	2 UDP：薄到极致的传输层	8 字节头部、无连接、消息边界、校验和	**高**
	3 TCP：可靠字节流的全套机制	头部字段、序号确认、重传、滑动窗口、拥塞控制、粘包	**极高**
	4 TCP vs UDP 对比与选型	对比表、场景选型、常见追问	**极高**
三 · 连接管理	5 三次握手	状态与序号、为什么三次、半/全连接队列、SYN Flood、异常	**极高**
	6 四次挥手	半关闭、TIME_WAIT、CLOSE_WAIT、RST、异常	**极高**
四 · 可靠 UDP	7 为什么要在 UDP 上造轮子	队头阻塞、握手时延、内核僵化、可靠性光谱	**高（游戏岗必问）**
	8 亲手实现可靠 UDP	序号/ACK、RTO、ARQ、窗口、乱序去重、拥塞、FEC	**高**
	9 KCP：以带宽换延迟	RTO×1.5、快速重传、四个队列、参数调优	**高（游戏岗）**
	10 QUIC：下一代传输协议	连接 ID、0-RTT、流、包号、帧重传	**中高**
五 · 实战与面试	11 场景选型与游戏网络	网页 / 视频 / 游戏三场景方案	**中**
	12 面试题库 + 附录速记卡	40 道高频题、口述模板	**冲刺必看**

赶时间的最短路径：**第 3 章 → 第 5 章 → 第 6 章 → 第 4 章 → 第 12 章**（约 2.5 小时）。面游戏客户端 / 服务器岗位，再加上 **第 7、8、9 章**。

书中的标记

> ✅ **面试答法**
>
> 可以直接组织成口头回答的要点。

> ⚠️ **易错点**
>
> 常见的错误理解，或者面试官爱挖的坑。

> 🔬 **深入内核**
>
> Linux 内核或 RFC 里的具体实现细节。第一遍可以跳过，二面、三面前务必读。

> 🧪 **实测证据**
>
> 本书配套程序在本机运行得到的真实输出，用来证明正文结论。

> 🎮 **游戏开发视角**
>
> 这个知识点在游戏客户端 / 服务器中的应用。

> 📦 **场景案例**
>
> 用网页请求、视频传输、在线游戏等具体场景佐证讲解。

每章开头有**本章目标**，结尾有黑底的**本章小结**和可以展开答案的**思考题**。

关于配套程序

配套源码在本书同目录的 `demos/` 下，全部用 **MSVC 14.44（VS2022）**、`/std:c++20 /O2` 编译，在 Windows 11 x64 上实际运行，书中的输出均为原样粘贴。双击 `demos/build_and_run.bat` 就能在你自己的机器上复现。

程序	证明什么	所在章节
`d1_tcp_udp_behaviors`	真实 socket 实测：UDP 保留消息边界 / TCP 粘包；UDP 发往空端口"成功"但收到 ICMP；TCP 连空端口被 RST 拒绝（Windows 耗时约 2 s）；四次挥手中 `CLOSE_WAIT` / `FIN_WAIT_2` / `TIME_WAIT` 的出现时机；半关闭；`SO_LINGER` 发 RST	第 2、3、5、6 章
`d2_rudp_sim`	虚拟时钟仿真（RTT 60 ms、可调丢包）：停等 / GBN / SR / 快速重传 / 类 TCP / 类 KCP / 冗余发送 在批量传输与 100 Hz 实时流两种场景下的完成时间、重传占比和时延分布	第 8、9、11 章

> ⚠️ **关于仿真数据**
>
> `d2` 是**协议逻辑真实、网络是模型**的仿真：本机回环几乎不丢包也没有延迟，用真实 socket 看不出协议差异。仿真里的"类 TCP"是教学用简化版（没有 TLP、RACK、PRR、BBR 等现代 Linux 优化），绝对数值偏悲观，**请看趋势和相对大小，不要把绝对值当作真实 TCP 的性能。**书中引用时会再次说明。

PART ONE

第一部分　网络基础：传输层站在哪里

先搞清楚传输层上面是谁、下面是谁、下面的 IP 到底承诺了什么、没承诺什么。TCP 和 UDP 的一切设计，都是对"IP 只尽力而为"这件事的回应。

CHAPTER 01

从一次网页请求说起

在浏览器里敲下一个网址，到页面开始出现，网线上依次走过了 UDP、TCP、TLS、HTTP。

本章目标

- 能画出一次 HTTPS 网页请求的完整时序，并说出每一段属于哪一层、花了几个 RTT
- 说清 TCP/IP 分层模型、封装与解封装，以及各层 PDU 的名字
- 理解 IP 的"尽力而为"和端到端原则，明白为什么可靠性要放在传输层做
- 掌握端口、套接字、五元组、MTU 与 MSS 这几个后面反复出现的基本概念

1.1 敲下回车之后：一张时序图看全流程

假设你在浏览器里输入 `https://www.example.com` 并回车，且本机没有任何缓存。抛开浏览器内部的解析和渲染，单看网络，大致会发生四件事：

1. **DNS 解析**：把域名翻译成 IP 地址。浏览器（通过操作系统）向 DNS 服务器发一个查询，**通常用 UDP 的 53 端口**，一问一答。
2. **TCP 三次握手**：和 Web 服务器的 443 端口建立一条 TCP 连接。
3. **TLS 握手**：在 TCP 连接之上协商加密参数、验证服务器证书。TLS 1.3 需要 1 个 RTT。
4. **HTTP 请求与响应**：发送 `GET /`，收到 HTML。

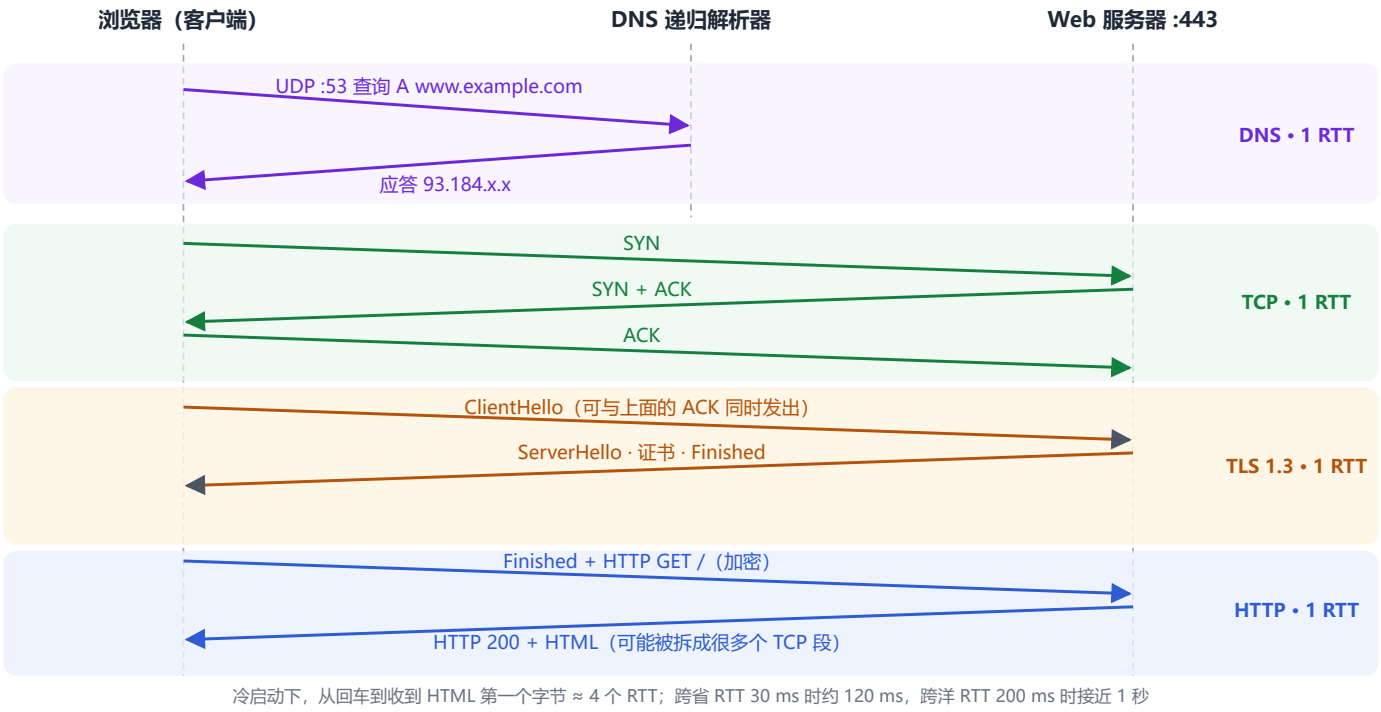


图 1-1 一次冷启动的 HTTPS 网页请求：DNS 走 UDP，网页本身走 TCP + TLS；每一段都至少花 1 个 RTT

这张图里已经藏着本书的大部分主题：

- **DNS 为什么用 UDP?** 一问一答、报文小，用 TCP 光握手就要多花 1 个 RTT，丢了重问一次即可（第 2 章、第 4 章）。
- **SYN、SYN+ACK、ACK 各自携带了什么、为什么是三次?** （第 5 章）
- **HTML 被拆成很多个 TCP 段，丢了一个怎么办？乱序了怎么办？发太快怎么办?** （第 3 章）
- **能不能把 TCP 握手和 TLS 握手合并，把 4 个 RTT 压缩下来?** 这正是 QUIC / HTTP/3 的动机之一（第 10 章）。

> 📦 **场景案例：RTT 才是网页慢的主因**
>
> 很多人以为网页慢是因为"带宽不够"。实际上，打开一个普通网页只需要几百 KB，100 Mbps 的带宽传完只要几十毫秒；真正拖慢首屏的是**来回次数（RTT 的个数）**。这就是为什么工程上大量优化都在"省 RTT"：DNS 预解析、TCP 连接复用（HTTP Keep-Alive）、TLS 会话恢复、TCP Fast Open、HTTP/3 的 0-RTT。记住这个视角，后面讨论握手次数时你会更有体感。

1.2 分层模型：每层只做一件事

网络通信被拆成若干层，每一层只依赖下一层提供的服务，并向上一层提供服务。教科书里有两套模型：

OSI 七层	TCP/IP 四层（五层教学版）	负责什么	典型协议	PDU 名称
应用层 / 表示层 / 会话层	应用层	具体业务语义	HTTP、DNS、TLS*、SSH、游戏协议	报文（message）
传输层	传输层	**进程到进程**：端口复用、可靠性、流量 / 拥塞控制	**TCP、UDP**、SCTP、QUIC*	TCP 段（segment）/ UDP 数据报（datagram）
网络层	网际层	**主机到主机**：寻址与路由，跨越多个网络	IPv4、IPv6、ICMP	分组 / 包（packet）
数据链路层	网络接口层（链路层 + 物理层）	同一链路上相邻节点之间传帧	以太网、Wi-Fi、PPP	帧（frame）
物理层		比特 ↔ 电 / 光 / 无线信号	双绞线、光纤	比特

* TLS 通常被视为位于传输层和应用层之间；QUIC 运行在 UDP 之上，但承担的是传输层职责，所以常被称为"用户态传输层"。

封装与解封装

发送时，数据从上往下走，每经过一层就在前面加上一层"信封"（头部）；接收时从下往上走，每一层拆掉属于自己的信封，再根据头部里的字段决定交给上面的谁。

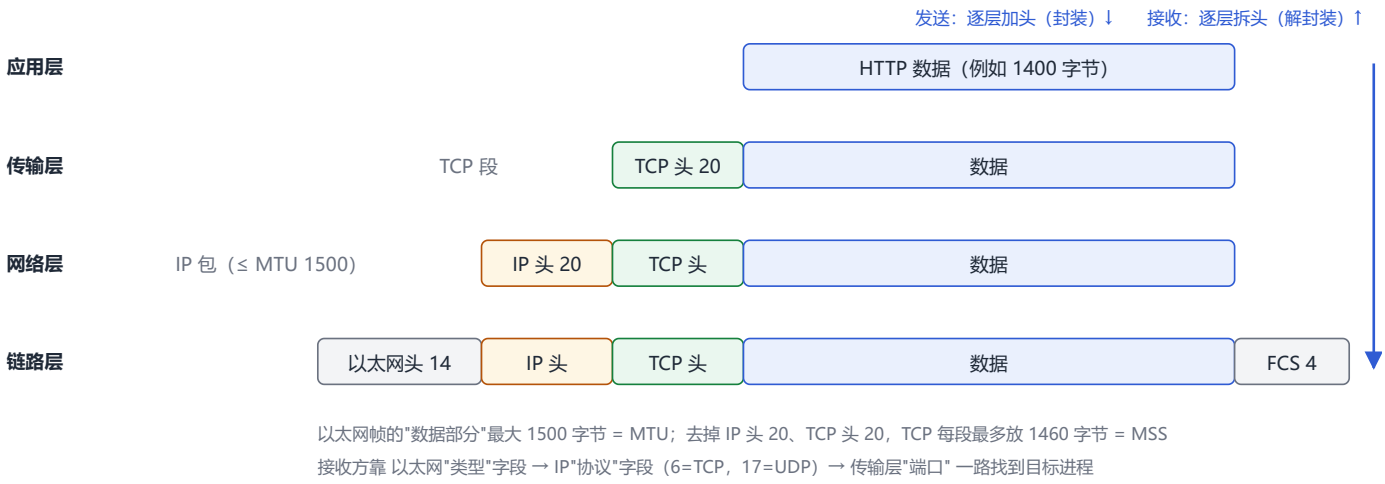


图 1-2 封装：一个 1400 字节的 HTTP 数据在发出前被套上三层头部

✅ 面试答法：为什么要分层?

① **解耦**：每层只关心自己的职责，换掉其中一层（如把有线换成 Wi-Fi、把 IPv4 换成 IPv6）不影响其他层；② **复用**：所有应用共享同一套 TCP/IP 实现；③ **标准化**：不同厂商只要遵守层间接口就能互通。代价是每层都有头部开销，层与层之间也可能出现信息不对称（例如 TCP 不知道底层是无线链路，把无线干扰导致的丢包误当成拥塞，第 3 章会讲）。

1.3 IP 的承诺：尽力而为

网络层的 IP 协议负责把一个包从源主机送到目标主机，中间可能经过几十个路由器。但 IP **只承诺"尽力而为"（best effort）**，具体来说：

❌ 不保证送达

路由器队列满了就直接丢包；TTL 减到 0 也丢。丢了不会通知任何人（最多偶尔回一个 ICMP）。

❌ 不保证顺序

先发的包可能走了更长的路径，后发先至。

❌ 不保证不重复

链路层重发、路由抖动都可能让同一个包到达两次。

❌ 不保证数据正确

IPv4 头部校验和只校验**头部**，不管数据；IPv6 连头部校验和都去掉了。

❌ 不控制速率

✅ 只保证一件事

你发多快它就转多快，直到某个路由器扛不住开始丢包。

按目标 IP 地址尽量把包往正确的方向转发。

为什么不让 IP 层直接做可靠？——端到端原则

如果让每个路由器都保证可靠（逐跳确认、逐跳重传），路由器就得为每条流保存状态、缓存数据，成本极高；而且即使每一跳都可靠，数据仍可能在终端主机的内存里出错，**最终还是要由两端再确认一次**。这就是著名的**端到端原则（End-to-End Argument）**：

> 📐 **端到端原则**
> 某个功能只有在通信的端点才能被完整、正确地实现时，就应该放在端点实现；网络中间只做简单、通用、高效的事情。
> → 所以 IP 保持"笨而快"，**可靠性交给两端主机的传输层**（TCP），或者干脆交给应用自己（UDP + 应用层可靠）。

这条原则也解释了本书最后一部分的存在：既然可靠性本来就是端点的事，那它放在**内核里的 TCP** 还是**用户态的 KCP / QUIC**，只是工程取舍，没有原则上的对错。

1.4 端口、套接字与五元组：找到具体的进程

IP 地址只能定位到**主机**，而一台主机上同时跑着浏览器、微信、游戏客户端。传输层的第一个职责就是**复用与分用**（multiplexing / demultiplexing）：发送时把多个进程的数据汇入同一个 IP 层，接收时再按端口号分发给正确的进程。

- **端口（port）**：16 位无符号整数，0～65535。0～1023 是知名端口（HTTP 80、HTTPS 443、DNS 53、SSH 22），1024～49151 是注册端口，49152～65535 是动态（临时）端口。Linux 默认的临时端口范围是 `32768～60999`（`net.ipv4.ip_local_port_range`）。
- **套接字（socket）**：操作系统提供给应用的通信端点抽象。应用读写 socket，内核负责把数据交给 TCP/UDP 协议栈。
- **五元组**：`{协议, 源 IP, 源端口, 目的 IP, 目的端口}`，唯一标识一条通信。

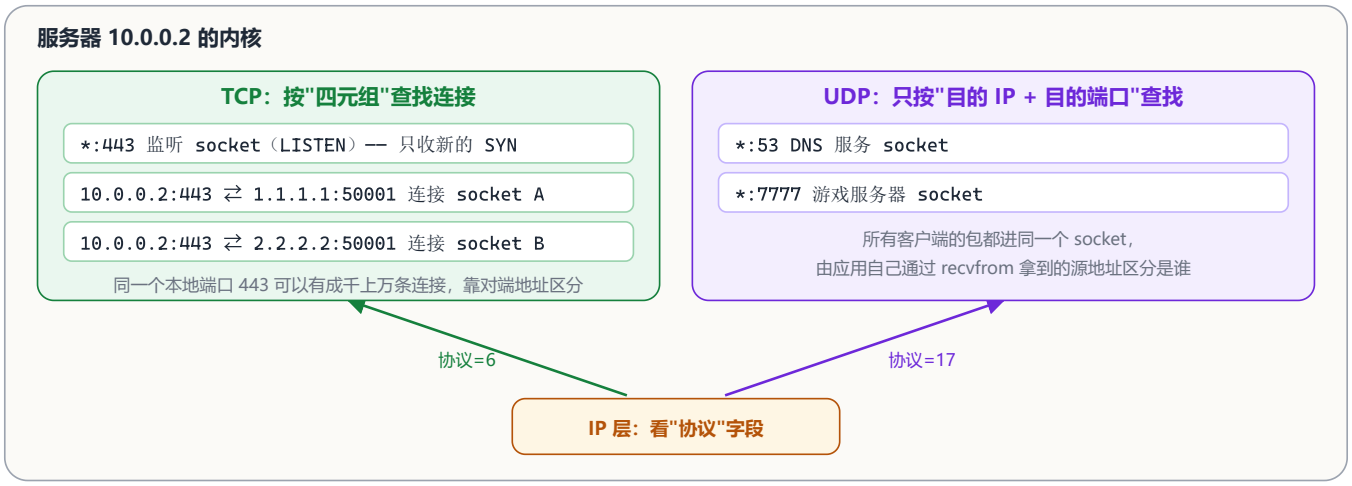


图 1-3 分用：TCP 每条连接一个 socket（按四元组区分）；UDP 一个端口一个 socket，"连接"的概念需要应用自己维护

> ⚠️ **易错点：一台服务器最多只能有 65535 个 TCP 连接?**
>
> 错。端口号的 65535 限制针对的是**同一个四元组中的一个端口字段。**服务器只监听一个端口（443），每条连接由 `{客户端 IP, 客户端端口}` 区分，理论上限是"客户端 IP 数 × 客户端端口数"，远超 65535。实际限制来自**文件描述符上限和内存**（每条连接的内核结构加收发缓冲区要占几 KB 到几十 KB）。
>
> 反过来，**客户端**连同一个服务器的同一个端口时，确实最多只能用掉约 2.8 万个（Linux 默认临时端口范围）本地端口——这就是第 6 章 TIME_WAIT 过多会导致"端口耗尽"的根源。

1.5 MTU、MSS 与分片

链路层一次能承载的最大数据长度叫 **MTU（Maximum Transmission Unit）**，以太网是 1500 字节。比 MTU 大的 IP 包要么被**分片**（IPv4 允许路由器分片），要么被丢弃并回一个 ICMP "需要分片"（设置了 DF 位或 IPv6）。

- **分片的坏处**：任何一个分片丢了，整个 IP 包都要作废；分片重组消耗接收方资源；很多防火墙和 NAT 会直接丢弃分片。所以**无论 TCP 还是 UDP，都应该尽量避免 IP 分片。**
- **TCP 怎么避免**：三次握手时双方在 SYN 里通过 **MSS 选项**告诉对方"我能收的最大段长"，一般是 MTU − 40 = 1460；之后 TCP 自己把字节流切成不超过 MSS 的段。再配合**路径 MTU 发现（PMTUD）**适应中间更小的 MTU（如 PPPoE 的 1492）。

- **UDP 怎么避免**：UDP 不切分，应用交给它多大，它就发多大的数据报。所以应用必须**自己控制每个数据报的大小**：以太网下 UDP 负载不超过 1500 − 20 − 8 = **1472** 字节；考虑到隧道、VPN、移动网络，实践中常取 **1200～1400**（QUIC 规定 IPv4/IPv6 下至少支持 1200 字节，KCP 默认 MTU 为 1400）。

> 🎮 **游戏开发视角**
>
> 游戏的同步包通常只有几十到几百字节，天然不会触发分片；但"进入场景时下发全量状态""发送一张大地图快照"这类大消息就必须在应用层**手动分片、编号、重组**，这正是第 8 章可靠 UDP 里"消息分片"机制要解决的问题。UE 的网络层把一个大的 Bunch 拆成多个 Partial Bunch 也是同一个道理。

1.6 传输层的任务清单

现在可以给传输层列一份任务清单了。所有传输层协议都必须做第 1 项；剩下几项做不做、做到什么程度，就是 TCP、UDP 和各种可靠 UDP 的根本区别：

#	任务	UDP	TCP	可靠 UDP（KCP / QUIC 等）
1	进程间复用 / 分用（端口）	✓	✓	✓（借用 UDP 端口，常再加会话 ID）
2	差错检测（校验和）	✓（IPv4 可选）	✓ 必选	✓（UDP 校验和，QUIC 还有 AEAD 认证）
3	可靠交付（不丢、不重）	✗	✓	按需：可对每条消息选择
4	按序交付	✗	✓（整条连接严格有序）	按需：可按流 / 通道有序
5	流量控制	✗	✓	✓
6	拥塞控制	✗	✓	可选 / 可替换算法
7	连接管理	✗	✓（三次握手 / 四次挥手）	各自实现（QUIC 有，KCP 没有）

本章小结

- 一次冷启动 HTTPS 请求 ≈ DNS(UDP) + TCP 握手 + TLS 握手 + HTTP，约 4 个 RTT；**网页慢多半慢在 RTT 个数。**
- TCP/IP 四层：应用 → 传输（进程到进程）→ 网际（主机到主机）→ 网络接口；PDU 依次叫报文、段 / 数据报、分组、帧。
- IP **尽力而为**：可能丢、乱、重、错、拥塞。按**端到端原则**，可靠性放在端点做。
- 端口 16 位；TCP 连接按四元组区分，UDP socket 按目的地址区分；服务器连接数不受 65535 限制。
- MTU 1500 → TCP MSS 1460、UDP 安全负载 ≤ 1472（实践取 1200 ~ 1400）；尽量避免 IP 分片。

▶ **思考题 1：为什么 DNS 一般用 UDP，什么情况下又会改用 TCP?**

▶ **思考题 2：服务器只监听 443 一个端口，为什么能同时服务几十万个客户端？瓶颈在哪?**

▶ **思考题 3：一个游戏要用 UDP 发送 10 KB 的场景快照，直接 sendto 一次行不行?**

PART TWO

第二部分　TCP 与 UDP：两种截然不同的答案

面对一个尽力而为的 IP 层，UDP 的答案是"我什么也不加，交给你"，TCP 的答案是"我全包了"。先看薄的，再看厚的，最后放在一起比较。

CHAPTER 02

UDP：薄到极致的传输层

UDP = IP + 端口 + 校验和。它的价值恰恰在于"不做什么"。

本章目标

- 背出 UDP 头部 4 个字段及含义，会算最大负载，知道伪首部为什么存在
- 理解"无连接""保留消息边界""尽力交付"三个特性的具体含义，并能用实测输出佐证
- 掌握 UDP 与 TCP 的 socket 编程调用序列差异
- 说出 UDP 的优缺点与典型应用场景，以及使用 UDP 时必须由应用补上的东西

2.1 一句话定义

UDP（User Datagram Protocol，用户数据报协议，RFC 768）是一种**无连接、不可靠、面向数据报**的传输层协议。它在 IP 之上只额外提供了两样东西：**端口**（让数据能交给正确的进程）和**校验和**（检测数据是否损坏）。RFC 768 全文只有 3 页，这在 IETF 的协议里极其罕见。

> **一个类比：寄明信片**
>
> UDP 像寄明信片：写上地址（IP）和收件人（端口）就丢进邮筒，不需要事先打电话确认对方在家；每张明信片独立投递，可能丢、可能晚到、两张寄出的顺序和收到的顺序可能相反；但**每张明信片都是完整的一张**，不会出现半张。TCP 则像打电话：先拨号接通（握手），然后连续说话，对方听不清会让你重说，最后说再见挂断（挥手）。

2.2 报文结构：只有 8 字节

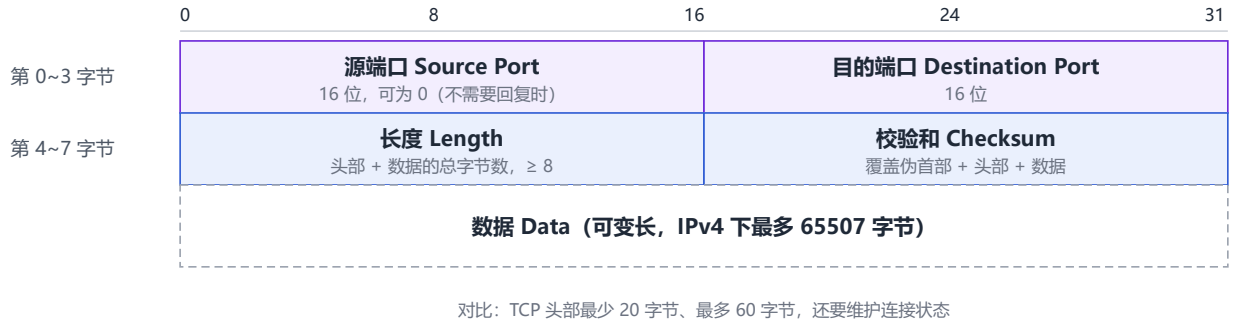


图 2-1 UDP 头部：两个 32 位字，共 8 字节

字段	位数	含义与细节
源端口	16	发送方端口。客户端一般由系统分配临时端口；如果不需要对方回复，可以填 0。
目的端口	16	接收方端口，内核据此找到目标 socket。找不到时回复 ICMP "端口不可达"。
长度	16	UDP 头部加数据的总长度，最小 8。最大 65535，所以 IPv4 下数据最多 65535 − 8 − 20（IP 头）= **65507** 字节。
校验和	16	把**伪首部 + UDP 头部 + 数据**按 16 位反码求和再取反。IPv4 下可以填 0 表示"未计算"；**IPv6 下必须计算**（因为 IPv6 去掉了 IP 头部校验和）。

伪首部：为什么校验和要把 IP 地址也算进去

计算校验和时，会在 UDP 报文前面临时拼一个 12 字节的**伪首部（pseudo header）**：源 IP、目的 IP、一个全 0 字节、协议号 17、UDP 长度。伪首部**不会被发送**，只参与计算。

目的是**检测"送错了地方"**：如果 IP 头部在传输中出错，导致一个本该发给 A 的数据报被交给了 B，单看 UDP 头部和数据是发现不了的；把 IP 地址纳入校验后，B 计算出的校验和就对不上，会丢弃这个数据报。TCP 的校验和用的是同样的方法（协议号为 6）。

16 位反码求和（UDP / TCP / IP 校验和通用算法）

```
uint16_t InternetChecksum(const uint8_t* data, size_t len, uint32_t sum = 0) {
    for (size_t i = 0; i + 1 < len; i += 2)
        sum += (uint32_t(data[i]) << 8) | data[i + 1];   // 按 16 位大端累加
    if (len & 1) sum += uint32_t(data[len - 1]) << 8;    // 奇数长度：末尾补 0
    while (sum >> 16) sum = (sum & 0xFFFF) + (sum >> 16); // 进位回卷（反码加法）
    return uint16_t(~sum);                                // 取反
}
// 计算 UDP 校验和：先把伪首部（源IP、目的IP、0、17、UDP长度）累加进 sum，再累加 UDP 头部和数据
// 接收方对"伪首部 + 整个 UDP 报文（含校验和字段）"再算一遍，结果为 0 说明无错
```

> ⚠️ **易错点：校验和能保证数据正确吗？**
>
> 不能完全保证。16 位反码和很弱：两个 16 位字互换位置、或两处错误恰好抵消，它都发现不了。链路层的 CRC32 能挡住绝大部分随机错误，但对恶意篡改毫无办法。**需要防篡改就必须用密码学手段**（TLS、DTLS、QUIC 的 AEAD 加密认证）。

2.3 无连接：想发就发，发完不管

"无连接"的意思是：通信前不需要握手，内核里**不为"对方"保存任何状态**。一个 UDP socket 可以给任意多个地址发数据，也可以收任意多个地址发来的数据。这带来三个直接后果：

1. **发送没有前置延迟**：第一个包就能携带业务数据，省掉握手的 1 个 RTT。
2. **服务器没有连接状态**：一个 socket 就能服务海量客户端，内存占用极低，也不存在 TIME_WAIT 之类的连接管理问题。
3. **发送方不知道对方是否存在**：`sendto` 返回成功，只说明数据已经交给了本机内核，**不代表对方收到了，甚至不代表对方存在。**

💉 实测证据（d1 • E2）：发给一个没人监听的端口

```
d1_tcp_udp_behaviors.exe · E2

== E2 UDP 无连接：发给没人监听的端口 ==
  sendto 返回 4（发送"成功"——UDP 不知道对方在不在）
  随后 recvfrom 返回 -1，WSAGetLastError=10054（10054=WSAECONNRESET：内核收到 ICMP 端口不可达）
  关闭 SIO_UDP_CONNRESET 后：recvfrom 返回 -1，WSAGetLastError=10060（10060=超时，错误被静默丢弃）
```

sendto 照样返回发出的 4 字节。对方主机的内核发现 40012 端口没人监听，回了一个 **ICMP 端口不可达**。Windows 会把这个 ICMP 错误"记"在 socket 上，让下一次 `recvfrom` 报 `WSAECONNRESET` ——对一个"无连接"协议来说这个错误名相当有迷惑性。

🎮 游戏开发视角：Windows 上 UDP 服务器的经典坑

游戏服务器用一个 UDP socket 服务所有玩家。某个玩家断线后，服务器继续往他的地址发包，对端回 ICMP 端口不可达，于是服务器的**下一次 recvfrom 报 WSAECONNRESET**。如果代码把这个错误当成"socket 坏了"去关闭重建，所有玩家都会掉线。解决办法就是实测中的 `WSAIoctl(SIO_UDP_CONNRESET, FALSE)`，或者把这个错误码当作可忽略的提示。Linux 上，只有对 UDP socket 调用过 `connect()` 时才会把 ICMP 错误报给应用（`ECONNREFUSED`），未 connect 的 socket 默认不报。

UDP 上的 connect() 是什么意思

UDP socket 也可以调用 `connect()`，但**不会发送任何报文**，只是在内核里记下一个默认对端：之后可以用 `send/recv` 代替 `sendto/recvfrom`，内核只接收来自这个对端的数据报，并且能把 ICMP 错误报告给应用。它是"本地过滤器"，不是"连接"。QUIC 客户端和很多游戏客户端都会对 UDP socket 调用 `connect()`，顺便还能让内核少做一次路由查找。

2.4 面向数据报：保留消息边界

UDP 的每一次 `sendto` 都对应一个独立的数据报，接收方每次 `recvfrom` 恰好取出一个完整的数据报，**既不会把两个合并，也不会只给你半个**。这叫"保留消息边界"。TCP 则是**字节流**，没有消息边界的概念。

🧪 实测证据（d1 • E1）：三次发送，两种结果

d1_tcp_udp_behaviors.exe · E1

```
== E1 消息边界：UDP 保留边界，TCP 是字节流 ==
  UDP recvfrom #1 →  5 字节："hello"
  UDP recvfrom #2 → 12 字节："tcp-udp-book"
  UDP recvfrom #3 → 27 字节："reliable-transport-over-udp"
  TCP 发送方 send() 3 次（5+12+27=44 字节），接收方 recv() 1 次 → 44 字节："hellotcp-udp-bo
okreliable-transport-over-udp"
```

同样是发送三条消息、接收方稍晚一点再读：UDP 原样收到三条；TCP 一次 `recv` 就把 44 个字节全拿走了，三条消息"粘"在一起，接收方无从知道原来的边界在哪——这就是第 3 章要讲的**粘包**。

⚠️ 易错点：接收缓冲区比数据报小会怎样?

数据报**多出来的部分会被直接丢弃**，不会留到下一次 recvfrom。Linux 会在 `msg_flags` 里置 `MSG_TRUNC`，Windows 的 recvfrom 返回错误 `WSAEMSGSIZE`（缓冲区里仍是截断后的数据）。所以 UDP 接收缓冲区要按协议的最大包长来开（例如 1500 或 64 KB）。

2.5 尽力交付：UDP 不做的事

不做的事	具体表现	如果你需要，应该怎么补
不确认、不重传	丢了就是丢了，发送方完全不知道	应用层序号 + ACK + 重传（第 8 章）
不排序	按到达顺序交给应用	应用层序号 + 接收缓冲重排，或只取"最新"的
不去重	网络重复的包会被交付两次	序号 + 已收集合（滑动位图）
不做流量控制	接收方 socket 缓冲区满了，新来的数据报被内核**静默丢弃**	增大 `SO_RCVBUF`；应用层窗口通告
不做拥塞控制	应用以多快的速度 sendto，内核就以多快的速度往外发	应用层限速（令牌桶）、拥塞控制算法
不分片	超过 MTU 的数据报交给 IP 分片	应用层控制包长 ≤ 1200 ~ 1400，大消息自己切

看到这里可能会疑惑：UDP 什么都不做，凭什么被大量使用？答案是：**"不做"本身就是一种能力**——它把这些决定权留给了最了解业务需求的应用。语音通话里一个 20 ms 的语音帧丢了，最好的处理是让解码器插值补上，而不是等 200 ms 重传过来时这句话早已说完。这个思路是第四部分的出发点。

2.6 编程模型：UDP 与 TCP 的调用序列

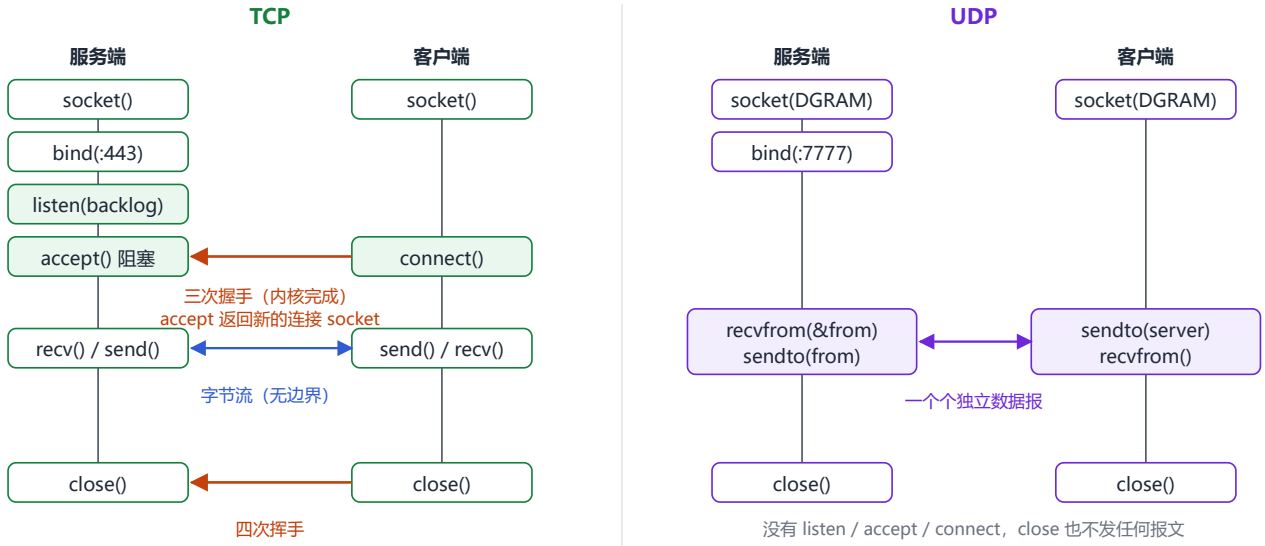


图 2-2 调用序列对比：TCP 有"监听 → 接受 → 连接 socket"的三段式，UDP 只有一个 socket 收发所有人

最小 UDP 回显服务器（Winsock，省略错误处理）

```
SOCKET s = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
sockaddr_in addr{}; addr.sin_family = AF_INET; addr.sin_port = htons(7777); addr.sin_addr.s_ac
bind(s, (sockaddr*)&addr, sizeof addr);
char buf[1500];
for (;;) {
    sockaddr_in from{}; int fromLen = sizeof from;
    int n = recvfrom(s, buf, sizeof buf, 0, (sockaddr*)&from, &fromLen);  // 一次取一个完整数据
    if (n > 0) sendto(s, buf, n, 0, (sockaddr*)&from, fromLen);           // 原样发回给来源地址
}
```

2.7 UDP 独有的能力：广播与多播

TCP 是严格的一对一连接，而 UDP 因为无连接，可以把一个数据报发往**广播地址**（如 `255.255.255.255` 或子网广播 `192.168.1.255`）或**多播组**（`224.0.0.0/4`），一次发送、多个接收者同时收到。

- **DHCP**：新设备还没有 IP 地址，只能用广播（客户端 68 → 服务器 67）寻找 DHCP 服务器。
- **局域网游戏发现**：客户端广播"有房间吗"，局域网内的主机回复房间信息。
- **mDNS / SSDP**：打印机、投屏设备的自动发现。
- **IPTV / 行情分发**：机房内用多播把同一份数据推给大量订阅者。

2.8 优缺点与典型应用

优点

① **低延迟**：无握手，第一个包就能带数据；无重传等待，不会被前面丢失的包卡住。
② **开销小**：头部 8 字节，内核无连接状态。
③ **保留消息边界**：天然适合"一个包一条消息"的协议。
④ **灵活**：支持广播 / 多播；可靠性、拥塞控制都能按需在应用层定制。

缺点

① **不可靠**：丢、乱、重全要应用自己处理。
② **无拥塞控制**：滥发会挤占网络，也会被运营商 QoS 限速。
③ **NAT 不友好**：NAT 映射超时短（常见 30 s ~ 数分钟），必须定期发心跳保活。
④ **易被滥用**：源地址可伪造，可被用于反射放大攻击（DNS、NTP、Memcached）。

场景	为什么用 UDP	可靠性怎么补
DNS 查询	小报文一问一答，省掉握手 RTT	超时重问；大应答回退 TCP
DHCP	没有 IP 时只能广播	超时重发
语音 / 视频通话（RTP、WebRTC）	实时性第一，迟到的数据没用	FEC、NACK 选择性重传、丢帧插值、抖动缓冲
低延迟直播	减少重传卡顿	FEC + 有限重传（SRT 等协议）
在线游戏	位置、输入只关心最新值；每毫秒都很宝贵	按消息分级：状态不可靠、指令可靠（第 8、11 章）
HTTP/3	绕开内核 TCP，实现多路复用与 0-RTT	QUIC 在用户态完整实现可靠传输（第 10 章）
VPN / 隧道（WireGuard、VXLAN）	避免"TCP 套 TCP"的双重重传	交给隧道内部的 TCP 处理

✅ 面试答法：说说你对 UDP 的理解

UDP 是无连接、不可靠、面向数据报的传输层协议，头部只有 8 字节：源端口、目的端口、长度、校验和。它在 IP 之上只加了端口复用和差错检测，不建立连接、不确认、不重传、不排序、不做流量和拥塞控制，但保留消息边界、支持广播多播。优点是延迟低、开销小、灵活，缺点是可靠性和拥塞控制都要应用自己负责。典型场景是 DNS、DHCP、音视频实时通信、在线游戏，以及 HTTP/3 底层的 QUIC。*（加分项）*如果在 UDP 上需要可靠性，可以在应用层实现序号、确认、重传和窗口，比如 KCP 或 QUIC。

本章小结

- UDP 头部 8 字节：**源端口、目的端口、长度、校验和**；IPv4 最大负载 65507 字节。
- 校验和覆盖**伪首部**（IP 地址、协议号、长度），用来发现"送错地方"；IPv4 可选、IPv6 必选；弱校验，防不了篡改。
- **无连接**：sendto 成功 ≠ 对方收到；Windows 上 ICMP 端口不可达会让下一次 recvfrom 报 WSAECONNRESET（实测）。
- **保留消息边界**：一次 sendto 对应一次 recvfrom（实测）；缓冲区不够时多余部分被丢弃。
- 不重传、不排序、不去重、不限速，这些"缺点"正是它灵活、低延迟的来源。

▶ **思考题 1：UDP 包长度字段已经有了，为什么还需要 IP 头部的总长度字段？反过来呢？**

▶ **思考题 2：UDP 服务器的接收缓冲区满了会怎样？TCP 呢？**

▶ **思考题 3：既然 UDP 不可靠，为什么说"UDP 更适合游戏"？这句话完全对吗？**

CHAPTER 03

TCP：可靠字节流的全套机制

序号、确认、重传、窗口、拥塞控制——把一个会丢会乱的网络，包装成一根"不丢不乱的管道"。

本章目标

- 画出 TCP 头部，说出每个字段和 6 个常用标志位的作用，知道常用选项（MSS、窗口扩大、SACK、时间戳）
- 理解"字节流"与序号 / 确认号的计算，能解释粘包、拆包并给出解决方案
- 讲清超时重传（RTO 计算、Karn 算法、指数退避）、快速重传与 SACK
- 讲清滑动窗口、流量控制、零窗口、Nagle 与延迟确认
- 讲清慢启动、拥塞避免、快速恢复，以及 CUBIC、BBR 的思想；区分流量控制与拥塞控制

3.1 一句话定义

TCP（Transmission Control Protocol，传输控制协议，现行标准 RFC 9293）是一种**面向连接的、可靠的、基于字节流的、全双工**传输层协议。拆开看：

面向连接

通信前先三次握手，双方各自建立连接状态（序号、窗口、缓冲区）；结束后四次挥手释放。只能一对一，不支持广播多播。

可靠

数据**不丢、不重、不错、按序**交付给应用。靠校验和 + 序号 + 确认 + 重传实现。

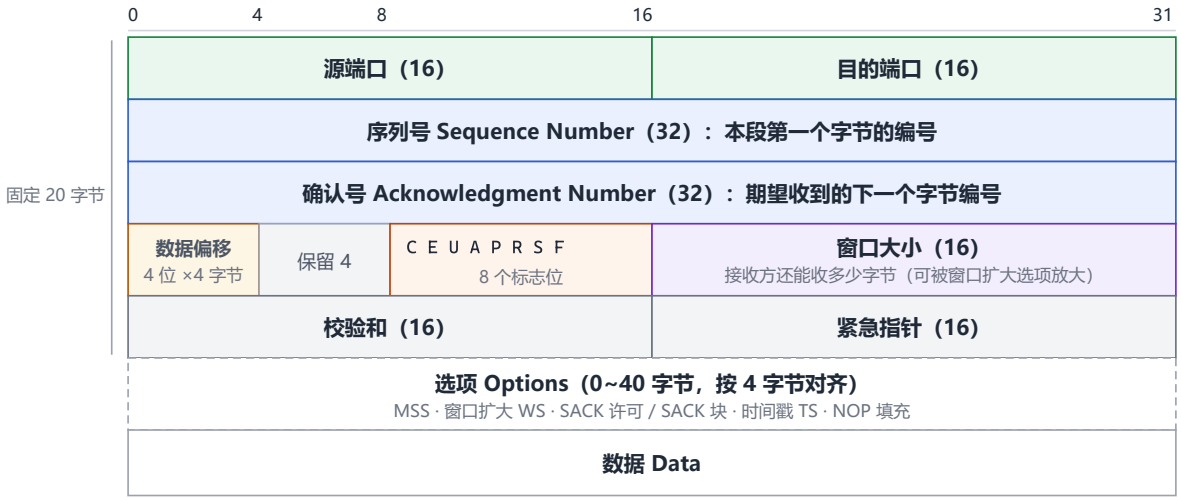
字节流

TCP 眼里只有一串连续的字节，没有"消息"的概念。应用写入的边界不会被保留。

全双工

双方可以同时收发，两个方向各有独立的序号空间和窗口，因此关闭时也要两个方向分别关闭。

3.2 头部结构：20 字节固定 + 最多 40 字节选项



C=CWR 拥塞窗口已减小　E=ECE ECN 回显（显式拥塞通知用）
U=URG 紧急指针有效　A=ACK 确认号有效（除第一个 SYN 外几乎所有段都置位）　P=PSH 尽快交付应用
R=RST 重置连接　S=SYN 同步序号（建立连接）　F=FIN 发送方数据已发完（关闭连接）
头部长度 = 数据偏移 × 4，范围 20~60 字节；SYN 和 FIN 本身各占用 1 个序号

图 3-1 TCP 头部（RFC 9293 布局）：前 20 字节固定，后面最多 40 字节选项

字段	位数	作用
源端口 / 目的端口	16 + 16	和 IP 地址一起组成四元组，标识一条连接。
序列号 seq	32	本段数据第一个字节在整个字节流中的编号。初始值 ISN 在握手时随机生成（第 5 章）。
确认号 ack	32	"我已经按序收到了 ack 之前的所有字节，下一个期望 ack"。只有 ACK 标志置位时有效。是**累计确认**。
数据偏移	4	头部长度，以 4 字节为单位，所以头部最大 15 × 4 = 60 字节。
标志位	8	见图下说明。面试最常问 SYN、ACK、FIN、RST、PSH、URG 六个。
窗口	16	接收窗口 rwnd，**流量控制**的核心。16 位最大 65535，需要窗口扩大选项才能表示更大的窗口。
校验和	16	和 UDP 一样覆盖伪首部 + 头部 + 数据，但 TCP 中**必须**计算。
紧急指针	16	URG 置位时有效，指出紧急数据的末尾。实际几乎不用（Telnet 的 Ctrl+C 是经典用例）。

常用选项

选项	出现时机	解决什么问题
MSS（kind 2）	仅 SYN / SYN+ACK	告诉对方"我能接收的最大段长"，避免 IP 分片。以太网下一般是 1460。
窗口扩大 WS（kind 3）	仅 SYN / SYN+ACK，双方都带才生效	窗口值左移 shift 位（最多 14），窗口上限扩到约 1 GB。高带宽、高时延的链路必需。
SACK 许可 / SACK（kind 4 / 5）	许可在 SYN 中协商；SACK 块在 ACK 中	告诉发送方"除了累计确认之外，我还收到了哪些不连续的块"，实现选择性重传。
时间戳 TS（kind 8）	SYN 协商后每个段都带	① 精确测量 RTT（包括重传段）；② **PAWS** 防止序号回绕时旧段被误收；③ 支撑 `tcp_tw_reuse`（第 6 章）。

> 📦 **场景案例：为什么 16 位窗口不够用**
>
> 发送方在收到确认之前最多能发"一个窗口"的数据，所以吞吐上限 ≈ 窗口 / RTT。窗口最大 64 KB、RTT 100 ms（北京到美西）时，吞吐上限只有 64 KB / 0.1 s ≈ **5 Mbps**——哪怕你的带宽是 1 Gbps。要跑满带宽，窗口至少要达到**带宽时延积（BDP）**：1 Gbps × 100 ms = 12.5 MB，所以必须用窗口扩大选项。下载海外大文件时速度上不去，常见原因之一就是某一端的接收窗口太小。

3.3 字节流、序号与"粘包"

序号按字节编号

TCP 给字节流中的**每一个字节**都编号，而不是给每个段编号。段的序号是它第一个字节的编号，确认号是"期望的下一个字节"。例如 ISN = 1000，SYN 占用 1 个序号，于是第一个数据字节的编号是 1001：

报文	seq	数据长度	接收方回复的 ack
段 1	1001	100	1101
段 2	1101	200	1301
段 3（丢失）	1301	100	—
段 4	1401	100	**仍然是 1301**（累计确认：1301 还没到，后面到了也不能确认）

最后一行引出两个问题：发送方怎么知道段 3 丢了（3.4 节的重传），以及段 4 已经到了却不能确认是不是太浪费（3.4 节的 SACK）。

粘包与拆包

因为 TCP 没有消息边界，应用写入的"消息"和接收方读出来的"块"之间没有任何对应关系：



图 3-2 字节流没有边界：接收方读出的块和发送方写入的消息没有对应关系，必须由应用层自己定义"帧"

第 2 章的 d1 · E1 实测就是这种情况：三次 send 的 44 字节被一次 recv 全部读出。粘包可能发生在发送端（Nagle 算法把小块合并成一个段）、也可能发生在接收端（多个段到达后还没被读走，在接收缓冲区里连成一片）。常见的应用层分帧方案有三种：

方案	做法	优点	缺点	例子
定长消息	每条消息固定 N 字节	最简单	浪费空间，不灵活	一些硬件 / 金融行情协议
分隔符	消息以特殊字符结尾	可读性好	消息体中出现分隔符要转义；需要逐字节扫描	HTTP/1.x 头部的 `\r\n\r\n`、Redis RESP
长度前缀（TLV）	头部写明长度，再跟消息体	高效、二进制安全，**最常用**	要防止恶意超大长度（需设上限）	HTTP/2 帧、游戏协议、Protobuf 分帧、Kafka

长度前缀分帧：处理"粘包 + 半包"的标准写法

```
// 接收缓冲 buf 里累积了从 socket 读到的字节；每次 recv 后调用一次
// 帧格式：[4 字节大端长度 len][len 字节消息体]
void OnBytes(std::vector<uint8_t>& buf, auto&& onMessage) {
    size_t off = 0;
    while (buf.size() - off ≥ 4) {                                    // 1) 头部够不够
        uint32_t len = (buf[off] << 24) | (buf[off+1] << 16) | (buf[off+2] << 8) | buf[off+3];
        if (len > kMaxMessage) throw std::runtime_error("bad frame");  // 2) 防御恶意长度
        if (buf.size() - off - 4 < len) break;                         // 3) 半包：消息体还没收
        onMessage(&buf[off + 4], len);                                 // 4) 一条完整消息
        off += 4 + len;                                                // 5) 可能还有下一条（粘
    }
    buf.erase(buf.begin(), buf.begin() + off);                         // 保留不完整的尾巴
}
```

⚠️ **易错点："UDP 会不会粘包？"**

不会。UDP 每个数据报都是独立的，内核按数据报为单位排队，recvfrom 一次只取一个。但 UDP 会**丢包、乱序**，以及缓冲区太小时**截断**。所以"粘包"是字节流协议特有的问题，面试里如果被问"怎么解决 TCP 粘包"，一定要先说明"这是因为 TCP 是字节流、没有消息边界"。

3.4 可靠传输之一：确认与重传

可靠传输的核心逻辑一句话就能说完：**发送方给数据编号并保留副本，接收方收到后回确认，发送方在一定时间内没有收到确认就重发。**难点全在细节：多久算"没收到"？怎么更早发现丢包？只丢了一个时能不能只重发一个？

超时重传：RTO 怎么定

重传超时时间 RTO（Retransmission TimeOut）太短会导致大量不必要的重传，太长则丢包后要干等很久。TCP 根据测得的往返时间 RTT 动态计算（RFC 6298）：

RFC 6298：RTO 计算

```
首次测得 RTT 样本 R:     SRTT = R           RTTVAR = R / 2
之后每个新样本 R':       RTTVAR = (1 − 1/4) * RTTVAR + 1/4 * |SRTT − R'|
                         SRTT   = (1 − 1/8) * SRTT   + 1/8 * R'
                         RTO    = SRTT + max(G, 4 * RTTVAR)        // G 为时钟粒度
约束：RTO 下限 1 s（RFC 建议；Linux 实际用 200 ms），上限 ≥ 60 s；首个样本之前 RTO = 1 s
超时后：RTO = RTO × 2（指数退避），直到收到新的确认后重新计算
```

- **为什么要加 4 × RTTVAR**：只用平均值的话，RTT 一抖动就会误判超时。方差项让 RTO 在网络抖动大时自动变宽。
- **Karn 算法**：重传过的段收到 ACK 时，**不能用来采样 RTT**——因为分不清这个 ACK 是对原始段还是对重传段的确认（重传歧义）。时间戳选项可以解决这个歧义；QUIC 则用"包号永不重复"从根本上消除了它（第 10 章）。
- **指数退避**：连续超时说明网络可能严重拥塞，每次把 RTO 翻倍，避免雪上加霜。代价是：连续丢几次后，RTO 会变成几秒甚至几十秒——这是游戏场景最痛恨的地方，也是 KCP 要改的第一件事（第 9 章）。

快速重传：不等超时

超时重传至少要等一个 RTO（Linux 至少 200 ms）。能不能更早发现丢包？可以利用累计确认的特性：当某个段丢失后，后续每个段到达时接收方都会回复同样的确认号（**重复 ACK**）。发送方**收到 3 个重复 ACK**，就认为该段丢了，立即重传，不等定时器。

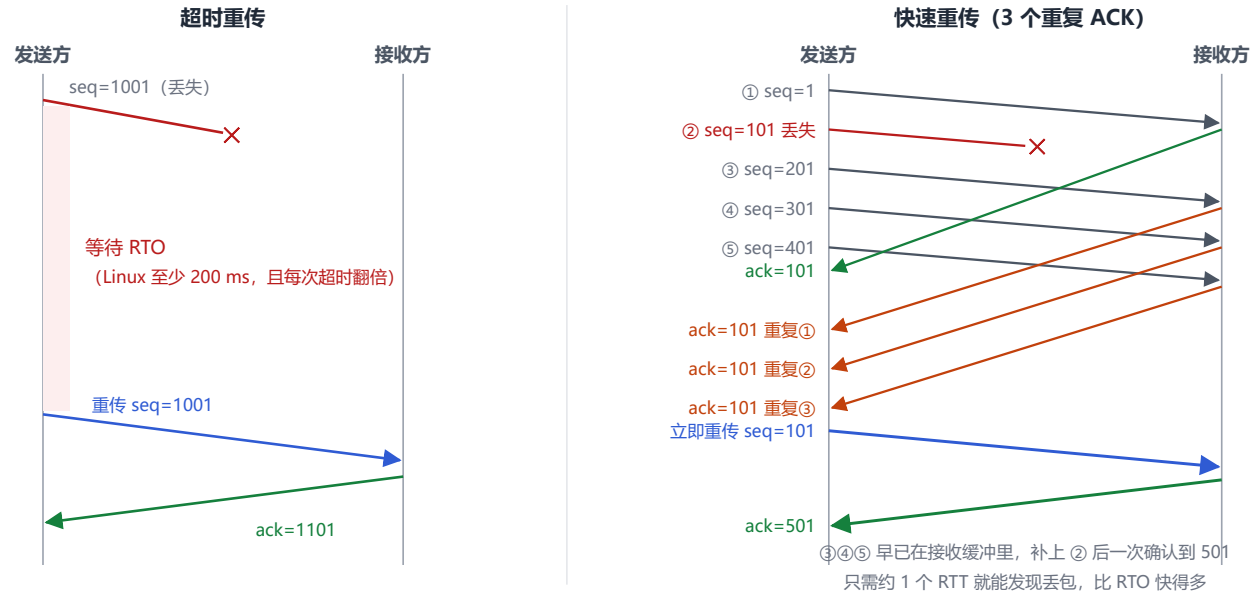


图 3-3 超时重传要干等一个 RTO；快速重传利用 3 个重复 ACK 提前发现丢包

> 🔬 **为什么是 3 个重复 ACK，而不是 1 个？**
>
> 因为网络存在轻微乱序：段 3 比段 2 先到，也会产生 1 个重复 ACK，但随后段 2 到达就恢复了。阈值设为 3，是在"尽早发现丢包"和"把乱序误判成丢包"之间的经验折中。现代 Linux 默认还启用了 **RACK**（基于时间而不是计数判断丢包，RFC 8985）和 **TLP 尾部丢失探测**（解决"最后几个段丢了，后面没有段来触发重复 ACK"的问题）。KCP 把这个阈值做成了可调参数 `resend`，游戏场景常设为 2（第 9 章）。

SACK：只重传真正丢了的

累计确认只能说"我连续收到哪里为止"。如果一个窗口里丢了多个不连续的段，发送方只靠重复 ACK 无法知道到底丢了哪几个，要么一个 RTT 只能修复一个洞，要么把后面全部重发（类似回退 N 步）。**SACK（选择确认，RFC 2018）**在 ACK 的选项中附带最多 3～4 个"已收到的不连续块"，例如 `ack=101, SACK=[201-501]`，发送方就能精确地只重传 101～200。Linux 默认开启 SACK（`net.ipv4.tcp_sack=1`）。

3.5 可靠传输之二：滑动窗口与流量控制

如果每发一个段都要等确认再发下一个（**停等协议**），信道利用率极低：RTT 100 ms、每次 1460 字节，吞吐只有 14.6 KB/s。解决办法是**允许连续发送多个段而不等确认**，这些"已发出未确认"加上"允许发出"的部分就构成了**发送窗口**。



图 3-4 滑动窗口：发送缓冲区被两个指针分成四段，窗口随确认向右滑动

流量控制：别把对方淹死

接收方的应用可能读得很慢（比如游戏服务器卡在一次数据库查询上），接收缓冲区就会逐渐被填满。TCP 让接收方在每个 ACK 的**窗口字段**里告诉发送方"我还剩多少空间"（rwnd），发送方保证在途数据不超过 rwnd。这就是**流量控制**：*端到端、由接收方驱动*，保护的是接收方。

- **零窗口**：rwnd = 0 时发送方停止发送，并启动**坚持定时器（persist timer）**，周期性发 1 字节的**零窗口探测**，防止"接收方窗口恢复的那个通告 ACK 丢了，双方永远互相等待"的死锁。
- **糊涂窗口综合征（Silly Window Syndrome）**：接收方每腾出几个字节就通告一次，发送方就发几个字节的小段，头部开销远大于数据。对策：接收方等腾出足够空间（一个 MSS 或缓冲区一半）再通告；发送方用 **Nagle 算法**。

Nagle 算法与延迟确认：一对"好心办坏事"的组合

Nagle 算法（发送方）

如果还有**已发送未确认**的数据，新产生的小数据（不足一个 MSS）先攒着，等收到 ACK 或攒满一个 MSS 再发。目的是减少网络上的小包数量。

延迟确认（接收方）

收到数据后不立即回 ACK，而是等一会儿（Linux 最少约 40 ms，Windows 默认 200 ms），希望能把 ACK 捎带在应答数据里，或者攒两个段一起确认。

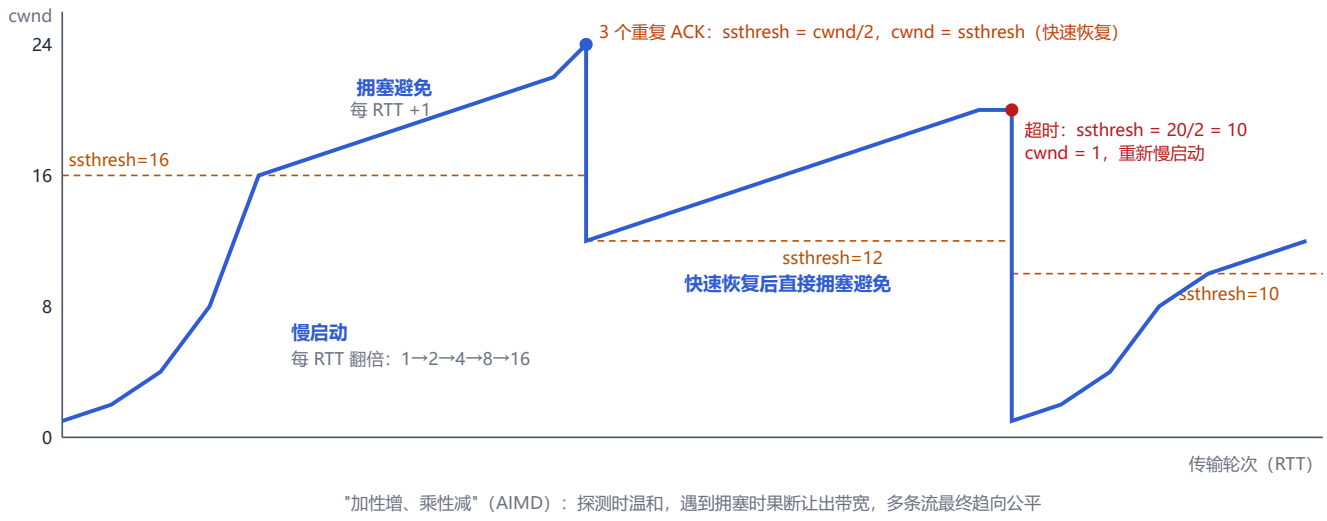
两者单独都合理，叠加起来却会出问题：客户端发了一个小请求的前半段，Nagle 要等 ACK 才发后半段；服务器收到前半段，延迟确认要等 40～200 ms 才回 ACK——于是每个请求都凭空多出几十到上百毫秒。

> 🎮 **游戏开发视角：TCP 游戏连接一定要开 TCP_NODELAY**
>
> 游戏的操作消息都很小、要求立即发出。用 TCP 承载时，务必 `setsockopt(fd, IPPROTO_TCP, TCP_NODELAY, &one, sizeof one)` 关闭 Nagle；Linux 上还可以用 `TCP_QUICKACK` 临时关闭延迟确认。Redis、gRPC、大多数 RPC 框架默认都会开启 TCP_NODELAY。

3.6 拥塞控制：别把网络挤爆

流量控制只关心接收方，但路径中间的路由器也有容量。如果所有发送方都只看 rwnd 全速发，路由器队列被打满，丢包、重传、更多丢包，网络吞吐会断崖式下跌——这在 1986 年真实发生过（"拥塞崩溃"，NSFNET 吞吐一度跌到原来的千分之一），之后才有了 Van Jacobson 的拥塞控制算法。

TCP 发送方维护一个**拥塞窗口 cwnd**，表示"我估计网络还能承受多少在途数据"，实际发送窗口取 **min(rwnd, cwnd)**。经典 Reno 算法有四个部分：



"加性增、乘性减"（AIMD）：探测时温和，遇到拥塞时果断让出带宽，多条流最终趋向公平

图 3-5 Reno 拥塞窗口变化示意（单位：MSS）：慢启动 → 拥塞避免 → 快速恢复 → 超时

阶段	触发条件	cwnd 怎么变	直觉
慢启动	连接开始 / 超时之后，cwnd < ssthresh	每收到一个 ACK，cwnd += 1 → **每 RTT 翻倍。**初始窗口现在通常为 10 个 MSS（RFC 6928）	"慢"是指起点低，增长其实是指数级的
拥塞避免	cwnd ≥ ssthresh	每 RTT cwnd += 1（线性增长）	接近上次出事的位置了，小心试探
快速重传 + 快速恢复	3 个重复 ACK	ssthresh = cwnd / 2；cwnd = ssthresh（+3）；然后进入拥塞避免	还能收到重复 ACK 说明网络还通，只是轻度拥塞
超时	RTO 到期	ssthresh = cwnd / 2；**cwnd = 1**；重新慢启动	连 ACK 都收不到了，说明拥塞严重

现代算法：CUBIC 与 BBR

CUBIC（Linux 默认，RFC 9438）

仍然以**丢包**为拥塞信号，但 cwnd 的增长按"距离上次丢包的时间"的三次函数变化：远离上次丢包点时增长快，接近时放缓并在附近长时间停留，丢包后再快速探测。增长和 RTT 无关，对长肥管道更公平、更高效。

BBR（Google，2016）

不再把丢包当拥塞信号，而是持续测量**瓶颈带宽 BtlBw**和**最小往返时延 RTprop**，按 BtlBw × RTprop 控制在途数据量、按 BtlBw 控制发送速率（pacing）。在有随机丢包的链路（无线、跨国）上吞吐远好于 CUBIC，也不会把路由器缓冲区塞满。

⚠️ **易错点：流量控制 vs 拥塞控制**

	流量控制	拥塞控制
保护谁	接收方（别把它的缓冲区淹了）	网络（别把路由器挤爆）
信息来源	接收方在 ACK 中**显式通告** rwnd	发送方根据丢包、RTT 等**自己推测** cwnd
作用范围	点对点	全局（所有共享瓶颈的流）
发送窗口	两者同时生效：发送窗口 = min(rwnd, cwnd)	

🎮 **游戏开发视角：基于丢包的拥塞控制在弱网下的副作用**

移动网络、Wi-Fi 上的丢包很多是**随机干扰**而不是拥塞，但 Reno / CUBIC 一律把它当成拥塞信号去减窗。对下载大文件来说只是速度慢一点；对每秒几十个小包的游戏来说，窗口被压到比实际需要还小，消息就会在发送缓冲里排队，延迟越积越高。第 8 章的仿真会量化这件事，第 9 章的 KCP 提供了关闭拥塞控制的开关 `nc=1`。

3.7 TCP 状态机总览

一条 TCP 连接在生命周期里会经历 11 种状态。第 5、6 章会逐一拆解，这里先给出全景图，后面两章随时可以翻回来对照。

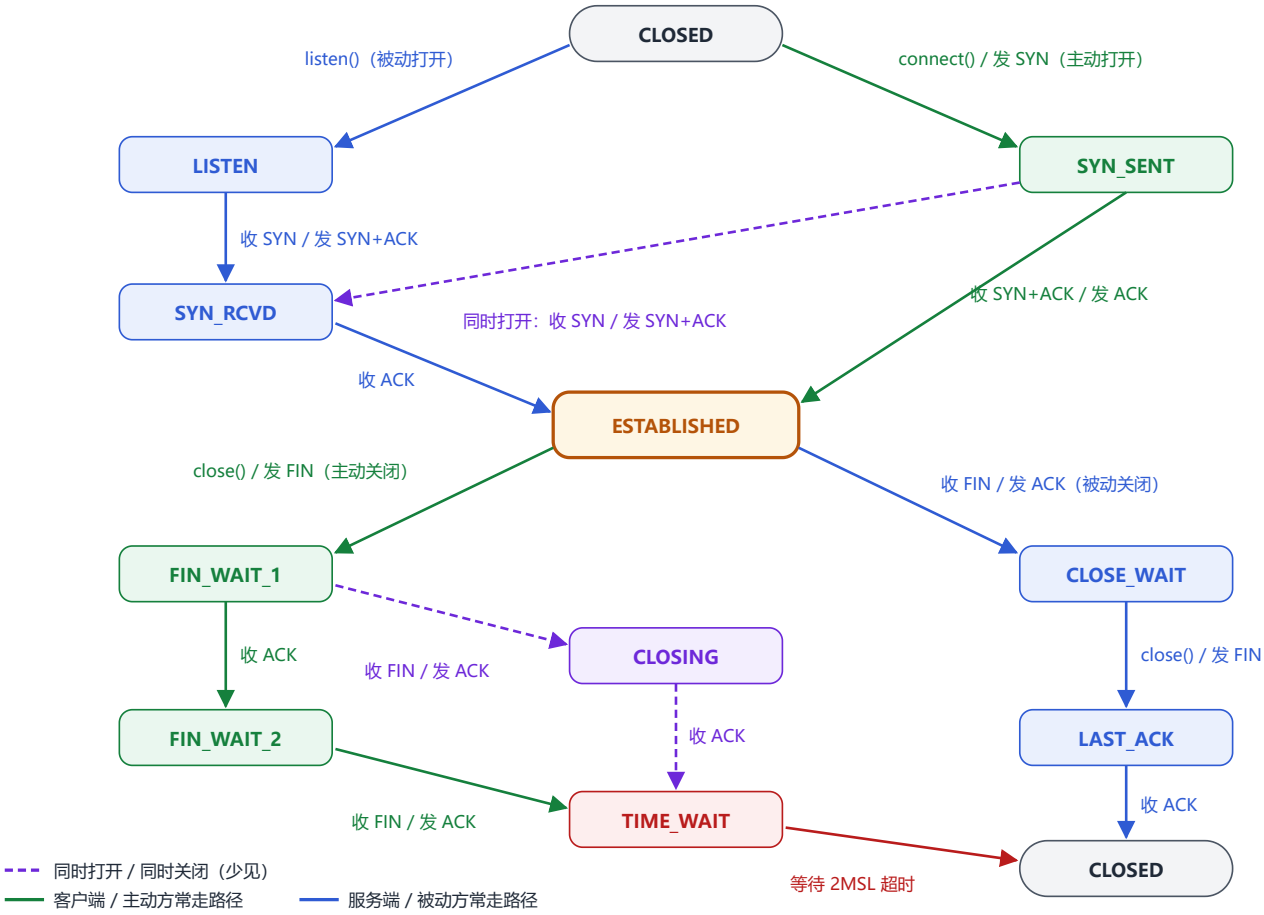


图 3-6 TCP 状态机（省略 RST、超时等异常转移）：上半部分是握手（第 5 章），下半部分是挥手（第 6 章）

✅ 面试答法：TCP 如何保证可靠传输?

按"问题 → 机制"答：① 数据出错：**校验和**，错了直接丢弃，当作丢包处理；② 丢包：**序号 + 确认 + 重传**，包括基于 RTT 动态计算 RTO 的超时重传，以及 3 个重复 ACK 触发的快速重传，配合 SACK 只重传缺失部分；③ 乱序和重复：接收方按**序号**在接收缓冲区里重排、丢弃重复段，只把连续数据交给应用；④ 接收方处理不过来：**滑动窗口流量控制**；⑤ 网络拥塞：**拥塞控制**（慢启动、拥塞避免、快速重传、快速恢复）；⑥ 双方状态同步：**三次握手和四次挥手**的连接管理。

本章小结

- TCP：面向连接、可靠、字节流、全双工。头部 20～60 字节；关键字段 seq / ack / 窗口 / 标志位。
- 序号按**字节**编号，ack 是"期望的下一个字节"（累计确认）；SYN、FIN 各占 1 个序号。
- 字节流没有边界 → 粘包 / 拆包；用**长度前缀**等应用层分帧解决；UDP 不存在粘包。

- RTO = SRTT + 4·RTTVAR，Karn 算法，超时指数退避；3 个重复 ACK 快速重传；SACK 精确重传。
- 发送窗口 = min(rwnd, cwnd)：rwnd 是**流量控制**（保护接收方），cwnd 是**拥塞控制**（保护网络）。
- Reno：慢启动（翻倍）→ 拥塞避免（+1）→ 3 dup ACK 减半 / 超时归 1；CUBIC 是 Linux 默认，BBR 基于带宽和时延建模。
- Nagle + 延迟确认会制造几十～几百毫秒延迟，交互式应用要开 `TCP_NODELAY`。

▶ **思考题 1：ISN = 5000，客户端依次发送 300 字节、200 字节两个段，服务器全部收到，最后的 ack 是多少?**

▶ **思考题 2：接收方通告窗口为 0 后，发送方怎么知道什么时候可以继续发？如果通告恢复的那个 ACK 丢了呢?**

▶ **思考题 3：为什么说"TCP 的拥塞控制在无线网络上表现不好"？有哪些改进?**

▶ **思考题 4：发送窗口是不是越大越好?**

CHAPTER 04

TCP vs UDP：全面对比与场景选型

不是"谁更好"，而是"你的数据最怕什么"：怕丢，还是怕晚？

本章目标

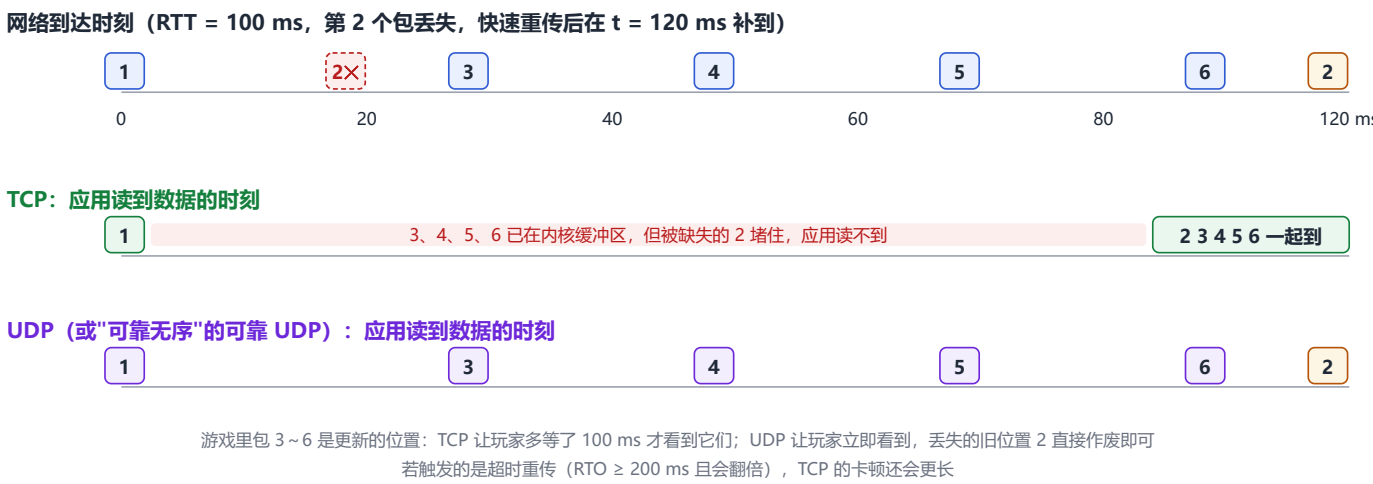
- 能从 12 个维度对比 TCP 与 UDP，并说出每个差异背后的原因
- 理解"按序可靠交付"带来的队头阻塞，这是两者在实时场景下差异的根源
- 能针对网页、视频、在线游戏给出协议选型并说出理由
- 应对"TCP 一定比 UDP 慢吗"等高频追问

4.1 全维度对比

维度	TCP	UDP	根本原因
连接	面向连接，三次握手 / 四次挥手	无连接	TCP 需要双方同步序号、窗口等状态
可靠性	不丢、不重、不错	尽力交付	TCP 有确认 + 重传；UDP 没有
有序性	严格按序交付	按到达顺序交付	TCP 接收缓冲区按序号重排
数据单位	字节流，无边界（会粘包）	数据报，保留边界	TCP 为了合并 / 拆分以提高效率
头部开销	20 ~ 60 字节	8 字节	TCP 要携带序号、确认号、窗口、选项
流量控制	有（滑动窗口 rwnd）	无	—
拥塞控制	有（慢启动、CUBIC、BBR 等）	无	—
通信模式	仅一对一	一对一、一对多、多对多（广播 / 多播）	连接是两端之间的状态
首包时延	至少 1 RTT 握手后才能发数据（TFO 除外）	0 RTT，第一个包即业务数据	握手
丢包时的时延	后续数据被阻塞，直到重传成功（队头阻塞）	不受影响，后续数据照常交付	按序交付的代价
服务器资源	每连接一份内核状态和缓冲区	一个 socket 服务所有客户端	连接状态
典型应用	HTTP/1.1、HTTP/2、文件传输、邮件、SSH、数据库	DNS、DHCP、音视频通话、直播、游戏、QUIC	业务"怕丢"还是"怕晚"

4.2 根源：队头阻塞

上表中最影响实时应用的一行是"丢包时的时延"。TCP 承诺**按序交付**：只要第 2 个段没到，就算第 3～6 个段早已躺在接收缓冲区里，应用也一个字节都读不到。这叫**队头阻塞（Head-of-Line Blocking，HOL）**。



游戏里包 3～6 是更新的位置：TCP 让玩家多等了 100 ms 才看到它们；UDP 让玩家立即看到，丢失的旧位置 2 直接作废即可
若触发的是超时重传（RTO ≥ 200 ms 且会翻倍），TCP 的卡顿还会更长

图 4-1 队头阻塞：TCP 的"按序交付"让一个丢包拖住它后面所有的数据

> 📦 **场景案例：HTTP/2 的"多路复用"为什么还会卡**
>
> HTTP/1.1 一条 TCP 连接同一时间只能处理一个请求，浏览器只好对同一域名开 6 条连接。HTTP/2 在**一条 TCP 连接**里同时传多个请求（多个流），解决了*应用层*的队头阻塞。可是这些流共享同一个 TCP 字节流：只要丢一个 TCP 段，**所有流**都被卡住——这是*传输层*的队头阻塞。在丢包率 2% 的弱网下，HTTP/2 甚至可能比开 6 条连接的 HTTP/1.1 更慢。HTTP/3 把传输层换成 QUIC，让每个流独立排序，才真正解决了这个问题（第 10 章）。

4.3 三个场景的选型分析

场景一：网页与 API

网页、REST API、文件下载的数据**一个字节都不能错**，晚几十毫秒用户感知不强，所以长期以来都用 TCP（HTTP/1.1、HTTP/2 over TLS over TCP）。变化发生在移动互联网时代：弱网、高 RTT、频繁切换网络让"握手太多、队头阻塞、换网断连"成了主要矛盾，于是出现了 **HTTP/3 over QUIC over UDP**。截至目前，主流浏览器和大型 CDN 都已支持 HTTP/3，但 TCP 依然是服务端内部通信的绝对主力。

场景二：视频

子场景	对延迟的要求	常用传输	理由
点播（看剧、短视频）	可以缓冲几秒到几十秒	HTTP（TCP 或 QUIC）分片下载：HLS / DASH	有缓冲区兜底，可靠更重要；可以借用 CDN
普通直播	2 ~ 10 秒	RTMP 推流（TCP）+ HLS / FLV 拉流	生态成熟，延迟可接受
低延迟直播 / 连麦	< 1 秒	WebRTC、SRT（UDP）	TCP 重传造成的卡顿不可接受
视频会议 / 语音通话	< 200 ~ 300 ms 端到端	RTP / SRTP over UDP（WebRTC）	迟到的帧没有意义；用 FEC、NACK、抖动缓冲、丢帧插值替代重传

规律很清楚：**延迟预算越小，越倾向 UDP + 应用层定制的可靠性。**视频会议的 NACK 机制其实就是"有选择的重传"：只有当 RTT 足够小、重传的包还来得及在播放前到达时才请求重传，否则直接放弃——这是 TCP 做不到的"可靠性按需"。

场景三：在线游戏

游戏类型	典型延迟要求	常用方案
回合制、卡牌、棋牌	数百毫秒都可接受	TCP（或 WebSocket）足够，开发简单
MMORPG 非战斗部分	100 ~ 300 ms	TCP 为主，部分项目战斗走 UDP
MOBA、FPS、格斗、竞速	每帧都敏感（几十毫秒）	UDP + 应用层可靠：KCP、ENet、引擎自带网络层（UE 的 NetDriver、Unity Netcode 等）

🎮 游戏开发视角：游戏数据要拆成三类

一个实时对战游戏里，数据对可靠性的需求截然不同：

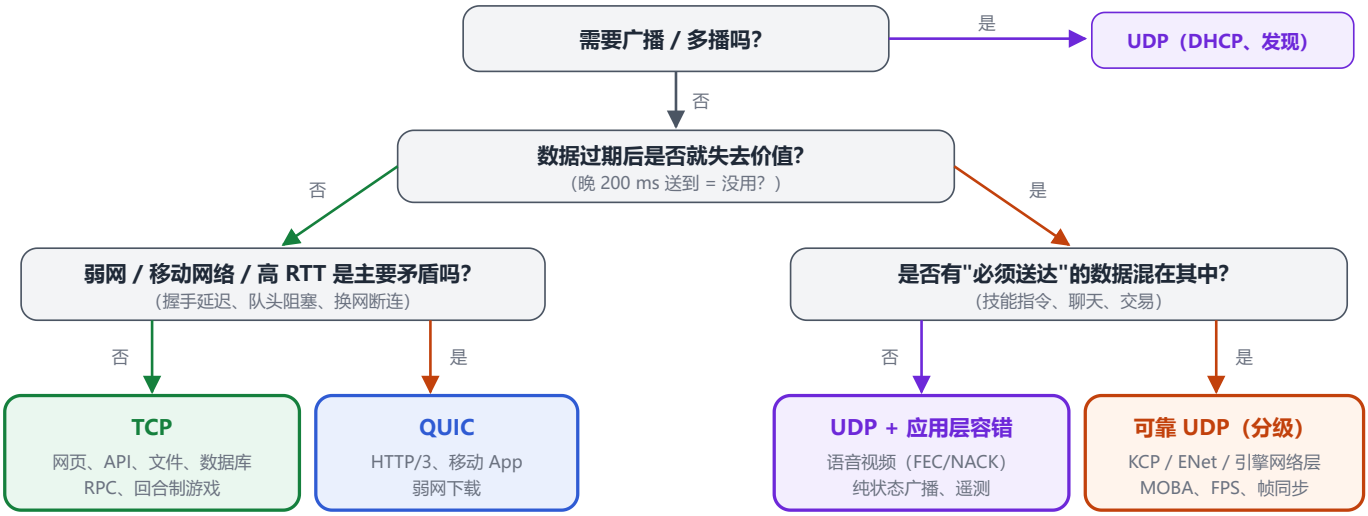
① **状态类**（位置、朝向、血量快照）：只要最新值，旧的丢了就丢了 → *不可靠、只取最新*；

② **事件类**（释放技能、购买装备、聊天）：一个都不能丢 → *可靠*，但不同事件之间未必需要严格有序；

③ **输入流**（帧同步下每帧的操作）：每帧都需要，且必须按帧序执行 → *可靠有序*，常用"每个包冗余携带最近几帧"的方式降低重传依赖。

TCP 只能对所有数据一视同仁地"可靠有序"，所以①会被②③的丢包重传拖累。这就是游戏需要可靠 UDP 的根本原因，第 7 章会系统展开。

4.4 选型决策树



现实工程中还要考虑：团队对自研协议的维护能力、运营商对 UDP 的 QoS 限速、企业防火墙是否放行 UDP（需要 TCP 兜底通道）

很多游戏的做法是"UDP 优先，探测失败时自动降级到 TCP"

图 4-2 传输协议选型决策树：核心判据是"数据是否怕晚"以及"是否混有必须送达的数据"

4.5 高频追问

Q1：TCP 一定比 UDP 慢吗?

不一定，要区分**吞吐**和**时延**。

吞吐：大块数据传输时 TCP 往往**更快**。内核对 TCP 有大量优化（TSO/GSO 分段卸载、GRO 合

并、零拷贝 sendfile），而 UDP 每个数据报都要走一遍系统调用和协议栈，高包率下 CPU 开销更大（需要 `sendmmsg/recvmmsg`、UDP GSO 等手段弥补）。这也是 QUIC 服务端 CPU 开销通常高于 TCP+TLS 的原因之一。
时延：网络良好时两者几乎一样（都是一个 RTT/2 的传播时延）；**差别出现在丢包和建连**：TCP 需要 1 RTT 握手，且丢包时有队头阻塞和 RTO 退避。

Q2：同一个端口号能同时被 TCP 和 UDP 绑定吗?

可以。TCP 和 UDP 是两个独立的端口空间，内核按 IP 头的协议字段先区分 TCP/UDP，再查各自的端口表。DNS 服务器就同时监听 TCP 53 和 UDP 53。

Q3：既然 UDP 不可靠，QUIC 为什么还要基于 UDP，而不是设计一个新的 IP 协议号?

因为**中间设备僵化（ossification）**：互联网上大量 NAT、防火墙只认识 TCP 和 UDP，一个新的协议号（比如 SCTP 的 132）基本无法穿越公网。同时，TCP 实现在操作系统内核里，升级要等系统更新，周期以年计。基于 UDP 就能穿越现有网络，在用户态快速迭代。UDP 在这里只充当"能穿越网络的最薄封装"。

Q4：用 TCP 做实时游戏会有什么问题？如果必须用 TCP 怎么优化?

问题：① 队头阻塞，一个丢包卡住后面所有新状态；② RTO 指数退避，连续丢包时卡顿可达秒级；③ Nagle + 延迟确认带来额外几十～几百毫秒；④ 拥塞控制把弱网随机丢包当拥塞，降低发送速率。优化：开 `TCP_NODELAY`；减小发送缓冲区避免数据在内核堆积（过时的状态宁可在应用层丢弃）；应用层心跳快速检测断线；状态同步只发最新值并做合并；必要时开多条连接隔离不同优先级的数据。

Q5：UDP 包丢了，发送方知道吗?

不知道。UDP 没有确认机制，唯一可能的反馈是 ICMP（如端口不可达），但它不可靠、可能被防火墙过滤，也不对应"包在中途丢失"的情况。需要知道就要在应用层实现 ACK。

✅ 面试答法：TCP 和 UDP 的区别（30 秒版）

TCP 面向连接、可靠、有序、基于字节流，有流量控制和拥塞控制，头部 20 字节起，只支持一对一，适合要求数据完整的场景，比如网页、文件传输。UDP 无连接、不保证可靠和顺序、面向数据报、保留消息边界，头部只有 8 字节，没有流控和拥塞控制，支持广播多播，延迟低开销小，适合实时性要求高、能容忍丢包的场景，比如音视频、游戏、DNS。 *（追加一句体现深度）*两者实时性差异的根源是 TCP 的按序可靠交付带来的队头阻塞，所以 QUIC、KCP 这类协议在 UDP 之上按需实现可靠性。

本章小结

- 选型的核心问题：数据**怕丢**还是**怕晚**；是否混有必须送达的数据；是否需要广播多播。
- 实时场景中 TCP 的根本劣势是**队头阻塞**：按序交付让一个丢包拖住后面所有数据。
- HTTP/2 解决了应用层队头阻塞，却仍受 TCP 层队头阻塞影响 → HTTP/3 改用 QUIC。
- 视频：点播 TCP、普通直播 TCP、低延迟直播与通话 UDP；游戏：回合制 TCP、实时对战 UDP + 分级可靠。
- TCP 并不一定慢：大流量吞吐往往更优；差异主要在建连与丢包时的时延。

▶ 思考题 1：直播推流常用 RTMP（基于 TCP），而视频会议用 UDP。同样是实时视频，为什么选择不同?

▶ 思考题 2：一个 MOBA 游戏如果把"位置同步"和"技能释放"放进同一条可靠有序的通道，会有什么问题?

▶ 思考题 3：为什么说"UDP 的无拥塞控制"既是优点也是社会责任问题?

PART THREE

第三部分　连接管理：握手与挥手

面试出现频率最高的两道题。不仅要背出流程，更要讲清每一步"为什么"，以及每一步出了问题会怎样。

CHAPTER 05

三次握手：建立一条连接需要什么

握手的本质不是"打招呼"，而是让双方就各自的初始序号达成一致，并确认双向通路都是通的。

本章目标

- 画出三次握手的完整时序：每一步的标志位、seq、ack 以及双方状态变化
- 说清 ISN 为什么要随机，以及"为什么是三次而不是两次或四次"的三个理由
- 理解内核中的半连接队列与全连接队列，以及它们满了之后的行为
- 讲清 SYN Flood 攻击原理和 SYN Cookie 等防御手段
- 逐一分析握手中每个报文丢失、端口未监听、队列溢出等异常

5.1 握手要完成的四件事

1. **确认双方的收发能力**：客户端能发、服务器能收；服务器能发、客户端能收。
2. **同步双方的初始序号（ISN）**：TCP 是全双工的，两个方向各有一套序号，*双方都要把自己的 ISN 告诉对方，并得到对方的确认*。这是最核心的一件事，SYN 就是 Synchronize Sequence Numbers 的缩写。
3. **协商连接参数**：MSS、窗口扩大因子、是否支持 SACK 和时间戳。这些选项只能出现在 SYN 报文中。
4. **分配连接资源**：双方各自建立 TCB（传输控制块）、收发缓冲区。

5.2 逐步拆解

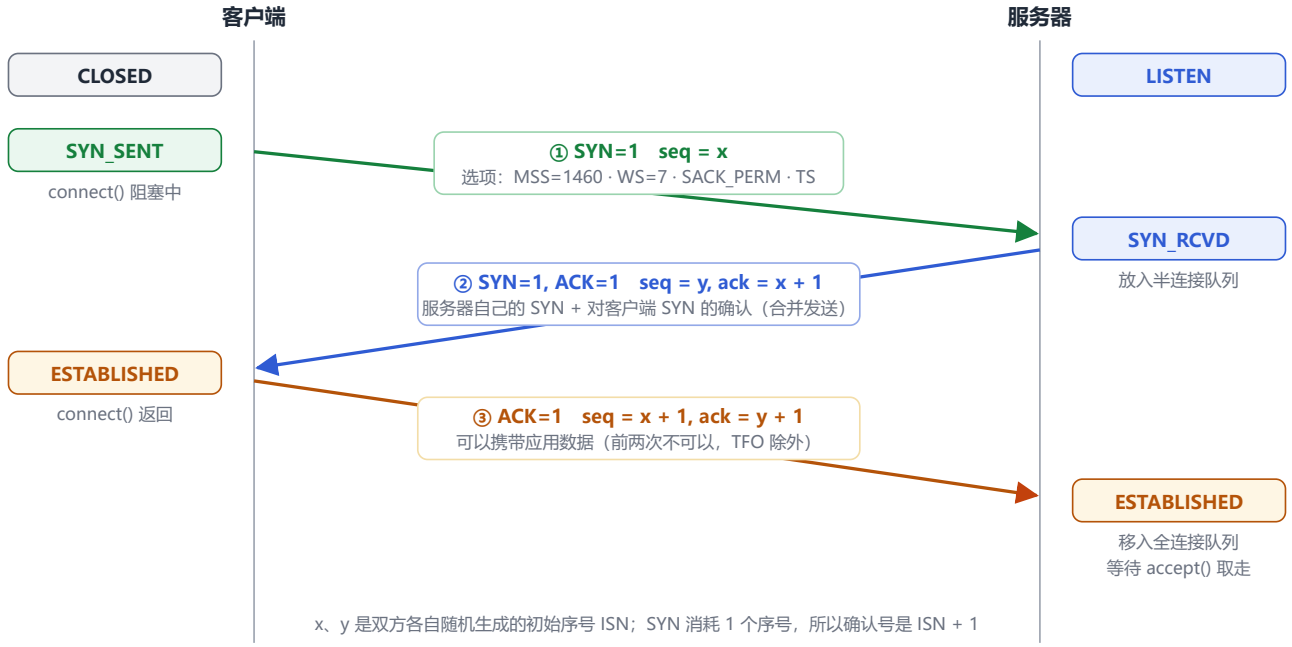


图 5-1 三次握手：两个 SYN 各自同步一个方向的序号，两个 ACK 各自确认对方的 SYN，中间两者合并成一个报文

步骤	方向	标志位	seq / ack	状态变化	这一步证明了什么
①	C → S	SYN	seq = x	C：CLOSED → SYN_SENT S（收到后）：LISTEN → SYN_RCVD	服务器知道：客户端能发、自己能收
②	S → C	SYN + ACK	seq = y，ack = x+1	C（收到后）：SYN_SENT → ESTABLISHED	客户端知道：双方都能收发；自己的 ISN 已被确认
③	C → S	ACK	seq = x+1，ack = y+1	S（收到后）：SYN_RCVD → ESTABLISHED	服务器知道：客户端能收；自己的 ISN 已被确认

`tcpdump` 中的一次握手（典型输出，地址与序号为示例）

```
10:00:00.000 IP 10.0.0.1.52344 > 10.0.0.2.443: Flags [S],  seq 1705322911, win 64240, options [mss 1460,sackOK,TS val 1 ecr 0,nop,wscale 7], length 0
10:00:00.030 IP 10.0.0.2.443 > 10.0.0.1.52344: Flags [S.], seq 3891201650, ack 1705322912, win 65160, options [mss 1460,sackOK,TS val 9 ecr 1,nop,wscale 7], length 0
10:00:00.060 IP 10.0.0.1.52344 > 10.0.0.2.443: Flags [.],  ack 3891201651, win 502, length 0
```

tcpdump 中 `S` 表示 SYN，`.` 表示 ACK，`S.` 即 SYN+ACK；第三行的 ack 默认显示为相对值时会是 1。win 502 是窗口扩大前的原始值，实际窗口 = 502 × 2^7 = 64256 字节。

> ⚠️ **易错点**
>
> ① **ack 不是"收到的序号"，而是"期望的下一个序号"**，所以是 x+1 而不是 x。
>
> ② **SYN 不携带数据，却消耗一个序号**（FIN 也一样）；纯 ACK 不消耗序号，所以第三次握手后客户端发的第一个数据段 seq 仍是 x+1。
>
> ③ 服务端的 `accept()` **并不参与握手**：握手完全由内核完成，accept 只是从全连接队列里取出一个已完成握手的连接。即使应用从不调用 accept，客户端的 connect 也能成功返回。
>
> ④ "SYN_RCVD"在一些资料里写作 "SYN_RECV"（Linux `ss` 的显示），两者是同一个状态。

5.3 ISN 为什么要随机

如果每条连接的 ISN 都从 0 开始，会有两个问题：

- **与旧连接的报文混淆**：同一个四元组的连接关闭后又马上重建，旧连接在网络中滞留的报文可能恰好落在新连接的序号窗口里，被当成新数据接收。
- **序号预测攻击**：攻击者如果能猜到 ISN，就能伪造源地址完成握手、向连接中注入数据（1994 年著名的 Mitnick 攻击就是利用了当时可预测的 ISN）。

RFC 6528 推荐的生成方式是 `ISN = M + F(本地IP, 本地端口, 远端IP, 远端端口, 密钥)`：M 是一个大约每 4 微秒加 1 的计时器（32 位约 4.55 小时回绕一次），F 是带密钥的哈希函数。这样**同一四元组的 ISN 随时间单调递增**（避免和旧连接重叠），而**外人无法预测**（因为有密钥）。Linux 的 `secure_tcp_seq()` 就是按这个思路实现的。

5.4 为什么是三次，而不是两次或四次

> 📞 **先看类比：打电话**
>
> A："喂，听得到吗？"（A 确认自己能说）
>
> B："听得到，你听得到我吗？"（B 确认自己能听、也能说）
>
> A："听得到。"（B 这才确认 A 能听）
>
> 少了第三句，B 无法知道自己的声音 A 是否听见了。但类比只能解释"双向确认"，面试真正要的是下面三条技术理由。

理由一（RFC 793 给出的首要原因）：阻止历史连接

网络可能把一个很久以前发出的 SYN 延迟送达。例如客户端先发了 SYN(seq=90)，迟迟没回应就放弃了（或者客户端重启了），接着又发了新的 SYN(seq=100)。如果旧 SYN 先到：

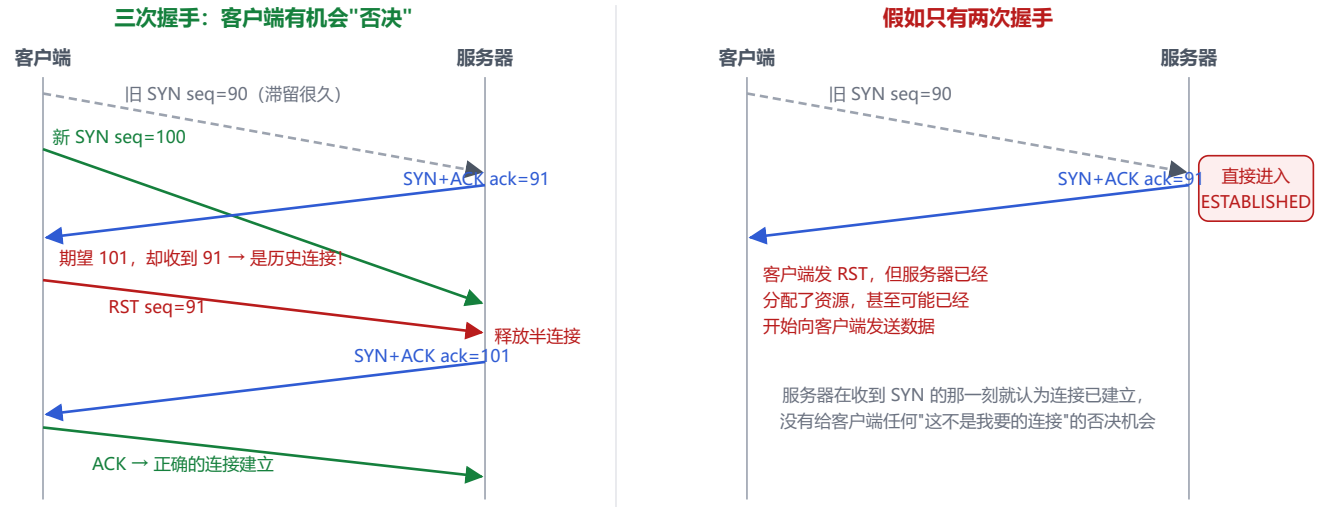


图 5-2 历史连接问题：三次握手让发起方在第三步确认"这是不是我当前想要的连接"

关键在于：**只有发起方（客户端）才知道哪个 SYN 是"当前有效"的。**三次握手把"最终拍板"的权力留给了客户端：服务器回 SYN+ACK 之后先不建立连接，由客户端根据确认号判断，是当前连接就回 ACK，是历史连接就回 RST。

理由二：可靠地同步双方的初始序号

双方都有自己的 ISN，**每个 ISN 都需要"告知 + 确认"一来一回**，理论上需要 4 个报文：C 发 SYN(x)、S 确认 x、S 发 SYN(y)、C 确认 y。中间两个都是服务器发给客户端的，可以合并成一个 SYN+ACK，所以是 3 个。两次握手只能保证客户端的 ISN 被确认，服务器的 ISN 发出去后不知道对方有没有收到。

理由三：避免重复连接浪费资源

如果只有两次握手，客户端因为超时重传了多个 SYN，服务器每收到一个 SYN 就建立一条连接、分配一次资源，而客户端只会使用其中一条，其余全部成为空占资源的"僵尸连接"。

为什么不是四次?

四次也能工作（同时打开时其实就是四个报文），只是没必要：服务器的"确认"和"自己的 SYN"可以放在同一个报文里发出，合并后三次已经足以完成全部目标。**三次是满足可靠同步的最少次数。**

> ✅ **面试答法：为什么是三次握手?**
>
> 三次握手的目的是同步双方的初始序号并确认双向通路。三次是最少次数：① 主要原因是**防止历史连接**，旧的 SYN 到达服务器后，客户端可以在第三步通过 RST 否决，两次握手做不到；② 双方的 ISN 都需要被对方确认，两次只能确认客户端的；③ 两次握手下，重传的重复 SYN 会让服务器建立多条无用连接。不需要四次，因为服务器的 ACK 和 SYN 可以合并在一个报文里。

5.5 内核视角：半连接队列与全连接队列



图 5-3 Linux 中一个监听 socket 维护两个队列：握手进行中的放在半连接队列，握手完成的放在全连接队列等 accept

队列满了	Linux 的行为	如何观察 / 调整
半连接队列满	默认开启 SYN Cookie（`tcp_syncookies=1`）时改用 Cookie 继续服务；否则丢弃新的 SYN	`netstat -s | grep "SYNs to LISTEN"`；调大 `tcp_max_syn_backlog`
全连接队列满	`tcp_abort_on_overflow=0`（默认）：**丢弃第三次握手的 ACK**，服务器停留在 SYN_RCVD 并重传 SYN+ACK，期望应用稍后腾出空间；`=1`：直接回 RST	`ss -lnt` 中监听 socket 的 Recv-Q（当前积压）/ Send-Q（队列上限）；`netstat -s | grep overflowed`；调大 `listen` 的 backlog 和 `net.core.somaxconn`（Linux 5.4 起默认 4096，之前为 128）

> ⚠️ **易错点：客户端 connect 成功了，服务端却"没有这条连接"**
>
> 全连接队列满且 `tcp_abort_on_overflow=0` 时，客户端收到 SYN+ACK 就进入了 ESTABLISHED，connect 返回成功，开始发送请求；而服务端把第三次握手的 ACK 丢了，连接还在半连接队列里。客户端的请求数据会被服务端丢弃或者等待，表现为"连接成功但请求超时"。线上排查"偶发连接超时"时，监听 socket 的 Recv-Q 是第一个要看的指标。

5.6 SYN Flood 攻击与防御

原理

攻击者伪造大量不存在的源 IP 发送 SYN。服务器为每个 SYN 在半连接队列中分配一个条目并回复 SYN+ACK，但这些 SYN+ACK 发往不存在的地址，永远收不到第三次握手的 ACK。每个半连接要等 SYN+ACK 重传若干次（Linux 默认 `tcp_synack_retries=5`，总计约 63 秒）才会被清理。只要攻击者发送 SYN 的速度超过清理速度，半连接队列很快就会被占满，**正常用户的 SYN 被丢弃，服务不可用**。它的厉害之处在于：攻击者几乎没有成本（不需要维持任何状态），却能耗尽服务器的资源——这是"**状态不对称**"导致的。

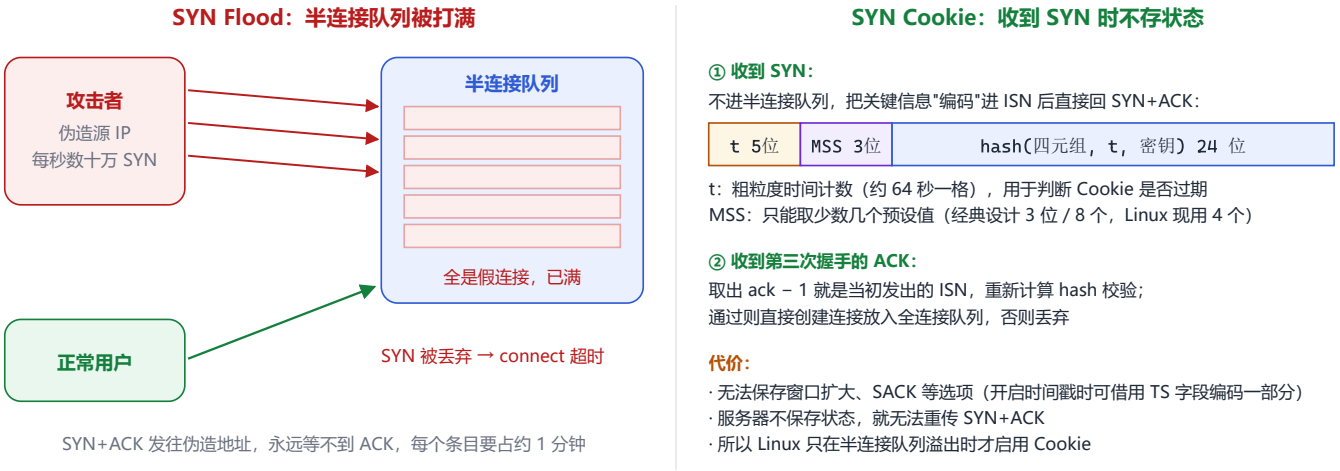


图 5-4 左：SYN Flood 利用"攻击者无状态、服务器有状态"的不对称性；右：SYN Cookie 把状态编码进 ISN，让服务器在握手完成前也不存状态

防御手段

手段	做法	作用
SYN Cookie	`net.ipv4.tcp_syncookies = 1`（Linux 默认）	根治"半连接队列被打满"，代价见上图
增大半连接队列	调大 `tcp_max_syn_backlog`，同时调大 `somaxconn` 与 backlog	提高承受上限
减少 SYN+ACK 重传次数	调小 `tcp_synack_retries`（如 2）	半连接更快被清理，代价是正常的弱网用户更容易握手失败
SYN Proxy / 防火墙	由前置设备（高防、负载均衡、iptables SYNPROXY）代替服务器完成握手，验证通过后再与后端建连	把攻击挡在服务器之外
流量清洗与限速	运营商 / 云厂商的 DDoS 清洗；按源地址限速	应对超大流量攻击

🎮 延伸：这件事对可靠 UDP 有什么启示

UDP 服务器没有半连接队列，看上去不怕 SYN Flood，但自己实现的"连接握手"如果在收到第一个包时就分配会话资源，就会重蹈覆辙；如果对一个很小的请求回复很大的应答，还会被利用来做**反射放大攻击**。QUIC 的应对方式值得借鉴：服务器在验证客户端地址之前，发送的数据量不得超过收到的 **3 倍**，并且可以发送 **Retry 令牌**（本质就是 SYN Cookie 的思想）要求客户端证明自己真的拥有这个地址（第 10 章）。

5.7 握手过程中的异常情况

异常	客户端表现	服务器表现	最终结果
① SYN 丢失	停留在 SYN_SENT；按 1、2、4、8、16、32 s 指数退避重传 SYN，默认 `tcp_syn_retries=6`	毫不知情	重传全部失败（共约 127 s）后 connect 返回 `ETIMEDOUT`
② SYN+ACK 丢失	以为 SYN 丢了，**重传 SYN**	停留在 SYN_RCVD，**重传 SYN+ACK**（默认 `tcp_synack_retries=5`）	双方都在重传，任何一个成功即可继续；都失败则客户端超时、服务器清理半连接
③ ACK 丢失	已进入 ESTABLISHED，connect 已返回	停留在 SYN_RCVD，重传 SYN+ACK；客户端收到后会再回 ACK	若客户端此时发送数据，**数据段本身带有 ACK 标志和正确的确认号，服务器收到后同样会进入 ESTABLISHED**；若客户端一直不发数据且服务器重传耗尽，服务器删除半连接，客户端之后发数据会收到 RST
端口无人监听	收到 RST → `ECONNREFUSED`	内核直接回 RST	连接失败（Windows 会重试 SYN，见下方实测）
主机不可达 / 被防火墙丢弃	收不到任何回应，同"SYN 丢失"；若收到 ICMP 不可达则返回 `EHOSTUNREACH`	—	超时或立即失败
全连接队列满	connect 成功，但请求无响应	丢弃第三次 ACK，重传 SYN+ACK	见 5.5 节
已建立的连接又收到一个 SYN（如客户端重启后恰好用了相同端口）	收到一个"挑战 ACK"，发现确认号不对，回 RST	按 RFC 5961 回复 **挑战 ACK**（而不是直接重置，防止伪造 SYN 攻击）	旧连接被 RST 清理，客户端重试后建立新连接

同时打开（双方同时向对方发 SYN）	双方都 SYN_SENT → 收到对方 SYN → SYN_RCVD → 发 SYN+ACK → 收到后 ESTABLISHED	4 个报文建立一条连接（P2P 打洞时可能出现）

🧪 实测证据（d1 • E3）：connect 到没人监听的端口

```
d1_tcp_udp_behaviors.exe · E3
```

```
== E3 TCP connect 到没人监听的端口 ==
  connect 返回 -1，WSAGetLastError=10061（10061=WSAECONNREFUSED，对端回了 RST），耗时 2048 ms
```

同一台机器的回环地址，RTT 不到 0.1 ms，可 connect 却花了整整 2 秒才失败。这是 Windows 的特有行为：收到 RST 后它不会立即放弃，而是认为"对方可能刚好在重启"，再重试几次 SYN。Linux 收到 RST 会立即返回 `ECONNREFUSED`（通常不到 1 ms）。所以在 Windows 上做"端口是否开放"的探测或者"服务未启动时快速失败重连"，不要指望 connect 很快失败，应改用非阻塞 connect + 自己控制超时。

5.8 握手优化：TCP Fast Open

普通握手中，客户端最早要在第三步才能发送数据，HTTP 请求至少要等 1 个 RTT。**TCP Fast Open（TFO，RFC 7413）**允许在 SYN 里就携带数据：

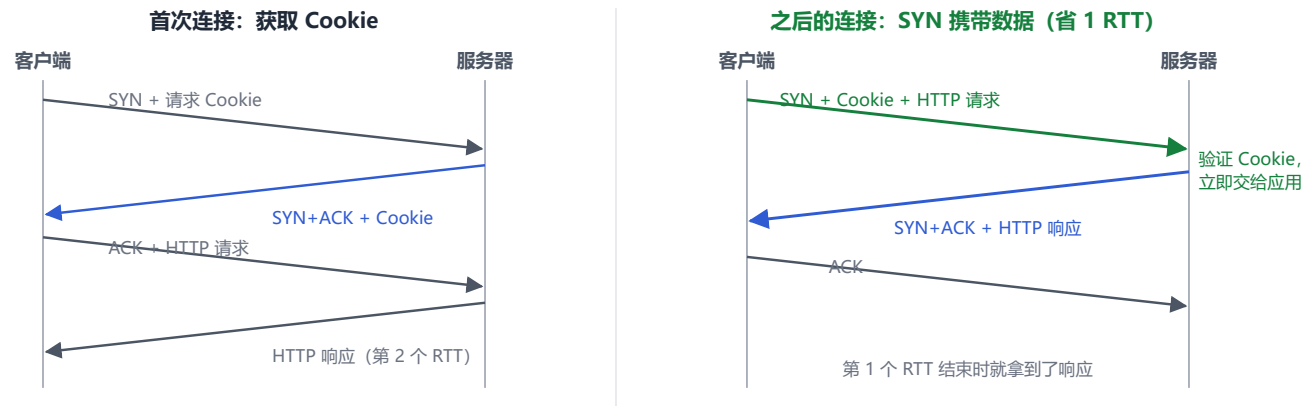


图 5-5 TCP Fast Open：首次连接拿到服务器签发的 Cookie，之后 SYN 即可携带数据

TFO 的局限：① SYN 中的数据可能被**重放**（攻击者截获后重发），所以只适合幂等请求；② 不少中间设备会丢弃带数据的 SYN 或去掉未知选项；③ 需要客户端和服务器双方都支持（Linux 用

`net.ipv4.tcp_fastopen` 开启）。实际部署并不广泛，"省握手 RTT"这个目标后来主要由 TLS 1.3 的 0-RTT 和 QUIC 实现。

本章小结

- 三次握手：**SYN(seq=x) → SYN+ACK(seq=y, ack=x+1) → ACK(ack=y+1)**；状态 C：SYN_SENT → ESTABLISHED，S：LISTEN → SYN_RCVD → ESTABLISHED。
- SYN 不带数据但占 1 个序号；第三次握手可以携带数据；accept 不参与握手。
- ISN 随机且随时间递增：防止与旧连接混淆、防止序号预测攻击（RFC 6528）。
- 为什么三次：**防历史连接**（首要）、同步双方 ISN、避免重复连接；ACK 与 SYN 合并所以不需要四次。
- 半连接队列（tcp_max_syn_backlog）/ 全连接队列（min(backlog, somaxconn)）；全连接队列满默认丢第三次 ACK。
- SYN Flood 利用状态不对称耗尽半连接队列；SYN Cookie 把状态编码进 ISN；另有调参、SYN Proxy、清洗。
- 异常：SYN 丢 → 客户端退避重传；SYN+ACK 丢 → 双方都重传；ACK 丢 → 服务端重传，客户端的数据段可代替 ACK。

▶ **思考题 1：第三次握手的 ACK 丢了，客户端紧接着发送了数据，会发生什么？**

▶ **思考题 2：服务器 listen(fd, 1024)，但 net.core.somaxconn = 128，全连接队列多大？大量短连接突发时会怎样？**

▶ **思考题 3：为什么 SYN Cookie 默认只在半连接队列满时才启用，而不是一直开着？**

▶ **思考题 4：客户端 connect 一个公网服务器，30 秒都没有返回，可能是什么原因？**

CHAPTER 06

四次挥手：一条连接如何体面地结束

常被口误为"四次握手"。它之所以是四次，是因为 TCP 是全双工的：两个方向要各自说一次"我说完了"。

本章目标

- 画出四次挥手的时序与双方状态，理解"半关闭"
- 说清为什么是四次、什么时候会变成三次
- 彻底讲透 TIME_WAIT：为什么存在、为什么是 2MSL、过多怎么办
- 会排查 CLOSE_WAIT 堆积、FIN_WAIT_2 悬挂；理解 RST、close 与 shutdown、Keepalive 与心跳
- 用实测输出佐证状态变化，用交互实验回放各种握手 / 挥手异常

6.1 逐步拆解

假设客户端先调用 `close()`（谁先关闭谁就是**主动关闭方**，服务器也可以是主动方）。客户端已发送的最后一个字节序号为 u−1，服务器为 v−1：

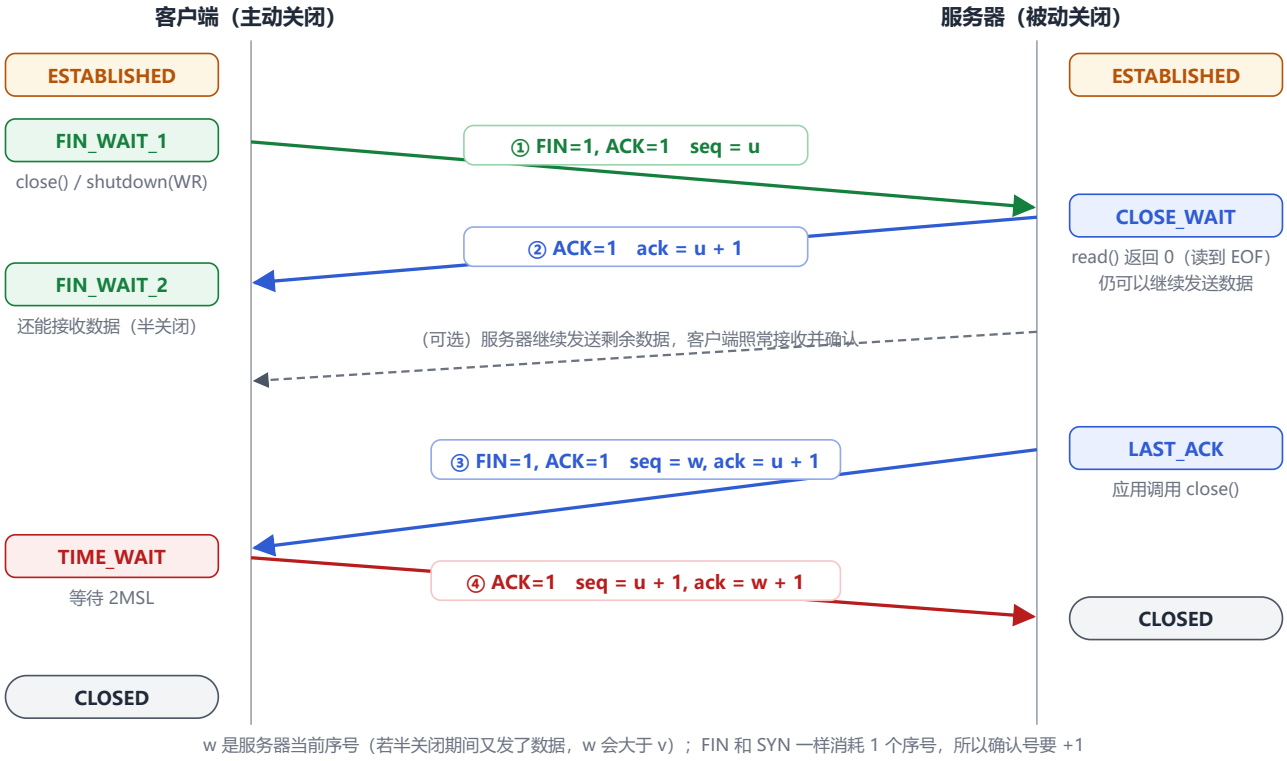


图 6-1 四次挥手：①② 关闭"客户端 → 服务器"方向，③④ 关闭"服务器 → 客户端"方向；①② 与 ③④ 之间是半关闭状态

步骤	方向	报文	发送后 / 收到后的状态	含义
①	C → S	FIN, seq=u	C：ESTABLISHED → FIN_WAIT_1 S：ESTABLISHED → CLOSE_WAIT（收到后）	"我没有数据要发了"
②	S → C	ACK, ack=u+1	C：FIN_WAIT_1 → FIN_WAIT_2（收到后）	"知道了"——由内核自动回复，不需要应用参与
③	S → C	FIN, seq=w	S：CLOSE_WAIT → LAST_ACK C：FIN_WAIT_2 → TIME_WAIT（收到后）	服务器应用调用 close 后才发出："我也说完了"
④	C → S	ACK, ack=w+1	S：LAST_ACK → CLOSED（收到后） C：TIME_WAIT 等待 2MSL → CLOSED	"知道了，再见"

🧪 实测证据（d1 • E4 / E5）：用系统连接表看四次挥手

程序在本机建立一条回环连接，在每一步之后调用 `GetTcpTable`（相当于 `netstat -an`）打印双方状态：

d1_tcp_udp_behaviors.exe · E4 / E5

```
== E4/E5 四次挥手的状态变化 + 半关闭 ==
  [① 三次握手完成，accept 返回后]
    服务端监听 127.0.0.1:40031 → 127.0.0.1:0      LISTEN
    服务端  127.0.0.1:40031 → 127.0.0.1:53735  ESTABLISHED
    客户端  127.0.0.1:53735 → 127.0.0.1:40031  ESTABLISHED
  [② 客户端 shutdown(SD_SEND) 发出 FIN，服务端内核自动回 ACK 之后]
    服务端监听 127.0.0.1:40031 → 127.0.0.1:0      LISTEN
    服务端  127.0.0.1:40031 → 127.0.0.1:53735  CLOSE_WAIT
    客户端  127.0.0.1:53735 → 127.0.0.1:40031  FIN_WAIT_2
  服务端 recv 返回 0（0 = 读到 FIN，对方不会再发数据了）
  客户端在 FIN_WAIT_2 中仍收到 21 字节: "server-still-can-send"  ← 这就是"半关闭"
  客户端 recv 返回 0（读到服务端 FIN）
  [③ 服务端 closesocket 发 FIN、客户端回最后一个 ACK 之后]
    服务端监听 127.0.0.1:40031 → 127.0.0.1:0      LISTEN
    客户端  127.0.0.1:53735 → 127.0.0.1:40031  TIME_WAIT
```

三个结论都被验证了：① 收到 FIN 的一方立刻进入 CLOSE_WAIT（内核自动回了 ACK，应用还什么都没做）；② 主动方在 FIN_WAIT_2 中**仍然可以接收数据**；③ 最后只有主动关闭方留下 TIME_WAIT，被动方的连接条目直接消失了。

6.2 为什么是四次？什么时候会是三次？

建立连接时，服务器的 SYN 和 ACK 可以合并，是因为服务器收到 SYN 时"没有什么需要等待的"，可以立刻发出自己的 SYN。关闭时则不同：**收到对方 FIN 只说明对方不再发数据，自己可能还有数据没发完。**所以 ACK（内核立即回）和 FIN（等应用处理完、调用 close 时才发）通常分两次发出，中间就是半关闭阶段。

🔬 抓包时常看到的"三次挥手"

如果被动方收到 FIN 时恰好没有待发数据，且应用很快就调用了 close（例如 HTTP 服务器读到 EOF 后立即关闭），由于**延迟确认**的存在，内核还没来得及发出单独的 ACK，FIN 就已经要发了，于是 ACK 和 FIN 合并成一个 `FIN+ACK` 报文，挥手就变成了 3 个报文。这完全合法，并不违反协议——"四次"是逻辑上的四个动作，不是必须的四个报文。

✅ 面试答法：为什么挥手要四次?

因为 TCP 是全双工的，两个方向需要分别关闭。主动方发 FIN 只表示它不再发送数据，但还能接收；被动方收到 FIN 后先由内核回 ACK，此时它可能还有数据要发，必须等应用发完并调用 close 才能发出自己的 FIN，所以 ACK 和 FIN 一般不能合并，总共四次。如果被动方此时没有数据要发，且启用了延迟确认，ACK 和 FIN 可以合并，就会变成三次。

6.3 TIME_WAIT：面试最爱问的状态

为什么需要 TIME_WAIT

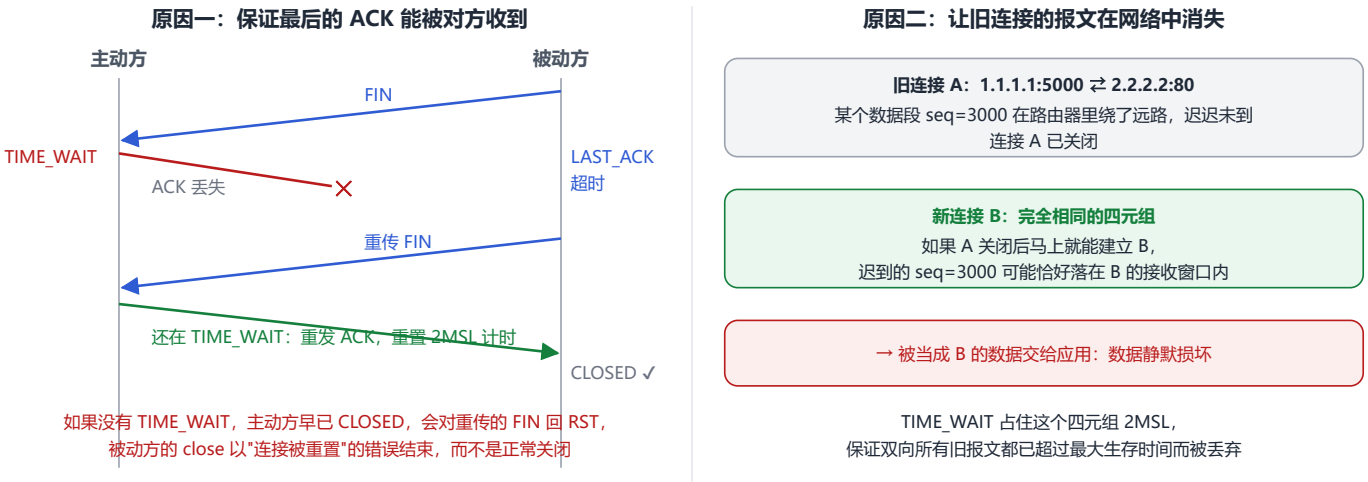


图 6-2 TIME_WAIT 存在的两个原因：可靠地终止连接、防止旧报文污染新连接

为什么是 2MSL

MSL（Maximum Segment Lifetime）是报文在网络中的最大生存时间。等待 2MSL 可以覆盖最坏情况：最后的 ACK 在路上最多走 1 个 MSL 后丢失，对方超时重传的 FIN 最多再走 1 个 MSL 到达——**一来一回**。同时，2MSL 也足以让两个方向上旧连接的所有报文都在网络中消亡。

系统	TIME_WAIT 时长	说明
RFC 793	2MSL = 4 分钟	MSL 建议 2 分钟（工程上普遍认为过长）
Linux	**60 秒**，写死在内核宏 `TCP_TIMEWAIT_LEN`	不能通过 sysctl 修改；`tcp_fin_timeout` 控制的是 FIN_WAIT_2，不是 TIME_WAIT（常见误解）
Windows	默认 120 秒左右（老版本 240 秒）	注册表 `TcpTimedWaitDelay` 可调

TIME_WAIT 过多有什么危害

TIME_WAIT 在**主动关闭方**产生。每个 TIME_WAIT 只占很少的内存（Linux 有专门的轻量结构），真正的问题是它**占用四元组**：

- **客户端（或作为客户端的代理 / 网关）**：频繁对同一个 `服务器IP:端口` 发起短连接并主动关闭，本地临时端口（约 2.8 万个）会被 TIME_WAIT 占满，新的 connect 报错 `EADDRNOTAVAIL`（端口耗尽）。按 60 秒计，每秒超过约 470 个新连接就会耗尽。
- **服务器**：服务端作为主动关闭方（如 HTTP 短连接由服务器关闭）时也会积累大量 TIME_WAIT，但由于对端 IP 和端口各不相同，一般不会耗尽什么，主要是统计数字吓人；监控和表项数量（`tcp_max_tw_buckets`）需要关注。

怎么优化

方法	原理	注意事项
长连接 / 连接池（首选）	HTTP Keep-Alive、数据库连接池，不再频繁建连关连	从根源上消除问题
`net.ipv4.tcp_tw_reuse = 1`	允许**新发起的连接**复用处于 TIME_WAIT 超过 1 秒的四元组，靠时间戳判断旧报文	只对客户端（connect 方）有效；需开启 `tcp_timestamps`；较新内核默认值为 2（仅对回环地址生效）
~~tcp_tw_recycle~~	快速回收 TIME_WAIT	在 NAT 后的多个客户端时间戳不一致时会导致连接失败，**Linux 4.12 已删除**，面试时要指出它的坑
扩大临时端口范围	`ip_local_port_range` 调成 `1024 65535`	缓解而非根治
让客户端主动关闭	TIME_WAIT 落在分散的客户端上	需要协议配合（如 HTTP 的 Connection: close 由谁发起）
`SO_LINGER` {on, 0}	close 时直接发 RST，不进入 TIME_WAIT	**危险**：未发送的数据被丢弃，对端收到"连接被重置"；只在明确知道后果时使用

⚠️ 易错点：SO_REUSEADDR 和 TIME_WAIT 是什么关系?

服务器进程重启时 `bind(:8080)` 报 "Address already in use"，往往是因为旧进程的连接还有 TIME_WAIT 占着 8080。设置 `SO_REUSEADDR` 后，即使有 TIME_WAIT 连接，也允许监听 socket 绑定该端口（Linux 语义）。所以**服务器的监听 socket 几乎都应该设置 SO_REUSEADDR**。它解决的是"重启 bind 失败"，和"客户端端口耗尽"是两个不同的问题。另外注意 Windows 上 SO_REUSEADDR 的语义不同（允许抢占端口），应使用 `SO_EXCLUSIVEADDRUSE` 保护服务端口。

6.4 CLOSE_WAIT 与 FIN_WAIT_2：两种"卡住"

大量 CLOSE_WAIT：几乎一定是你的 bug

CLOSE_WAIT 表示"对方已经关了，我的应用还没 close"。内核不会替应用关闭，所以如果代码在读到 EOF（`recv` 返回 0）后忘了 close，或者异常分支

FIN_WAIT_2 悬挂：等对方的 FIN

主动方发完 FIN 收到 ACK 后，要等对方发 FIN。如果对方迟迟不关（正好处于 CLOSE_WAIT 的 bug 状态），主动方就挂在 FIN_WAIT_2。

漏掉了 close，或者连接池没有检测并回收被对端关闭的连接，CLOSE_WAIT 就会一直堆积，直到耗尽文件描述符。
排查：`ss -tanp state close-wait` 找到进程和对端，再检查对应代码路径。

如果主动方是用 `close()` 关闭的（连接已成为孤儿），Linux 最多等 `tcp_fin_timeout`（默认 60 秒）后直接回收；如果是用 `shutdown(SHUT_WR)` 关闭的，应用还能读，内核会**一直等**。

6.5 挥手过程中的异常

异常	谁会察觉	处理
① FIN 丢失	主动方收不到 ACK	主动方在 FIN_WAIT_1 超时重传 FIN；孤儿连接的重传次数由 `tcp_orphan_retries` 控制，耗尽后直接关闭
② ACK 丢失	主动方收不到 ACK	主动方重传 FIN，被动方（已在 CLOSE_WAIT）再次回 ACK；若被动方的 FIN 先到，主动方从 FIN_WAIT_1 收到 FIN+ACK 也能直接进入 TIME_WAIT
③ 被动方的 FIN 丢失	被动方收不到 ACK	被动方在 LAST_ACK 重传 FIN；主动方仍在 FIN_WAIT_2 等待（孤儿连接最多等 tcp_fin_timeout）
④ 最后的 ACK 丢失	被动方收不到 ACK	被动方重传 FIN；主动方在 TIME_WAIT 中重发 ACK 并**重置 2MSL 计时器**（图 6-2 左）
同时关闭	双方同时发 FIN	双方：FIN_WAIT_1 → 收到对方 FIN → **CLOSING** → 收到 ACK → TIME_WAIT。双方都会有 TIME_WAIT
进程崩溃 / 被 kill	—	进程退出时内核会关闭它所有的 fd，**照常发送 FIN**，对端看到的是正常关闭（若接收缓冲区有未读数据则发 RST）
主机断电 / 网线被拔	对端完全不知道	没有任何报文发出，对端连接会一直保持 ESTABLISHED（"半打开"连接），直到它发送数据超时，或者 Keepalive / 应用心跳探测失败
close 时接收缓冲区还有未读数据	对端收到 RST	Linux 会直接发 RST 而不是 FIN（RFC 2525 建议的行为），提示对方"你发的数据我没读到"

6.6 RST：不体面但有时必要的结束方式

RST（Reset）表示"立即终止连接，不用再回复"。它不需要确认，收到 RST 的一方立即释放连接，并把**未发送的数据全部丢弃**，应用的下一次读写会得到 `ECONNRESET`（"Connection reset by peer"）；发 RST 的一方也**不会进入 TIME_WAIT**。常见触发场景：

- SYN 发往无人监听的端口（第 5 章 E3 实测）；
- 收到不属于任何连接的报文（例如服务器重启后，客户端仍用旧连接发数据）；
- 设置了 `SO_LINGER {1, 0}` 后调用 close（下方实测）；
- close 时接收缓冲区有未读数据；
- 防火墙、负载均衡主动掐断连接（"连接被重置"在跨境网络中很常见）。

实测证据（d1 • E6）：SO_LINGER{1,0} 直接发 RST

d1_tcp_udp_behaviors.exe · E6

```
== E6 SO_LINGER{1,0}: 用 RST 代替四次挥手 ==
  服务端 recv 返回 -1, WSAGetLastError=10054（10054=WSAECONNRESET）
  [RST 之后（注意：客户端没有 TIME_WAIT）]
    服务端监听 127.0.0.1:40041 → 127.0.0.1:0      LISTEN
```

对比 E4 的结果：正常挥手后主动方留下了 TIME_WAIT；用 RST 关闭后双方的连接条目都立即消失，但对端得到的是"连接被重置"的错误，而不是表示正常结束的 recv 返回 0。

close() 与 shutdown() 的区别

	`close(fd)`	`shutdown(fd, SHUT_WR)`
作用对象	文件描述符：引用计数减 1，**减到 0 才真正关闭连接**（fork 后父子进程共享 socket 时要注意）	直接作用于连接，不管有多少个 fd 引用
方向	同时关闭读和写	可以只关写（发 FIN）、只关读、或都关
之后还能读吗	不能（连接成为孤儿）	SHUT_WR 后仍然可以读，用于"半关闭"
典型用途	普通关闭	"我发完了，但还要等你的回复"：例如客户端上传完文件后 shutdown 写端，再读取服务器的结果

6.7 保活：Keepalive 与应用层心跳

"主机断电"那一行说明：TCP 本身**无法及时发现对端消失**。TCP 提供了 Keepalive 机制，但默认参数非常保守：

参数（Linux）	默认值	含义
`tcp_keepalive_time`	7200 秒	连接空闲 2 小时后才开始探测
`tcp_keepalive_intvl`	75 秒	探测间隔
`tcp_keepalive_probes`	9 次	连续失败多少次判定断开

默认要 2 小时 11 分钟才能发现死连接，且 Keepalive 需要每个 socket 用 `SO_KEEPALIVE` 显式开启（可以用 `TCP_KEEPIDLE` 等选项为单个连接缩短时间）。更重要的是，Keepalive 只能证明**对端内核还活着**，证明不了对端应用没有卡死。

🎮 游戏开发视角：为什么游戏都用应用层心跳

游戏需要在几秒内发现玩家掉线（以便托管 AI 接管、通知队友、判定逃跑），而且要知道的是"客户端逻辑是否还在运行"。所以几乎所有游戏都在应用层每隔 1～5 秒发一个心跳包，服务器若连续 N 个周期没有收到任何包就判定断线。心跳还能顺便测量 RTT、给 NAT 映射保活（UDP 尤其需要）。对于基于 UDP 的可靠协议，这就是它的"连接管理"的一部分（第 8 章）。

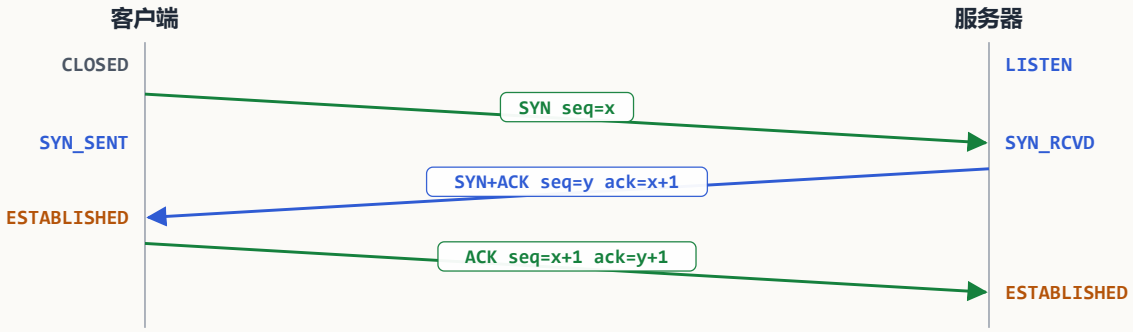
6.8 交互实验：握手与挥手场景回放

选择一个场景，逐步回放报文和双方状态的变化。红色虚线 × 表示报文丢失。建议先把每个场景的结果在心里推一遍，再点击验证。

🧪 交互实验 1 • TCP 连接管理场景回放

场景： ① 正常三次握手 ◀ 上一步 下一步 ▶ 全部展开

↺ 重置



第 1 步：客户端 connect() 发出 SYN；服务器收到后进入 SYN_RCVD，请求放入半连接队列，内核立即回 SYN+ACK。

第 2 步：服务器的 SYN 与对 x 的确认合并发送。客户端收到后进入 ESTABLISHED，connect() 返回。

▶ 第 3 步：服务器收到第三次握手，连接移入全连接队列，等待 accept() 取走。

本章小结

- 四次挥手：**FIN → ACK →（半关闭，可继续发数据）→ FIN → ACK**；主动方 FIN_WAIT_1 → FIN_WAIT_2 → TIME_WAIT → CLOSED，被动方 CLOSE_WAIT → LAST_ACK → CLOSED（已实测）。
- 四次是因为全双工、两个方向分别关闭；被动方无数据可发且有延迟确认时会合并成三次。
- TIME_WAIT 在**主动关闭方**，持续 2MSL（Linux 固定 60 s）：保证最后的 ACK 送达、让旧报文消亡。
- TIME_WAIT 过多 → 客户端端口耗尽；优先用长连接，其次 tcp_tw_reuse；tcp_tw_recycle 已删除；SO_LINGER 0 有丢数据风险。
- CLOSE_WAIT 堆积 = 应用没 close（bug）；FIN_WAIT_2 孤儿连接由 tcp_fin_timeout 回收。
- RST 立即终止、丢弃数据、不进 TIME_WAIT；close 按引用计数关闭两个方向，shutdown 可单向半关闭。
- TCP Keepalive 默认 2 小时才探测，游戏等场景用应用层心跳。

▶ **思考题 1：服务器上用 netstat 看到大量 TIME_WAIT，说明了什么？要不要处理？**

▶ **思考题 2：线上服务出现几千个 CLOSE_WAIT，并且还在增长，你怎么排查？**

▶ **思考题 3：为什么 TIME_WAIT 放在主动关闭方，而不是被动关闭方？**

▶ **思考题 4：客户端进程被 kill -9 和客户端机器断电，服务器看到的现象有什么不同？**

PART FOUR

第四部分　可靠 UDP：在用户态重新组合可靠性

TCP 已经把可靠传输做到了极致，为什么还有人要在 UDP 上重造一遍？因为 TCP 给出的是"一种"可靠性，而实时应用需要的是"可以挑选的"可靠性。

CHAPTER 07

为什么要在 UDP 上造轮子

先把 TCP 在实时场景下的痛点逐条列出来，再画出"可靠性光谱"，最后给出一个可靠 UDP 协议的整体架构。

本章目标

- 说出 TCP 用于实时场景的六个痛点，并能和前几章的机制对应起来
- 理解"可靠性光谱"：可靠 / 不可靠 × 有序 / 无序 / 只取最新，并能给游戏数据分类
- 画出可靠 UDP 协议的分层架构，知道每一层要解决什么
- 认清自研可靠 UDP 的代价与风险

7.1 TCP 在实时场景下的六个痛点

#	痛点	来自哪个机制	实时场景下的后果	能在 TCP 内解决吗
1	**队头阻塞**	按序可靠交付（第 3、4 章）	一个包丢了，后面所有新数据都要等它	**不能，这是 TCP 语义本身**
2	**重传慢**	RTO 下限 200 ms、指数退避；快速重传需要 3 个重复 ACK	小流量时凑不够 3 个重复 ACK，只能等 RTO；连续丢包时等待时间翻倍增长	**部分（Linux 可按路由调 rto_min，有 TLP/RACK，但不能每个连接随意改）**
3	**拥塞控制误判**	丢包即拥塞（Reno / CUBIC）	无线随机丢包导致窗口被压缩，数据在发送缓冲排队，延迟越积越大	**部分（可换 BBR，但不能关掉）**
4	**延迟确认 + Nagle**	减少小包的优化（第 3 章）	额外 40 ~ 200 ms	**能（TCP_NODELAY、TCP_QUICKACK）**
5	**握手与换网**	三次握手 + TLS；连接绑定四元组	首包 2 ~ 3 RTT；Wi-Fi 切 4G 时 IP 变化，连接直接断开，要重新握手	**基本不能（TFO 部署受限；MPTCP 不普及）**
6	**一视同仁 + 难以迭代**	实现在内核；中间设备僵化	不能对不同消息选择不同的可靠性 / 优先级；新算法要等操作系统升级	**不能**

📦 **场景案例：一次 300 ms 的"卡顿"是怎么来的**

手游玩家在地铁里，网络 RTT 80 ms、丢包率 5%。游戏每 66 ms 发一个操作包（15 Hz），用 TCP 承载。某个包丢了：

① 后面每 66 ms 才有一个新包，要凑够 3 个重复 ACK 需要约 200 ms，往往还没凑够 RTO 先到了；

② 于是等到 RTO（≥ 200 ms，弱网下 RTTVAR 大，RTO 常常 300 ms 以上）才重传；

③ 这期间到达的所有新操作都被队头阻塞挡在内核缓冲区里；

④ 重传成功后，积压的操作一起交付，角色表现为"卡住 → 瞬移"。如果重传的包又丢了，RTO 翻倍，卡顿就变成 600 ms 以上。

而同样的网络，用 UDP + 应用层可靠，可以做到"后面的包不等前面"（痛点 1）、"被跳过 2 次就重传"（痛点 2）、"每个包都冗余携带最近 3 帧操作"（干脆不需要重传），把卡顿压到一个 RTT 甚至零。

7.2 可靠性光谱：不是所有数据都需要 TCP 那样的可靠

把"可靠性"拆开，至少有两个独立的维度：**是否保证送达**，以及**是否保证顺序**。顺序这一维还有一个很实用的中间态：**只要最新**（sequenced，旧的晚到就直接丢）。组合起来就是一张光谱：

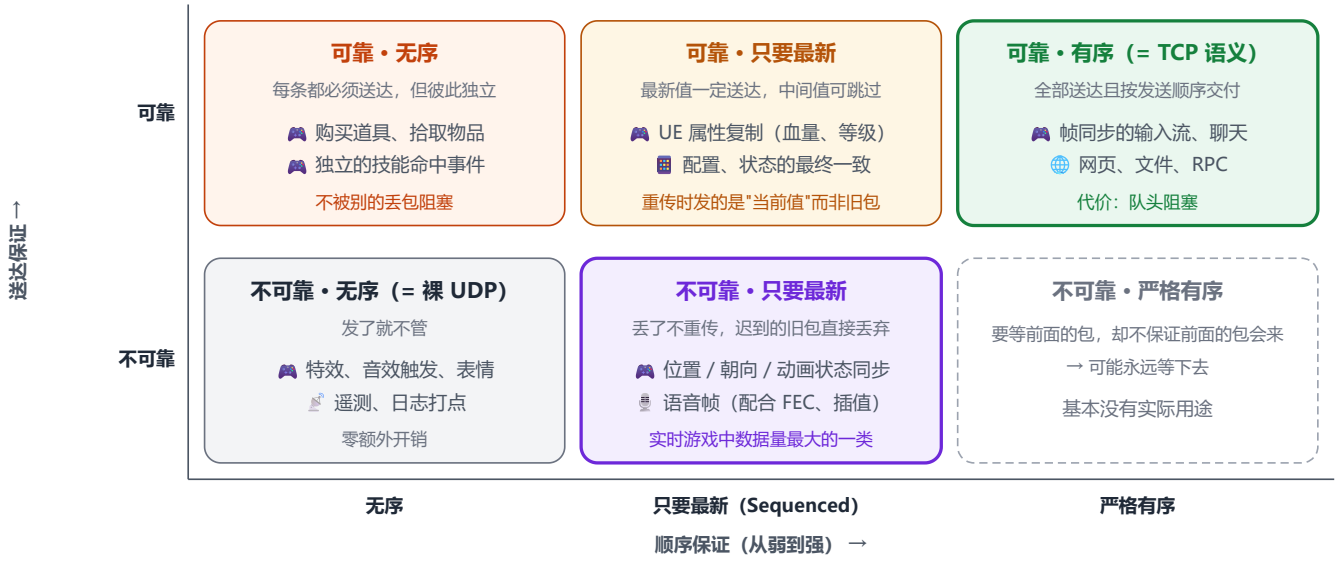


图 7-1 可靠性光谱：TCP 只提供右上角一格，实时应用的大部分数据其实落在其他格子里

这张图在业界有很直接的对应：老牌游戏网络库 **RakNet** 的 `PacketReliability` 枚举就是 `UNRELIABLE`、`UNRELIABLE_SEQUENCED`、`RELIABLE`、`RELIABLE_ORDERED`、`RELIABLE_SEQUENCED`；**ENet** 提供可靠 / 不可靠 / 不可靠有序（sequenced）三种包，并用多个通道（channel）隔离顺序；**Unreal Engine** 里 Reliable RPC 在同一 Actor 通道内有序可靠，Unreliable RPC 发了就不管，**属性复制**则是"不可靠发送 + 丢包后重新发送当前最新值"，正好是"可靠 · 只要最新"这一格。

> ✅ **面试答法：为什么游戏要用可靠 UDP 而不是 TCP?**
>
> 核心是"可靠性可选"。游戏数据分为不同类别：位置等状态只要最新值，丢了不需要重传；技能、交易等事件必须送达但不一定需要和其他消息保持顺序；帧同步的输入需要可靠有序。TCP 只能对所有数据提供可靠有序的字节流，一个丢包会通过队头阻塞拖累所有数据，加上 RTO 下限高、指数退避、拥塞控制把随机丢包当拥塞，弱网下延迟抖动很大。在 UDP 上自己实现，可以按消息类型分通道、选择不同可靠级别，并且激进地调整重传策略（如 KCP 的 RTO×1.5、快速重传阈值 2），用一些带宽换取更低更稳定的延迟。

7.3 一个可靠 UDP 协议的整体架构



图 7-2 可靠 UDP 的分层架构：KCP 只实现了第 ③ 层（加一点 ②），QUIC 则实现了全部六层

这张图同时回答了"KCP 和 QUIC 有什么区别"这个常见问题：**KCP 是一个纯 ARQ 算法库**，只负责第 ③ 层，连接、加密、FEC 都要使用者自己补；**QUIC 是一个完整的传输协议**，从流复用到加密到连接迁移全都标准化了，可以直接替代 TCP + TLS。

7.4 代价与风险：不要为了"用 UDP"而用 UDP

复杂度与正确性

可靠传输的边界情况极多：序号回绕、重传歧义、窗口死锁、ACK 风暴、分片丢失、重连时的状态残留……TCP 用了四十年打磨，自研协议的 bug 往往在线上弱网环境才暴露。

网络环境的敌意

部分运营商对 UDP 做 QoS 限速，部分企业 / 校园网直接封禁 UDP，一些公共 Wi-Fi 的 NAT 对 UDP 映射超时极短。**必须准备 TCP 兜底通道**并做自动降级。

CPU 开销

用户态协议栈没有 TCP 的硬件卸载（TSO/GRO），每个包都要系统调用、加解密、定

网络公平性

"关掉拥塞控制"在一个玩家的手机上很好，在一个网吧几十台机器同时这么做时就会互相挤爆出口带

时器管理。服务器端单机承载量可能明显低于 TCP，需要批量收发、多线程分片等优化。

宽。至少要有发送速率上限，以及在 RTT 明显上升时主动降级。

本章小结

- TCP 的实时痛点：**队头阻塞、重传慢（RTO 下限 + 退避）、拥塞误判、Nagle/延迟确认、握手与换网断连、一视同仁且难迭代。**
- 可靠性光谱 = {可靠, 不可靠} × {无序, 只要最新, 有序}；游戏数据大多落在"不可靠 · 只要最新"和"可靠 · 无序"。
- RakNet / ENet / UE 的可靠性分级与光谱一一对应；UE 属性复制属于"可靠 · 只要最新"。
- 可靠 UDP 六层：通道调度、分片、可靠传输、拥塞 / FEC、安全、连接管理。KCP 只做可靠传输层，QUIC 做全部。
- 代价：复杂度、UDP 被限速 / 封禁（需 TCP 兜底）、CPU 开销、公平性。

▶ **思考题 1：为什么"不可靠 • 严格有序"基本没有用途?**

▶ **思考题 2：一个 MOBA 游戏中，以下数据分别应该放在可靠性光谱的哪一格？英雄位置、技能释放指令、击杀公告、小地图信号、商店购买、结算数据。**

▶ **思考题 3：既然 QUIC 已经标准化了，为什么很多游戏还是用 KCP 或自研协议?**

CHAPTER 08

亲手实现可靠 UDP：机制逐个拆解

序号、确认、重传、窗口、乱序、去重、拥塞、纠错、分片、连接——把第 3 章的 TCP 机制在用户态重新实现一遍，并且每一处都想清楚"能不能做得不一样"。

本章目标

- 能设计一个可靠 UDP 的报文头，并说明每个字段的用途
- 掌握序号回绕比较、三种确认方式（逐包 ACK、UNA、ACK 位图）
- 掌握 RTT 测量、RTO 计算、退避策略与快速重传的实现细节
- 对比停等、回退 N 步、选择重传三种 ARQ，会算信道利用率
- 会写接收端的环形缓冲（乱序重排 + 去重）和发送端的定时驱动循环
- 理解拥塞避免、FEC、冗余发送、分片、连接管理与安全在可靠 UDP 中的做法
- 能用仿真数据解释"为什么选择重传 + 快速重传 + 温和退避"在丢包网络下最好

8.1 报文设计

先设计一个教学用的报文头。它和 KCP 的头部（第 9 章）非常接近，只多了一个"通道"字段和一个 ACK 位图：



共 28 字节；每个数据包都捎带 una 和 ack_bits，所以 ACK 不必单独成包（双向都有流量时）
cmd：DATA / ACK / PING / PONG / CONNECT / ACCEPT / CLOSE / WND_PROBE

图 8-1 教学版可靠 UDP 报文头：每个字段都对应 TCP 头部的一个"同行"

字段	对应 TCP 的	为什么这样设计
conv	四元组	移动网络下 NAT 可能更换客户端的公网端口（NAT 重绑定），用会话 ID 识别连接就不会因此断线；还可以在一个 UDP 端口上复用很多会话
sn	seq	**按包编号**而不是按字节编号：UDP 本身就保留消息边界，按包编号更简单，确认与重传也以包为单位
una	ack	累计确认，语义和 TCP 的 ack 一样："下一个期望的包"
ack_bits	SACK 选项	定长 32 位，比 TCP 的 SACK 块更紧凑，每个包都能带
ts	时间戳选项	接收方在 ACK 里回显发送时间，发送方 `now - ts` 就是 RTT，**重传的包也能正确采样**（解决 Karn 问题）
wnd	窗口	流量控制
chan	（无）	不同通道有独立的序号空间和可靠级别，互不阻塞
frg	（无，TCP 是字节流）	大消息分片：还剩几片，为 0 表示最后一片

8.2 序列号：回绕与比较

序号字段总有上限：32 位约 43 亿，16 位只有 65536。游戏每秒 60 个包，16 位序号 18 分钟就会回绕。回绕后 `5 < 65530` 这样的直接比较就错了。正确做法是**序列号算术（RFC 1982）**：把差值解释成有符号数。

回绕安全的序号比较（TCP 内核中的 before()/after() 也是这么写的）

```
// a 是否"在 b 之后"：差值按有符号数解释，只要两者相距不超过序号空间的一半就是对的
inline bool SeqAfter(uint32_t a, uint32_t b)  { return int32_t(a - b) > 0; }
inline bool SeqBefore(uint32_t a, uint32_t b) { return int32_t(a - b) < 0; }
inline int32_t SeqDiff(uint32_t a, uint32_t b) { return int32_t(a - b); }

// 16 位序号同理（Glenn Fiedler 的游戏网络协议就用 16 位以节省包头）
inline bool Seq16After(uint16_t a, uint16_t b) { return int16_t(uint16_t(a - b)) > 0; }

// 例：Seq16After(5, 65530) == true，因为 (uint16_t)(5 - 65530) = 11 > 0
```

⚠️ **易错点：窗口必须小于序号空间的一半**

如果在途的包跨度超过序号空间的一半，`SeqAfter` 就无法分辨"新包"和"很旧的包"。所以发送窗口必须远小于 2^{31}（32 位）或 2^{15}（16 位）。TCP 在高速网络中序号回绕更快（10 Gbps 下 32 位字节序号只要 3.4 秒），所以要靠时间戳选项做 PAWS 保护。

8.3 确认：三种方式及其组合

方式	做法	优点	缺点	代表
逐包 ACK	每收到一个包，回一条 `ACK(sn, ts)`，可以把多条合并进一个包	信息最精确，每条都能测 RTT	ACK 本身丢了就丢了这条信息；ACK 量大	KCP 的 ACK 列表
累计确认 UNA	只告诉对方"una 之前的全收到了"	一个数字就能确认很多包，对 ACK 丢失不敏感（下一个 ACK 包含了上一个的信息）	无法表达"una 之后收到了哪些"，会导致不必要的重传	TCP 的 ack、KCP 的 una
ACK 位图	una（或最新收到的 sn）+ 后续 32 个包的接收位图	**高度冗余**：同一个包的确认信息会在连续 32 个 ACK 里重复出现，ACK 丢几个也无所谓	只能覆盖 32（或 64）个包的范围	Glenn Fiedler《Reliability over UDP》、很多游戏引擎



图 8-2 UNA + ACK 位图：一个定长字段同时表达累计确认和选择确认

> 🎮 **游戏开发视角：ACK 捎带与"ACK 也会丢"**
>
> 游戏双向都在持续发包（客户端发输入，服务器发状态），所以确认信息**捎带在每个数据包的头部**里，几乎没有单独的 ACK 包。由于每个包都带"最近 32 个包的接收情况"，即使一半的包都丢了，发送方仍然能收到足够的确认信息。这比 TCP 的"一个 ACK 丢了就只能等下一个"要健壮得多。

8.4 超时重传：RTT、RTO 与退避

测量 RTT

发送时在包头写入当前时间 `ts`，接收方在确认中原样回显，发送方收到后 `rtt = now - ts`。因为回显的是**那一次发送**的时间戳，重传的包也能被正确采样，不存在重传歧义。计算 RTO 时直接套用 RFC 6298 的公式（第 3 章），差别在于各个参数可以自由调整：

参数	TCP（Linux）	可靠 UDP 常见取值	考虑
RTO 下限	200 ms	30～100 ms（KCP 极速模式 30 ms）	越小重传越快，但 RTT 抖动大时会产生伪重传
超时后退避	× 2（指数）	× 1.5（KCP）、固定不退避、或设上限（如不超过 4×SRTT）	退避是为了在严重拥塞时让网络喘口气；实时应用更怕等
定时器粒度	每连接一个重传定时器	每个包各自记录 `resend_at`，由一个固定周期的 update 统一检查	按包定时更灵活；update 周期（KCP 的 interval，常设 10 ms）决定了时延精度
快速重传阈值	3 个重复 ACK	被后续包"跳过" 2 次（KCP resend=2）	越小越快，但对乱序越敏感

快速重传：用"跳过次数"代替"重复 ACK"

有了逐包确认或 ACK 位图，就不需要靠重复 ACK 去猜了：每当发送方得知某个比 X 更大的包已被收到，就给 X 的 `fastack` 计数加一；计数达到阈值就立即重传 X。图 8-2 中的 101 已经被 102、103、105、108 跳过了 4 次。

> 🔬 **实现细节：别对刚重传的包重复计数**
>
> 如果 X 刚被重传过，那些在重传*之前*就已经发出的更大的包，它们的确认会继续"跳过" X，导致 X 被反复快速重传。严谨的做法是：只有**发送时间晚于 X 最近一次发送**的包被确认，才算一次有效跳过（这就是 Linux RACK 算法的思想）。KCP 选择了更简单的做法：照常计数，但用 `fastlimit`（默认 5）限制一个包最多被快速重传几次——宁可多发一点，也不要等。本书的仿真程序两种都实现了。

8.5 三种 ARQ 策略

ARQ（Automatic Repeat reQuest，自动重传请求）是"确认 + 超时重传"这一类机制的统称。按照**一次允许多少个未确认的包**和**出错时重传哪些包**，分为三种：

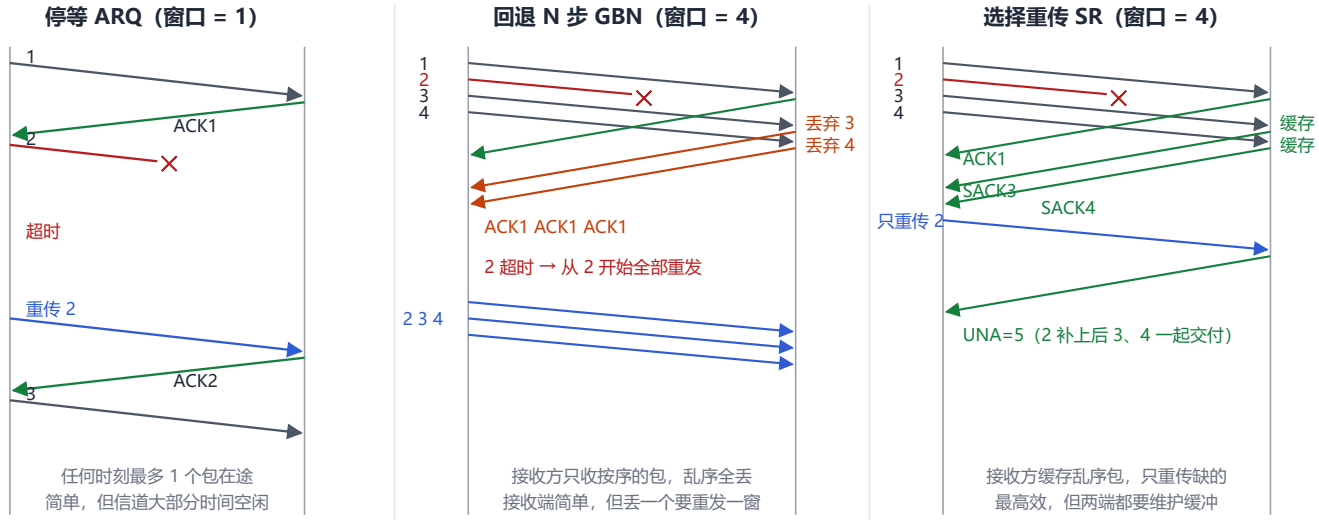


图 8-3 三种 ARQ：停等每次一个；GBN 出错后"回退"到丢失点全部重发；SR 只重发真正丢失的包

信道利用率

设发送一个包需要 T（发送时延 = 包长 / 带宽），往返时延 RTT，窗口 W 个包，无丢包时的利用率：

滑动窗口的信道利用率

```
U = min(1,  W × T / (T + RTT))

停等（W=1）：T = 1 ms（1200 字节 @ ~10 Mbps），RTT = 60 ms  →  U = 1 / 61 ≈ 1.6%
滑动窗口 W=64：                                              →  U = min(1, 64 / 61) = 100%
跑满带宽所需的最小窗口 W ≥ (T + RTT) / T ≈ RTT / T = 带宽时延积（以包为单位）
```

这就是为什么第 3 章说"窗口要至少等于 BDP"。三种 ARQ 的完整对比：

	停等	回退 N 步（GBN）	选择重传（SR）
发送窗口	1	N	N
接收窗口	1	1（只收按序包）	N（缓存乱序包）
确认方式	逐包	累计确认	逐包 / 选择确认（+ 累计）
出错时重传	当前包	从丢失包开始的整个窗口	只重传丢失的包
序号空间要求	≥ 2	≥ N + 1	**≥ 2N**（否则新旧包混淆）
适用	低速 / 简单链路	低误码率链路	高时延、有丢包的链路（TCP+SACK、KCP、QUIC 都属于此类）

8.6 滑动窗口与流量控制

发送端维护三个窗口取最小值：`可发送数 = min(本地发送窗口, 对端通告的接收窗口 wnd, 拥塞窗口 cwnd) − 在途包数`。接收端在每个包头里填写 `wnd = 接收缓冲区剩余槽位`。

- **窗口为 0 时**：和 TCP 一样会有"窗口更新包丢了导致双方死等"的风险。KCP 的做法是发送方定时发送 `WASK`（询问窗口）命令，接收方回 `WINS`（告知窗口）——相当于 TCP 的零窗口探测。
- **接收窗口要比发送窗口大**：KCP 默认发送窗口 32、接收窗口 128。因为接收方还要容纳乱序到达的包，接收窗口太小时，乱序包会因为"超出窗口"被丢弃，然后被重传。

8.7 乱序处理与去重：接收端环形缓冲



图 8-4 接收端用环形缓冲同时完成三件事：乱序暂存、按序交付、重复检测

接收端：乱序重排 + 去重 + 按序交付（可靠有序通道）

```
struct Receiver {
    static constexpr uint32_t kWnd = 128;
    uint32_t rcvNxt = 0;                                    // 下一个期望交付的 sn
    std::array<std::optional<std::string>, kWnd> slots;     // 环形缓冲，下标 sn % kWnd
    std::vector<uint32_t> ackList;                          // 待回复的确认

    void OnData(uint32_t sn, std::string payload, auto&& deliver) {
        ackList.push_back(sn);                              // ① 不管新旧，先记下要确认（重复包也要
        if (SeqBefore(sn, rcvNxt)) return;                  // ② 已经交付过的旧包：去重
        if (SeqDiff(sn, rcvNxt) ≥ int32_t(kWnd)) return;    // ③ 超出接收窗口：丢弃，等对方重传
        auto& slot = slots[sn % kWnd];
        if (slot) return;                                   // ④ 窗口内的重复包：去重
        slot = std::move(payload);                          // ⑤ 乱序暂存
        while (slots[rcvNxt % kWnd]) {                      // ⑥ 从 rcvNxt 起把连续的包按序交付
            deliver(std::move(*slots[rcvNxt % kWnd]));
            slots[rcvNxt % kWnd].reset();
            ++rcvNxt;
        }
    }
    uint16_t Wnd() const { /* 剩余空槽数，填进包头的 wnd */ return uint16_t(kWnd - CountUsed());
    uint32_t Una() const { return rcvNxt; }
    uint32_t AckBits() const {                              // una+1 .. una+32 的接收位图
        uint32_t bits = 0;
        for (uint32_t i = 1; i ≤ 32 && i < kWnd; ++i) if (slots[(rcvNxt + i) % kWnd]) bits |=
        return bits;
    }
    uint32_t CountUsed() const { uint32_t n = 0; for (auto& s : slots) n += s.has_value(); ret
};
```

8.8 发送端：定时驱动的主循环

可靠 UDP 通常由一个固定周期（例如每 10 ms）调用的 `Update(now)` 驱动：检查超时、处理快速重传、在窗口允许时发送新数据、把 ACK 和数据合并成包发出去。

发送端：超时重传 + 快速重传 + 窗口发送（选择重传）

```
struct Segment { uint32_t sn; uint32_t ts; uint32_t resendAt; uint32_t rto; int xmit = 0; int

struct Sender {
    std::deque<std::string> sendQueue;          // 应用提交、尚未进入窗口的消息
    std::map<uint32_t, Segment> sendBuf;        // 已发送未确认（按 sn 有序）
    uint32_t sndNxt = 0, sndUna = 0, rmtWnd = 128, sndWnd = 64;
    RttEstimator rtt;                           // SRTT / RTTVAR / RTO（RFC 6298）
    int fastResend = 2; double backoff = 1.5;  // KCP 极速模式的取值

    void OnAck(uint32_t una, uint32_t ackBits, uint32_t echoTs, uint32_t now) {
        rtt.Sample(now - echoTs);                                   // 时间戳回显：重传包也能采样
        for (auto it = sendBuf.begin(); it ≠ sendBuf.end() && SeqBefore(it→first, una);) it
        uint32_t maxAcked = una - 1;
        for (int i = 0; i < 32; ++i) if (ackBits >> i & 1) { sendBuf.erase(una + 1 + i); maxAc
        for (auto& [sn, seg] : sendBuf) if (SeqBefore(sn, maxAcked)) seg.fastack++;   // 被更
        sndUna = sendBuf.empty() ? sndNxt : sendBuf.begin()→first;
    }

    void Update(uint32_t now, auto&& output) {
        for (auto& [sn, seg] : sendBuf) {
            bool timeout = int32_t(now - seg.resendAt) ≥ 0;
            bool fast = fastResend > 0 && seg.fastack ≥ fastResend;
            if (!timeout && !fast) continue;
            if (timeout) seg.rto = std::min<uint32_t>(uint32_t(seg.rto * backoff), 10000);
            seg.xmit++; seg.fastack = 0; seg.ts = now; seg.resendAt = now + seg.rto;
            output(seg);
        }
        uint32_t cwnd = std::min(sndWnd, rmtWnd);
        while (!sendQueue.empty() && SeqDiff(sndNxt, sndUna) < int32_t(cwnd)) {
            Segment seg{sndNxt++, now, now + rtt.Rto(), rtt.Rto(), 1, 0, std::move(sendQueue.f
            sendQueue.pop_front();
            output(seg);
            sendBuf.emplace(seg.sn, std::move(seg));
        }
    }
};
```

8.9 拥塞避免：关掉它之前要想清楚

可靠 UDP 的拥塞控制有三种常见取舍：

策略	做法	适用
照搬 TCP	在用户态实现慢启动 / AIMD，或 CUBIC、BBR（QUIC 就是这样）	大流量传输（文件、视频），需要与其他流公平共享
关闭拥塞窗口 + 限速	KCP `nc=1`，再用令牌桶限制最大发送速率；发送窗口仍受接收窗口限制	游戏：流量小（几 KB/s ~ 几十 KB/s）而稳定，远低于瓶颈带宽，丢包多为随机丢包
基于时延的降级	监测 RTT 相对基线的上升（排队在加剧）或丢包率突增，主动降低快照频率 / 精度	游戏、实时音视频（WebRTC 的 GCC 算法就是基于时延梯度）

令牌桶限速：允许短时突发，长期速率不超过 `rate`

```
struct TokenBucket {
    double rate, burst, tokens, last;            // rate：字节/秒；burst：桶容量
    bool TrySend(size_t bytes, double nowSec) {
        tokens = std::min(burst, tokens + (nowSec - last) * rate);
        last = nowSec;
        if (tokens < bytes) return false;        // 令牌不够：本轮先不发（留在队列里）
        tokens -= bytes;
        return true;
    }
};
```

8.10 用带宽换延迟：冗余发送与 FEC

重传至少要付出一个 RTT 的代价。如果能**预先多发一些数据**，让接收方在部分丢包时自己恢复出原文，就可以完全不等重传。

冗余发送（游戏最常用）

每个包除了本帧的输入，还捎带最近 k 帧的输入。只要连续 k+1 个包没有全丢，接收方就能拿到每一帧。独立丢包率为 p 时，残余丢失率约为 p^{k+1}：

FEC 前向纠错

每 n 个数据包额外生成 m 个校验包，收到任意 n 个就能恢复全部。最简单的是异或（m=1）；Reed-Solomon 码可以任意配置 (n, m)。实时音视

p=10%、k=3 时只有万分之一。输入数据很小（几字节到几十字节），冗余几乎不增加带宽。

频（WebRTC 的 FlexFEC / ULPFEC）、kcp-go、加速器广泛使用。

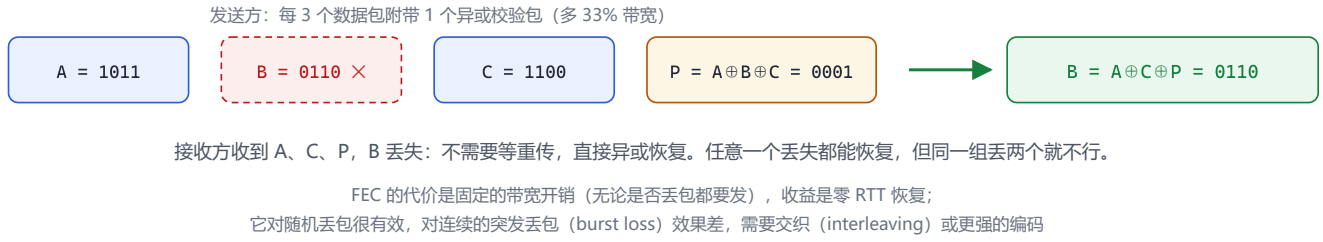


图 8-5 最简单的 FEC：异或校验包可以恢复同组中任意一个丢失的包

8.11 分片、连接管理与安全

分片与重组

大于 MTU 的消息在发送端切片，每片独立编号（占用连续的 sn），并在 `frg` 字段中标注"后面还有几片"。接收端按序收到 `frg = 0` 的那一片时，把前面连续的片拼接成完整消息交给应用。KCP 正是这样做的：在消息模式下，`ikcp_recv` 只有在一条消息的所有分片都到齐后才返回。

连接管理

- **握手**：客户端发 CONNECT；服务器**不分配会话资源**，回一个带 Cookie（如 HMAC(客户端地址, 时间, 密钥)）的 CHALLENGE；客户端带着 Cookie 再发一次，服务器校验通过才分配 conv 和会话。这同时防住了"伪造源地址耗尽会话"（SYN Flood 的 UDP 版）和"反射放大"。服务器的第一次回复不应该比客户端的请求大。
- **会话识别**：用 conv 而不是四元组查找会话；收到同一 conv 但源地址变了的包（NAT 重绑定），验证后更新地址。
- **心跳与断线**：没有数据时定期发 PING（顺便测 RTT、保活 NAT 映射）；N 秒内没有收到对方任何包就判定断线。
- **关闭**：发 CLOSE 并重复几次（它也可能丢），对方收到后释放资源；最终仍以超时兜底。

安全

UDP 包很容易伪造和篡改。面向公网的可靠 UDP 至少需要：**认证加密**（AES-GCM / ChaCha20-Poly1305，或直接用 DTLS），包含序号的 nonce 防**重放**，握手阶段的**密钥协商**（如 ECDH）。KCP 不提供任何安全能力，kcp-go 提供了多种对称加密可选；QUIC 则强制 TLS 1.3。

8.12 实测：仿真对比各种 ARQ 策略

配套程序 `d2_rudp_sim` 用虚拟时钟仿真：链路 1 包/ms、RTT ≈ 60 ms、双向独立丢包 5%、1% 的包"绕路"制造轻微乱序，传输 2000 个包。各方案的协议逻辑与本章代码同构。

🍼 实测证据（d2 • 实验 A）：批量传输 2000 个包

d2_rudp_sim.exe · 实验 A（固定随机种子，可复现）

```
实验 A: 批量传输 2000 个包 | 链路 1 包/ms | RTT≈60ms | 双向丢包 5% | 理论下限≈2060ms
方案                                    完成(ms)  发送次数  重传占比
A1 停等 ARQ（窗口=1）                   171807     2207     9.4%
A2 回退N步 GBN（窗口64）                36407     10036    80.1%
A3 选择重传 SR（窗口64，仅超时重传）      7820      2164     7.6%
A4 SR + 快速重传（阈值3）                 4000      2184     8.4%
T  类TCP：A4+SACK+延迟ACK+RTO×2+Reno      35563      2129     6.1%
T' 类TCP，但关闭拥塞控制                  4046      2197     9.0%
K  类KCP极速：快重传2+立即ACK+RTO×1.5      3609      2844    29.7%
```

逐行解读：

- **A1 停等**：每个包至少等 1 个 RTT，2000 × 61 ms ≈ 122 s，再加上丢包后的超时等待，171 s。和 8.5 节"利用率 1.6%"的计算吻合。
- **A2 GBN**：窗口 64 本应很快，但每丢一个包就要把后面整窗重发一遍，**80% 的发送是重传**，反而只比停等好 4.7 倍。
- **A3 → A4**：同样是选择重传，加上快速重传后耗时减半——绝大多数丢包在约 1 个 RTT 内就被发现，不必等 200 ms 的 RTO。
- **T vs T'**：两者唯一的区别是 Reno 拥塞控制。5% 的随机丢包让 Reno 不停地减半窗口，耗时多了近 9 倍。*（再次提醒：这里的"类 TCP"缺少现代 Linux 的 TLP / RACK / PRR / CUBIC 等优化，绝对值偏悲观，但"随机丢包被误判为拥塞"的方向是真实的。）*
- **K 类 KCP**：最快，但重传占比 29.7%，是 A4 的 3.5 倍——更激进的快速重传阈值和 KCP 式计数带来了大量"其实没丢"的重传。这就是 KCP 官方描述的"**以 10%～20% 的带宽浪费换取更低的延迟**"，这里因为仿真丢包率较高，浪费得更多。

交互实验：自己调参数

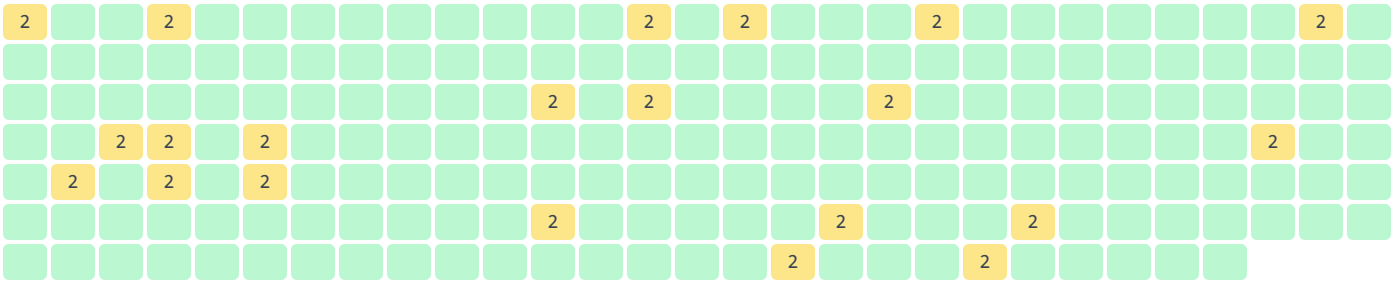
下面的仿真器与 d2 同构（为了在浏览器里秒出结果，只传 200 个包、去掉了延迟确认和拥塞控制）。拖动丢包率、窗口和 RTT，观察各协议的变化；方格颜色表示每个包被发送的次数（绿 1 次、黄 2 次、橙 3 次、红 4 次以上）。

交互实验 2 • ARQ 协议仿真器

协议： SR + 快速重传(3) 丢包率 5%

窗口 32 RTT 60 ms

随机种子 7 换一组随机数



SR + 快速重传(3)：传输 200 个包，耗时 701 ms（同窗口无丢包的理想值约 442 ms，慢 1.6 倍）
提示：窗口 32 < 带宽时延积 ≈ 61 个包，即使不丢包也跑不满链路
发送 221 次，重传 21 次，重传占比 9.5%

同参数下各协议对比：

	停等 ARQ	耗时	17976 ms	发送	222 次	重传占比 9.9%
	回退 N 步 GBN	耗时	2889 ms	发送	490 次	重传占比 59.2%
	选择重传 SR	耗时	1374 ms	发送	219 次	重传占比 8.7%
▶	SR + 快速重传(3)	耗时	701 ms	发送	221 次	重传占比 9.5%
	类 KCP 极速模式	耗时	644 ms	发送	281 次	重传占比 28.8%

建议做这几个小实验：① 丢包率调到 0，比较停等和 SR 的耗时，体会窗口的作用；② 窗口调到 1，五种协议几乎一样，说明它们的差别全部来自"窗口内的丢包怎么处理"；③ 丢包率调到 20%，看 GBN 的红色方格；④ RTT 调到 300 ms，看快速重传相对于超时重传的优势如何放大。

本章小结

- 报文头：conv（会话 ID）、sn、una、ack_bits、ts、wnd、chan、frg；**按包编号**，ACK 捎带在数据包里。
- 序号比较用 `int32_t(a - b) > 0`；窗口必须小于序号空间的一半（SR 要求序号空间 ≥ 2N）。
- 确认 = UNA（累计，抗 ACK 丢失）+ 逐包 ACK 或位图（选择确认，避免多余重传）。
- 时间戳回显测 RTT，避免重传歧义；RTO 下限、退避倍数、快速重传阈值都可以按业务调。
- 停等利用率 ≈ T/(T+RTT)；GBN 丢一个重发一窗；**SR 只重发丢失的包**，是现代协议的共同选择。
- 接收端环形缓冲：乱序暂存、按序交付、重复检测（重复包也要回 ACK）。
- 拥塞：照搬 TCP / 关窗 + 令牌桶限速 / 基于时延降级；冗余发送与 FEC 用带宽换零 RTT 恢复。
- 连接管理：Cookie 握手防伪造与放大、conv 识别会话、心跳、超时断线；公网必须加密认证。
- 仿真：SR + 快速重传比 SR 快一倍；GBN 80% 都是重传；随机丢包下 Reno 慢近 9 倍；类 KCP 最快但多花约 30% 带宽。

▶ **思考题 1：为什么选择重传要求序号空间至少是窗口的 2 倍?**

▶ **思考题 2：为什么接收方收到重复包时还要回 ACK?**

▶ **思考题 3：帧同步游戏中，"每个包冗余携带最近 3 帧输入"和"用 KCP 可靠传输每帧输入"相比，各有什么优缺点?**

▶ **思考题 4：如果你的可靠 UDP 只用 UNA 而不用任何选择确认，会退化成哪种 ARQ?**

CHAPTER 09

KCP：以带宽换延迟的 ARQ 协议

一千多行 C 代码、一个头文件、零系统调用。它的全部秘密就在"对 TCP 的六处修改"里。

本章目标

- 说出 KCP 的定位（纯算法、无 IO、无连接管理）与设计目标
- 画出 KCP 的四个队列和数据流，背出 24 字节报文头
- 逐条说清 KCP 相对 TCP 的改动：RTO 不翻倍、选择重传、快速重传、非延迟 ACK、UNA+ACK、非退让流控
- 会写 KCP 的接入代码，理解 `ikcp_nodelay` 等参数的含义与调优
- 能用仿真数据解释 KCP 的"带宽换延迟"，并知道在工程中 KCP 之外还要补什么

9.1 KCP 是什么

KCP 是由林伟（skywind3000）开源的一个快速可靠传输协议，项目 README 中对它的描述是：**能以比 TCP 浪费 10%～20% 带宽的代价，换取平均延迟降低 30%～40%、最大延迟降低三倍的传输效果。**它在手游、实时音视频、网络加速器等领域被广泛使用，也衍生出 kcp-go、KCP-Net 等多语言实现。

纯算法实现

KCP 不负责收发 UDP，使用者通过回调函数 `output` 告诉它"怎么把一个包发出去"，收到包后调用 `ikcp_input` 喂给它。连时钟都由外部传入，内部没有任何系统调用——所以能嵌入任何引擎、平台和 IO 模型。

只做 ARQ

对应图 7-2 中的第 ③ 层（外加消息分片）：序号、确认、重传、窗口、拥塞控制。**不包含**连接建立、会话管理、心跳、加密、FEC，这些都要使用者自己补。

流速优先，而非流量优先

TCP 的设计目标是充分利用带宽（每秒传多少 KB）；KCP 的目标是降低单个包从发出到交付的时间（一个包花多少毫秒）。两者的取舍方向不同。

9.2 架构：四个队列与时钟驱动

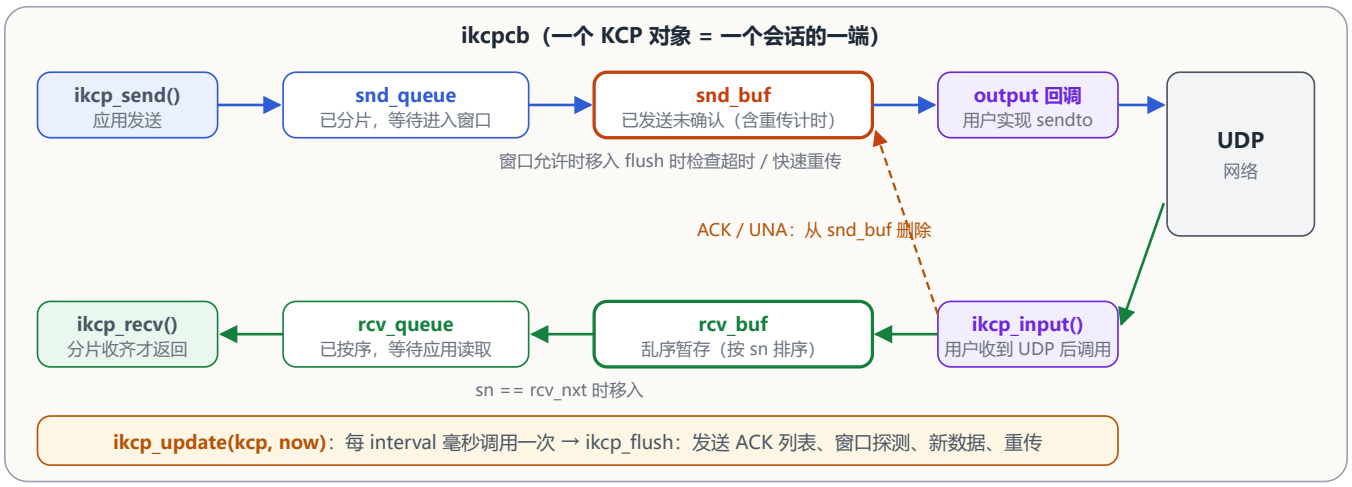


图 9-1 KCP 的四个队列：发送侧 snd_queue → snd_buf，接收侧 rcv_buf → rcv_queue；一切发送都在 update / flush 中完成

24 字节报文头



图 9-2 KCP 报文头（IKCP_OVERHEAD = 24 字节）。多个 KCP 段可以拼进同一个 UDP 包；ACK 段的 sn 是被确认的包序号、ts 是原包的时间戳

9.3 KCP 对 TCP 的六处改动

KCP 的 README 用"vs TCP"的方式列出了它的关键设计，每一条都和前面讲过的 TCP 机制一一对应：

#	TCP 的做法	KCP 的做法	效果 / 代价
1	**RTO 翻倍**：连续超时 3 次，等待时间变成 RTO × 8	nodelay 模式下每次超时 **RTO × 1.5**；RTO 下限从 100 ms 降为 30 ms	连续丢包时等待时间增长慢得多（见图 9-3）；代价是严重拥塞时退让不足
2	（早期 / 无 SACK）丢包后可能重传丢失点之后的所有数据	**选择性重传**：只重传真正丢失的包	减少无效重传
3	3 个重复 ACK 触发快速重传	**被跳过 resend 次即重传**（极速模式 resend=2）：发送 1～5，收到 ACK 1、3、4 时，2 已被跳过 2 次，立即重传	更早修复丢包；代价是轻微乱序也会触发重传
4	**延迟确认**（最多数百毫秒）	ACK 不刻意延迟，在下一次 flush（≤ interval，极速模式 10 ms）时发出	RTT 测量更准、重传更及时；代价是 ACK 包更多
5	只有 UNA（累计确认），SACK 是可选扩展	**UNA + 逐包 ACK**：每个包都带 una，同时有单独的 ACK 段	ACK 丢失时 UNA 兜底，逐包 ACK 保证精确
6	公平退让：发送窗口受拥塞窗口约束，丢包即减窗	**非退让流控**（nc=1）：发送窗口只由发送缓冲和对端接收窗口决定，跳过慢启动和丢包退让	小流量实时数据不会因随机丢包降速；代价是失去对网络的"礼让"

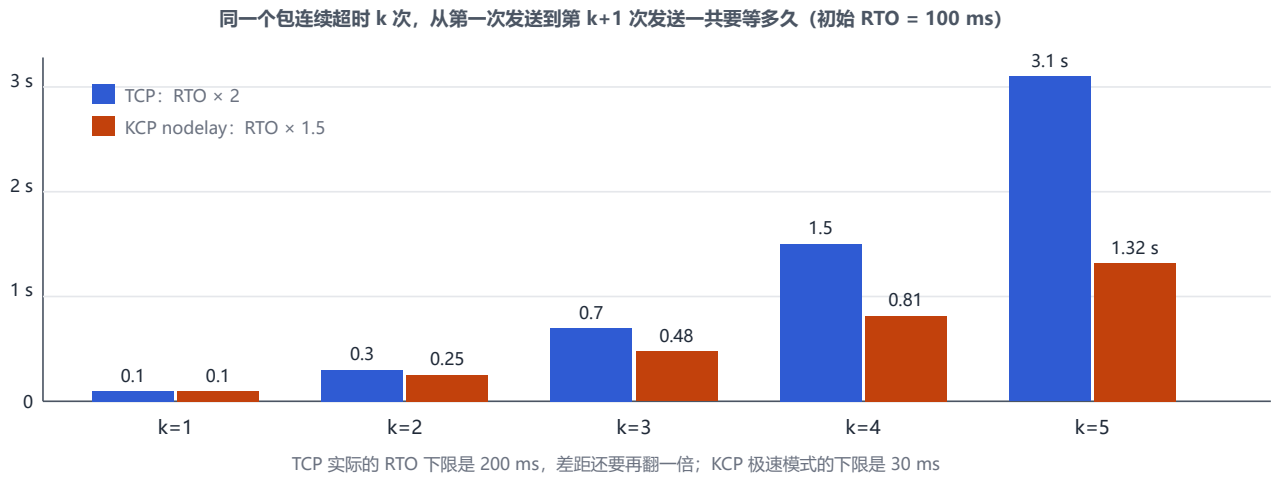


图 9-3 指数退避的累计等待：连续 5 次超时后，×2 要等 3.1 s，×1.5 只要 1.32 s

9.4 接入代码

KCP 接入骨架（客户端，单会话；省略错误处理）

```
#include "ikcp.h"

static int UdpOutput(const char* buf, int len, ikcpcb* kcp, void* user) {
    auto* s = static_cast<Session*>(user);
    return sendto(s->sock, buf, len, 0, (sockaddr*)&s->peer, sizeof s->peer);    // KCP 只管"要
}

void Init(Session* s, uint32_t conv) {
    s->kcp = ikcp_create(conv, s);            // conv 双方必须一致（通常由服务器在登录时分配）
    s->kcp->output = UdpOutput;
    ikcp_nodelay(s->kcp, 1, 10, 2, 1);        // 极速模式：nodelay=1, interval=10ms, resend=2, r
    ikcp_wndsize(s->kcp, 128, 128);           // 发送 / 接收窗口（默认 32 / 128）
    ikcp_setmtu(s->kcp, 1200);                // 默认 1400；移动网络建议更保守
    s->kcp->rx_minrto = 10;                   // 可选：进一步降低 RTO 下限
}

void Tick(Session* s, uint32_t nowMs) {       // 游戏主循环里每帧调用
    char buf[1500];
    for (int n; (n = recvfrom(s->sock, buf, sizeof buf, 0, nullptr, nullptr)) > 0;)
        ikcp_input(s->kcp, buf, n);           // ① 把收到的 UDP 包喂给 KCP（非阻塞 socket）
    ikcp_update(s->kcp, nowMs);               // ② 驱动时钟：发 ACK、发新数据、检查重传
    for (int n; (n = ikcp_recv(s->kcp, buf, sizeof buf)) > 0;)
        OnMessage(buf, n);                    // ③ 取出按序、完整的消息
}

void Send(Session* s, const void* msg, int len) {
    ikcp_send(s->kcp, (const char*)msg, len);    // 放入 snd_queue，下一次 update/flush 时才真正
    // 想更低延迟：ikcp_flush(s->kcp) 立即发送（会多一些小包）
}
```

🔬 **ikcp_check：服务器上省 CPU 的关键**

服务器上可能有上万个 KCP 会话，每 10 ms 对每个会话调用一次 `ikcp_update` 很浪费。`ikcp_check(kcp, now)` 返回"下一次真正需要调用 update 的时刻"（最近的重传时间或下一个 interval），服务器可以把会话按这个时间放进定时器堆或时间轮，只在需要时调度；收到包或有新消息要发时再立即调度一次。

9.5 参数详解与调优

参数	默认	极速模式推荐	含义与调优
`nodelay`	0	1	开启后：RTO 下限 30 ms（`IKCP_RTO_NDL`），超时退避 ×1.5。否则下限 100 ms、退避约 ×2
`interval`	100 ms	10 ~ 20 ms	内部 flush 的周期，决定 ACK 和重传的时间精度；越小延迟越低、CPU 越高
`resend`	0（关闭）	2	快速重传阈值。设得太小（1）时对乱序过于敏感
`nc`	0	1	1 = 关闭拥塞控制（非退让流控）。**需要配合应用层限速**
`sndwnd / rcvwnd`	32 / 128	按带宽时延积估算	窗口太小会限速（每 RTT 最多发 sndwnd 个包）；rcvwnd 应 ≥ sndwnd，且大于最大分片数
`mtu`	1400	1200 ~ 1400	单个 UDP 包的最大长度（含 24 字节头），超过会被分片
`rx_minrto`	100 / 30	10 ~ 30	RTO 下限；局域网或 RTT 稳定时可以调低
`fastlimit`	5	—	一个包最多被快速重传几次，超过后只能等超时
`dead_link`	20	—	一个包被重传这么多次仍未确认，`kcp->state` 置为 −1，可用于判定断线
`stream`	0	按需	1 = 流模式：像 TCP 一样不保留消息边界，可以把小消息合并进一个段，减少开销

README 给出的两组典型配置：普通模式 `ikcp_nodelay(kcp, 0, 40, 0, 0)`，极速模式 `ikcp_nodelay(kcp, 1, 10, 2, 1)`。

9.6 实测：100 Hz 实时消息流下的时延

批量传输看的是"多久传完"，实时应用更关心"每条消息花了多少毫秒"。`d2` 的实验 B 模拟一个每 10 ms 产生一条消息（100 Hz，接近游戏的输入 / 状态发送频率）的流，持续 10 秒，测量每条消息从产生到**按序交付**给应用的时延。

🧪 实测证据（d2 • 实验 B）：时延分布

d2_rudp_sim.exe · 实验 B（RTT≈60ms，单向基础时延 30 ms）

丢包	方案	平均ms	P99ms	最大ms	发送	丢失
0%	T 类TCP：A4+SACK+延迟ACK+RTO×2+Reno	31.0	33	39	1000	0
0%	T' 类TCP，但关闭拥塞控制	31.0	33	39	1000	0
0%	K 类KCP极速：快重传2+立即ACK+RTO×1.5	31.0	31	40	1011	0
0%	R 冗余发送k=3（不可靠）	31.0	33	40	1000	0
5%	T 类TCP：A4+SACK+延迟ACK+RTO×2+Reno	5085.4	10430	10676	1052	0
5%	T' 类TCP，但关闭拥塞控制	60.7	185	215	1077	0
5%	K 类KCP极速：快重传2+立即ACK+RTO×1.5	42.9	95	167	1249	0
5%	R 冗余发送k=3（不可靠）	31.6	41	51	1000	1
10%	T' 类TCP，但关闭拥塞控制	73.8	213	225	1133	0
10%	K 类KCP极速：快重传2+立即ACK+RTO×1.5	53.4	123	173	1474	0
20%	T' 类TCP，但关闭拥塞控制	150.0	513	603	1393	0
20%	K 类KCP极速：快重传2+立即ACK+RTO×1.5	72.9	163	183	1907	0

（完整输出见 demos/d2_rudp_sim.out.txt；T 在 10%、20% 丢包下时延达数十秒，R 的结果在第 11 章讨论）

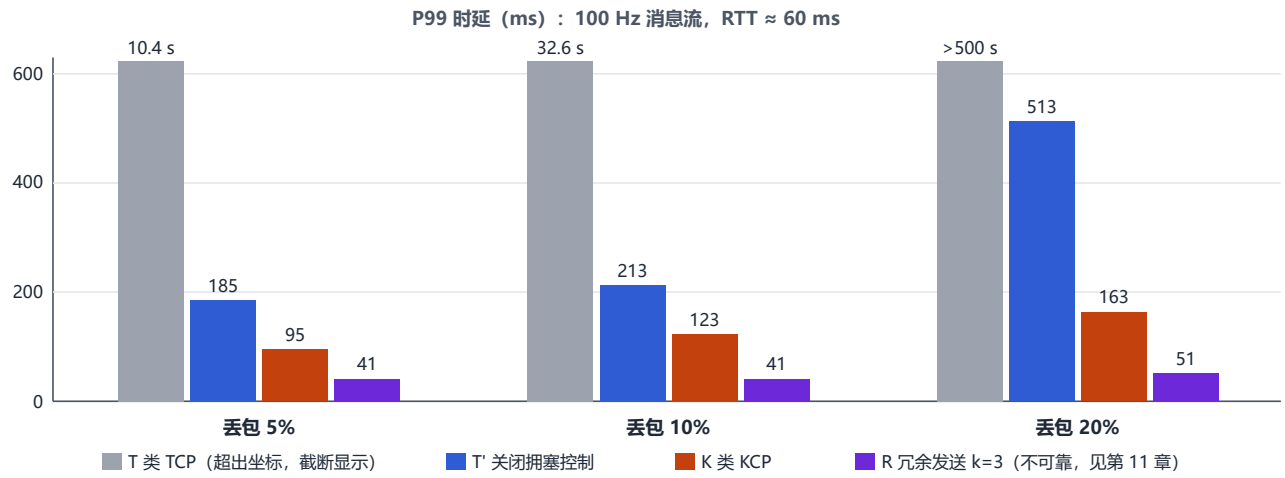


图 9-4 实验 B 的 P99 时延：拥塞控制是 T 崩溃的主因；在去掉拥塞控制之后，KCP 的激进重传仍把 P99 再压低 40% ~ 70%

从这组数据可以读出三个层次的结论：

1. **T → T'**：随机丢包下，基于丢包的拥塞控制把发送速率压到了应用速率（100 条/秒）以下，消息在发送队列里越堆越多，时延滚雪球般增长到十几秒。这对应 Mathis 公式：Reno 的吞吐上限约为 `MSS /`

`RTT × 1.22 / √p`，RTT 60 ms、p = 5% 时约 91 包/秒，恰好低于 100 Hz。*真实 Linux 有更多优化，不会这么极端，但"随机丢包被误判为拥塞"的问题确实存在。*

2. **T' → K**：同样没有拥塞控制，KCP 式参数（快重传 2、RTO 下限 30 ms、×1.5 退避、立即 ACK）把 P99 从 185/213/513 ms 压到 95/123/163 ms。
3. **代价**：K 的发送次数是 1249/1474/1907，比 T' 多 16%～37%——这就是 KCP 所说的"带宽浪费"。而在 0% 丢包时它也多发了 11 个包（轻微乱序触发的伪快速重传）。

9.7 工程实践：KCP 之外还要补什么

KCP 不提供的	常见做法
连接建立与 conv 分配	先用 TCP / HTTP 登录拿到 conv 和密钥，再切到 UDP；或者自己实现带 Cookie 的 UDP 握手
会话识别	服务器用 conv 查会话（`ikcp_getconv` 可以从包头取出 conv），不依赖源地址，可支持 NAT 换端口
心跳与断线	应用层心跳；监控 `kcp->state == -1`（dead_link）；N 秒无包判定断线
加密与防篡改	在 output / input 处加一层 AEAD 加密；kcp-go 内置了多种加密
FEC	kcp-go 内置 Reed-Solomon FEC；或自己实现异或 FEC
不可靠消息	KCP 只有可靠有序一种语义；状态同步等不可靠数据**绕过 KCP 直接走 UDP**，在同一个 socket 上用首字节区分
TCP 兜底	UDP 不通时自动降级到 TCP（KCP over TCP 没有意义，直接用 TCP 传消息即可）

> ⚠️ **易错点：KCP 不是"更快的 TCP"**
>
> KCP 降低的是**丢包场景下的时延和抖动**，代价是更多的带宽和 CPU。在不丢包的网络上，KCP 和 TCP 的时延几乎一样（实验 B 的 0% 行）；在大文件下载这类追求吞吐的场景，关闭拥塞控制的 KCP 可能挤占带宽、造成更多丢包，反而得不偿失。**用 KCP 之前先确认：你的问题是"延迟抖动"而不是"带宽不够"。**

✅ 面试答法：KCP 为什么比 TCP 快?

KCP 是一个运行在 UDP 之上的 ARQ 协议，"快"指的是丢包时延迟更低。它对 TCP 做了几处改动：① 超时重传时 RTO 只乘 1.5 而不是翻倍，RTO 下限也更低；② 选择性重传，只重传丢失的包；③ 快速重传阈值可配（比如被跳过 2 次就重传），不必等 3 个重复 ACK；④ ACK 不做延迟；⑤ 同时使用 UNA 和逐包 ACK；⑥ 可以关闭拥塞控制，避免随机丢包导致降速。代价是 10%～20% 甚至更多的额外带宽，以及需要自己处理连接管理、加密和不可靠消息。

本章小结

- KCP：纯算法 ARQ 库，无 IO、无系统调用；只做可靠传输层，不含连接、加密、FEC。
- 四个队列：snd_queue → snd_buf → output；input → rcv_buf → rcv_queue；由 `ikcp_update` 时钟驱动。
- 24 字节头：conv、cmd（PUSH/ACK/WASK/WINS）、frg、wnd、ts、sn、una、len。
- 六处改动：**RTO×1.5、选择重传、快速重传（resend）、非延迟 ACK、UNA+ACK、非退让流控（nc）**。
- 极速模式 `ikcp_nodelay(kcp, 1, 10, 2, 1)`；窗口默认 32/128，MTU 1400，RTO 下限 100/30 ms。
- 仿真：随机丢包下拥塞控制是 TCP 时延崩溃的主因；KCP 在此基础上把 P99 再降 40%～70%，多花 16%～37% 带宽。

▶ **思考题 1：KCP 关闭拥塞控制后，会不会把网络挤爆？应该怎么做?**

▶ **思考题 2：KCP 的 interval 设为 100 ms 和 10 ms，对延迟有什么影响?**

▶ **思考题 3：为什么 KCP 的接收窗口默认比发送窗口大（128 vs 32）？**

CHAPTER 10

QUIC：把 TCP + TLS 搬到用户态重写一遍

KCP 是一个 ARQ 算法，QUIC 是一个完整的、标准化的、加密的传输协议，也是 HTTP/3 的底座。

本章目标

- 说清 QUIC 的分层位置和它要解决的四个问题
- 理解 1-RTT / 0-RTT 握手、连接 ID 与连接迁移
- 理解 QUIC 的流为什么没有队头阻塞，以及"包号与流偏移分离"如何消除重传歧义
- 掌握 QUIC 的 ACK 帧、丢包检测（包阈值 / 时间阈值 / PTO）、两级流量控制、可插拔拥塞控制
- 知道 QUIC 的安全设计（抗放大、Retry、头部保护）与部署代价

10.1 背景与定位

QUIC 最早由 Google 于 2012 年前后提出并在 Chrome 中部署（后称 gQUIC），之后由 IETF 标准化：**RFC 9000（传输协议）**、**RFC 9001（与 TLS 1.3 的结合）**、**RFC 9002（丢包检测与拥塞控制）**于 2021 年发布，基于 QUIC 的 **HTTP/3（RFC 9114）**于 2022 年发布。它要解决的正是第 4、7 章列出的 TCP 痛点：

① 握手太慢

TCP 握手 1 RTT + TLS 1～2 RTT → QUIC 把传输握手和加密握手合并为 1 RTT，恢复连接时 0 RTT。

② 队头阻塞

HTTP/2 的多个流共享一个 TCP 字节流 → QUIC 原生支持多个独立的流，丢包只影响所在的流。

③ 换网断连

TCP 连接绑定四元组 → QUIC 用连接 ID 标识连接，IP / 端口变了连接仍在。

④ 协议僵化

TCP 在内核里、头部明文、被中间设备"看懂"并干预 → QUIC 在用户态实现，几乎所有头部字段都加

密，可以快速迭代。

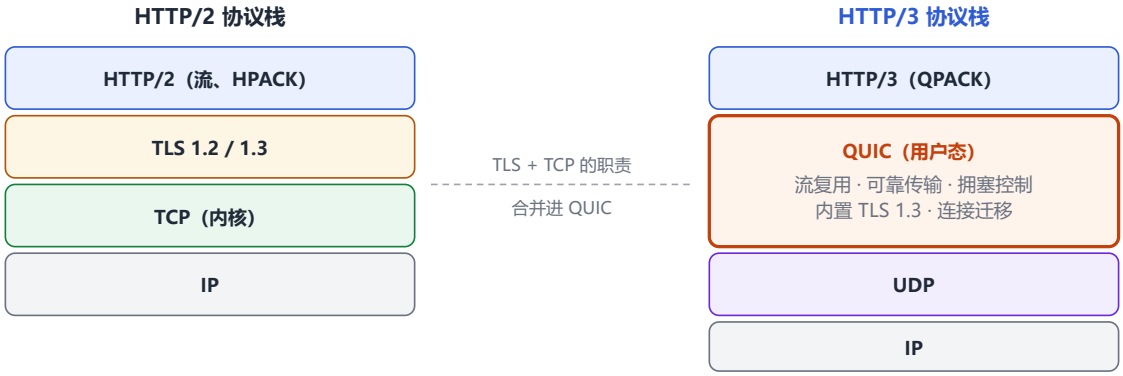


图 10-1 QUIC 把 TCP 的可靠传输、TLS 的加密和 HTTP/2 的流复用合并成一层，运行在 UDP 之上

10.2 握手：1-RTT 与 0-RTT

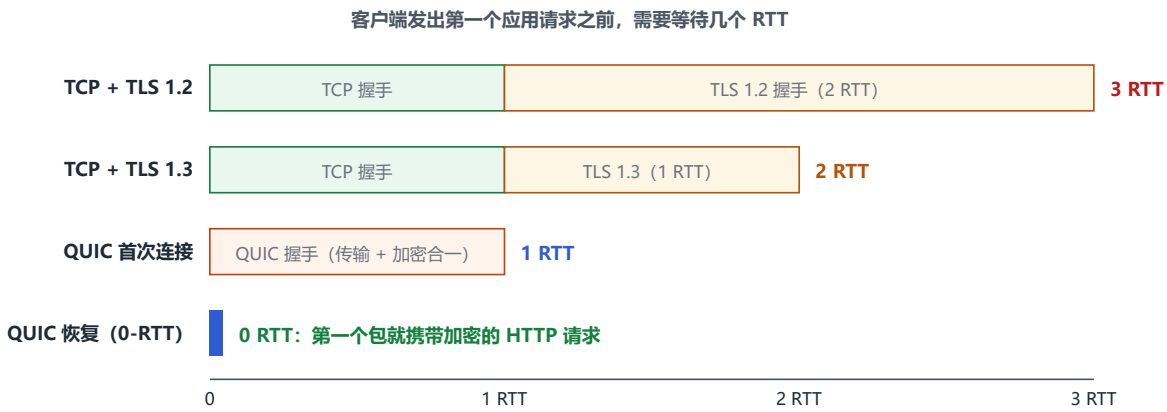


图 10-2 建连开销对比：冷启动 QUIC 比 TCP+TLS1.3 少 1 个 RTT；恢复连接时可以 0-RTT

- **1-RTT**：客户端的第一个包（Initial）就包含 TLS 1.3 的 ClientHello 和 QUIC 的传输参数（初始流控窗口、最大流数量等），服务器一次回复就完成密钥协商，客户端随即可以发送应用数据。传输握手和加密握手共用同一个往返。
- **0-RTT**：客户端曾经连过这台服务器，保存了会话票据（session ticket），再次连接时可以用它派生的密钥直接加密应用数据，和 Initial 一起发出。

> ⚠️ **易错点：0-RTT 数据可以被重放**
>
> 0-RTT 数据在服务器回复之前就发出了，不具备前向安全性，而且攻击者截获后可以原样重放给服务器。所以 0-RTT 只能用于**幂等请求**（如 GET），服务器端通常也会限制 0-RTT 能做的操作。这和 TCP Fast Open 面临的问题本质相同。

10.3 连接 ID 与连接迁移

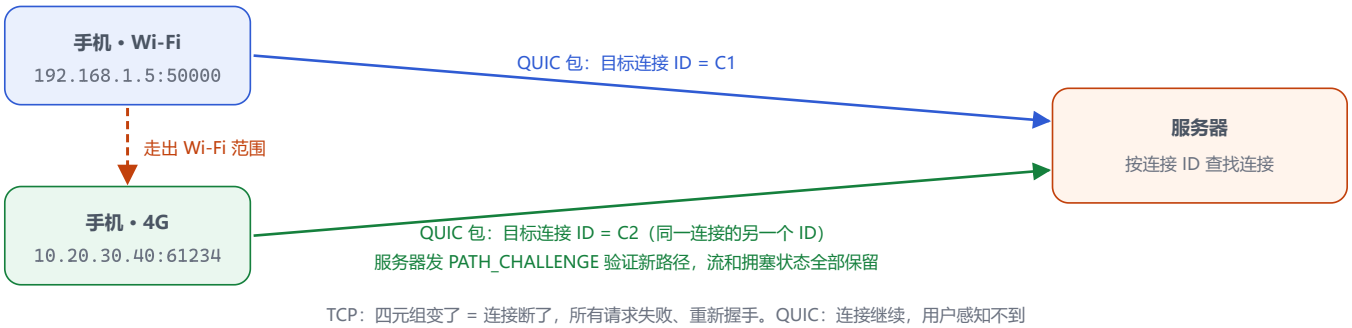


图 10-3 连接迁移：QUIC 连接由连接 ID 标识，换网后用新的连接 ID 继续通信（换 ID 是为了防止旁观者把新旧路径关联起来）

这和第 8 章自研协议用 `conv` 识别会话是同一个思想。QUIC 做得更完整：双方可以通过 `NEW_CONNECTION_ID` 帧提前交换多个连接 ID，迁移时换用新 ID 以保护隐私；切换到新路径时必须先用 `PATH_CHALLENGE / PATH_RESPONSE` 验证对方确实在新地址上（防止被利用来攻击第三方），并重新评估这条路径的 RTT 和拥塞窗口。

10.4 流：没有队头阻塞的多路复用

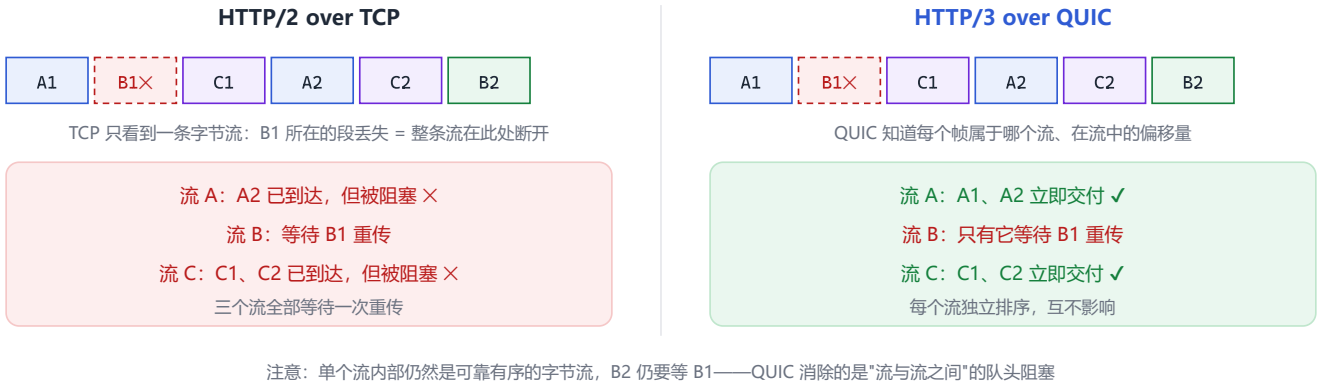


图 10-4 同样丢了一个包：TCP 让三个流一起等，QUIC 只让受影响的流等

QUIC 的流用 62 位的流 ID 标识，最低两位表示由哪一方发起、是双向还是单向。HTTP/3 中每个请求 / 响应占用一个双向流。流的开销很小，开新流不需要握手，这让 HTTP/3 可以随意并发请求。

10.5 可靠性设计：包号与流偏移分离

这是 QUIC 相对 TCP 最精巧的设计之一。TCP 用同一个序号同时表示"这是哪个包"和"这是字节流的哪个位置"，所以重传的段和原段序号相同，ACK 到达时分不清确认的是哪一次发送（第 3 章的重传歧义）。QUIC 把两者拆开：



- **包号**（Packet Number）：每发一个包加 1，**永不重复**，只用于确认、丢包检测和 RTT 测量 → 不存在重传歧义，每个 ACK 都能采样 RTT
- **流偏移**（Stream Offset）：帧中的数据在所属流中的字节位置，用于接收端重组 → 重传的数据可以放进任何新包，甚至和别的帧拼在一起
- QUIC 重传的是"**帧中承载的信息**"而不是"包"：丢失的 ACK 帧不重传（会发新的），丢失的 MAX_DATA 帧重传时发送**最新值**，而不是旧值
- ACK 帧最多可以携带很多个 ACK Range（远多于 TCP SACK 的 3～4 个块），还带有 ACK Delay 字段（接收方故意延迟了多久才确认），RTT 估计更准确
- 三个包号空间：Initial、Handshake、1-RTT（应用数据）各自独立编号与确认

图 10-5 包号只增不减、与流偏移分离：重传 = 把丢失的帧放进新包号的包里

丢包检测（RFC 9002）

机制	规则	对应 TCP 的
包阈值	某个包之后又有 **3 个**更大包号的包被确认（kPacketThreshold = 3），判定它丢失	3 个重复 ACK
时间阈值	某个包发出后超过 **9/8 × max(SRTT, 最新 RTT)** 仍未被确认，而它之后发送的包已被确认，判定丢失	RACK（RFC 8985）
PTO 探测超时	一段时间（SRTT + max(4×RTTVAR, 1ms) + max_ack_delay）内收不到任何确认时，**发送 1～2 个探测包**，而不是直接判定丢失	RTO + TLP；但 PTO 触发时**不会把拥塞窗口降到 1**，只有持续拥塞时才大幅降窗

10.6 流量控制与拥塞控制

- **两级流量控制**：每个流有自己的接收上限（`MAX_STREAM_DATA`），整个连接还有一个总上限（`MAX_DATA`）；另外用 `MAX_STREAMS` 限制对方能同时打开多少个流。这样一个"慢流"不会吃光整个连接的缓冲。
- **可插拔拥塞控制**：RFC 9002 给出了一个类似 NewReno 的参考算法，但实现可以自由替换为 CUBIC、BBR 等。因为在用户态，**升级拥塞算法只需要更新应用**，不需要等操作系统——这正是 Google 能在 QUIC 上快速迭代 BBR 的原因。
- **Pacing**：推荐按速率平滑发包，而不是一次性把整个窗口突发出去，减少路由器队列压力。

10.7 安全设计

几乎全部加密

除了少量必要字段（标志位、连接 ID），QUIC 头部和所有帧都被加密；连包号也用"头部保护"加密了。中间设备看不到、也就改不了 QUIC 的内部字段——这是对抗协议僵化的主动设计。

抗放大攻击

在验证客户端地址之前，服务器发送的数据**不得超过收到数据的 3 倍**；客户端的 Initial 包必须填充到至少 **1200 字节**，既保证了路径 MTU，也让放大倍数可控。

Retry 令牌

服务器可以先回一个 Retry 包，要求客户端带着令牌重新发送 Initial，证明自己真的拥有这个源地址

——这就是 SYN Cookie 思想在 QUIC 中的体现（第 5 章）。

10.8 代价与现状

- **CPU 开销**：用户态协议栈、每包加解密、缺少成熟的硬件卸载，服务器 CPU 消耗通常高于 TCP + TLS。业界通过 UDP GSO / GRO、`sendmmsg`、内核旁路等手段优化。
- **UDP 被限制**：部分企业网络封禁 UDP 443，部分运营商对 UDP 限速。浏览器会通过 `Alt-Svc` 头部发现服务器支持 HTTP/3，并在 QUIC 失败时**自动回退到 TCP**。
- **实现复杂**：一个完整的 QUIC 实现动辄数万行代码。主流开源实现有 Chromium QUIC、Cloudflare quiche、Microsoft MsQuic、ngtcp2、quic-go、LiteSpeed lsquic 等。
- **扩展**：RFC 9221 定义了 **DATAGRAM 帧**，在 QUIC 连接中发送不可靠、不重传的数据（共享加密和拥塞控制），使 QUIC 也能承载"不可靠 · 只要最新"类的实时数据（WebTransport 就基于此）。

10.9 QUIC vs TCP vs KCP

	TCP (+TLS)	KCP	QUIC
定位	内核传输协议	用户态 ARQ 算法库	用户态完整传输协议（IETF 标准）
建连	1 RTT + TLS 1～2 RTT	无（自行实现）	1 RTT，恢复时 0 RTT
连接标识	四元组	conv	连接 ID，支持迁移
多路复用	无（单字节流）	无（单消息流，需多个 KCP 对象）	原生多流，流间无队头阻塞
序号	字节序号，重传序号不变	包序号，重传 sn 不变（靠 ts 测 RTT）	包号永增 + 流偏移
确认	累计 ACK + SACK（3～4 块）	UNA + 逐包 ACK	ACK 帧（多 Range + ACK Delay）
重传策略	RTO×2，3 dup ACK，RACK/TLP	RTO×1.5，跳过 N 次快重传	包阈值 3 / 时间阈值 9/8 RTT / PTO
拥塞控制	CUBIC / BBR（内核）	类 Reno，可关闭	可插拔（NewReno / CUBIC / BBR）
加密	TLS（可选，头部明文）	无	强制 TLS 1.3，头部也加密
不可靠数据	不支持	不支持（需绕过 KCP 发裸 UDP）	DATAGRAM 扩展
典型场景	几乎所有可靠传输	游戏、加速器、实时信令	HTTP/3、移动 App、WebTransport

✅ 面试答法：QUIC 相比 TCP 有哪些改进?

① **握手更快**：传输和 TLS 1.3 握手合并，首次 1 RTT，恢复连接 0 RTT；② **无队头阻塞**：原生多流，每个流独立排序，一个丢包只影响所在的流；③ **连接迁移**：用连接 ID 而不是四元组标识连接，切换网络不断线；④ **更好的丢包恢复**：包号单调递增、与流偏移分离，消除重传歧义，ACK 帧可以携带大量范围和 ACK 延迟；⑤ **用户态实现**：拥塞控制可插拔、迭代快，且头部加密防止中间设备僵化；⑥ 内置抗放大和地址验证。代价是 CPU 开销更高，以及部分网络对 UDP 的限制需要回退 TCP。

本章小结

- QUIC：RFC 9000/9001/9002（2021），HTTP/3 RFC 9114（2022）；= 可靠传输 + TLS 1.3 + 流复用，跑在 UDP 上。
- 握手 1-RTT，恢复 0-RTT（0-RTT 数据可被重放，只用于幂等请求）。
- 连接 ID 标识连接 → 连接迁移；迁移需 PATH_CHALLENGE 验证，换用新 ID 保护隐私。
- 多流复用：流间无队头阻塞，流内仍有序。
- **包号永增、与流偏移分离** → 无重传歧义；重传的是帧不是包；丢包检测：包阈值 3、时间阈值 9/8 RTT、PTO。
- 两级流控（MAX_STREAM_DATA / MAX_DATA）；拥塞控制可插拔；抗放大 3 倍、Initial ≥ 1200 字节、Retry 令牌。
- 代价：CPU 高、UDP 可能被封需回退 TCP、实现复杂；DATAGRAM 扩展支持不可靠数据。

▶ **思考题 1：HTTP/3 一定比 HTTP/2 快吗?**

▶ **思考题 2：为什么说 QUIC 的"包号永不重复"解决了 TCP 的重传歧义？KCP 是怎么解决的?**

▶ **思考题 3：QUIC 运行在用户态，那它还能利用内核的拥塞控制吗？这对公平性有什么影响?**

PART FIVE

第五部分　实战与面试：把知识串成网

把前面所有机制放回真实场景：一个 MOBA 游戏的网络层怎么设计？视频会议为什么不怕丢包？然后用一张横向对照表把 TCP、KCP、QUIC 串起来，最后是面试题库与速记卡。

CHAPTER 11

场景选型与游戏网络实战

同一个问题，在 TCP、KCP、QUIC、游戏引擎里各有各的答案。看懂这些答案之间的对应关系，知识才算连成了网。

本章目标

- 掌握主流传输方案的能力对比，能为具体业务选型
- 能设计一个帧同步 MOBA 的网络层，并用仿真数据论证"冗余发送 + 可靠兜底"
- 理解状态同步、实时音视频、网页加载三类场景的传输策略
- 知道如何做弱网测试，以及用哪些指标衡量传输质量
- 用一张横向对照表把全书的机制串起来

11.1 方案全景

方案	可靠性模式	多路 / 通道	拥塞控制	加密	连接管理	典型使用
TCP	可靠有序	无	有（内核）	需 TLS	有	绝大多数业务、回合制游戏
裸 UDP	不可靠	应用自定	无	无	无	DNS、遥测、状态广播
KCP	可靠有序	无（多会话需多个对象）	可关闭	无	无	实时游戏可靠通道、加速器
ENet	可靠 / 不可靠 / 不可靠有序（sequenced）	多通道	带宽节流	无	有	独立游戏、中小型联机游戏
WebRTC DataChannel	可配：有序 / 无序，可靠 / 部分可靠（最大重传次数或最大生存时间）	多流	有	DTLS	有（ICE 打洞）	浏览器 P2P 数据、云游戏输入
QUIC	流：可靠有序；DATAGRAM：不可靠	多流	有（可插拔）	TLS 1.3 强制	有 + 连接迁移	HTTP/3、移动 App
游戏引擎网络层（如 UE IpNetDriver）	可靠 RPC / 不可靠 RPC / 属性复制（只要最新）	按 Actor 通道	按带宽预算与优先级调度	可选	有	引擎内置的多人游戏

WebRTC DataChannel 的"部分可靠"值得一提：它允许指定"最多重传 N 次"或"超过 T 毫秒就放弃"，正好落在第 7 章可靠性光谱的中间地带，非常适合"晚到一点可以接受、太晚就没用了"的数据。

11.2 实战一：帧同步 MOBA 的网络层设计

帧同步（Lockstep）的核心思想是：服务器不计算游戏逻辑，只负责收集所有玩家每个逻辑帧的操作并转发；每个客户端用相同的初始状态、相同的操作序列、确定性的逻辑，独立算出一致的结果。它对网络的

要求是：**每一帧的操作必须送达且按帧序执行**（可靠有序），但又必须**低延迟、低抖动**，否则所有玩家都会卡。

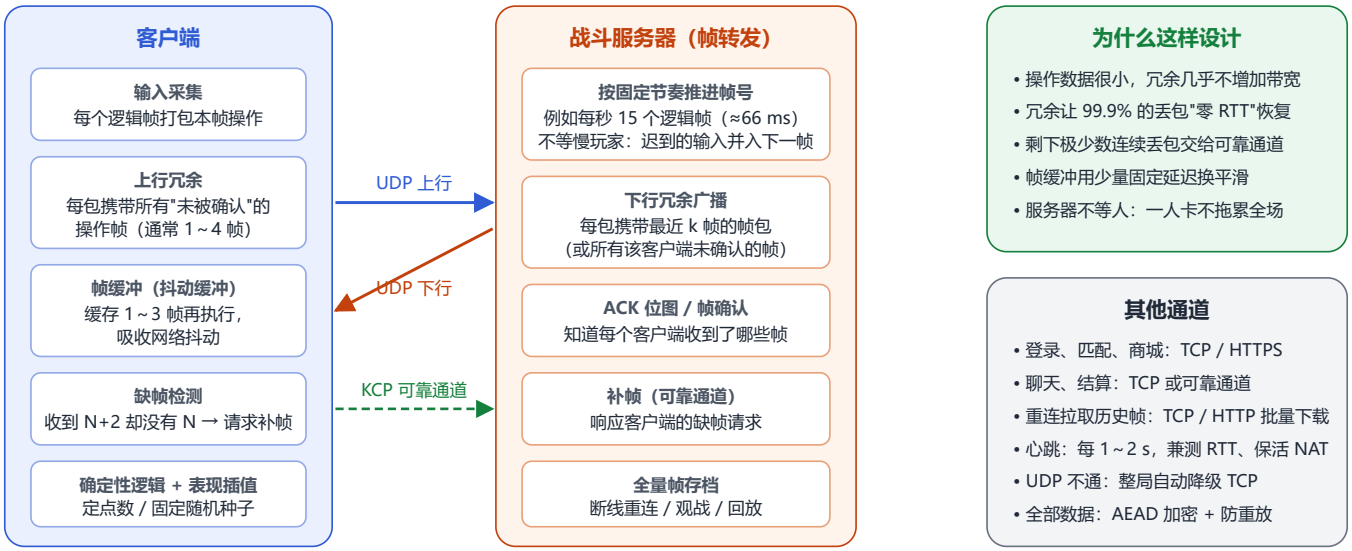


图 11-1 一个典型的帧同步网络层：UDP 冗余发送承担主要流量，可靠通道兜底，其他业务走 TCP

🍼 实测证据（d2 • 实验 B 中的 R 行）：冗余发送的效果

d2_rudp_sim.exe · 实验 B 摘录（100 Hz 消息流，RTT≈60 ms）

丢包	方案	平均ms	P99ms	最大ms	发送	丢失
5%	K 类KCP极速：快重传2+立即ACK+RTO×1.5	42.9	95	167	1249	0
5%	R 冗余发送k=3（不可靠）	31.6	41	51	1000	1
10%	K 类KCP极速：快重传2+立即ACK+RTO×1.5	53.4	123	173	1474	0
10%	R 冗余发送k=3（不可靠）	32.0	41	61	1000	0
20%	K 类KCP极速：快重传2+立即ACK+RTO×1.5	72.9	163	183	1907	0
20%	R 冗余发送k=3（不可靠）	33.3	51	61	1000	1

R 方案的每个包携带本条和前 3 条消息（共 4 条）。它的 P99 只比单程时延 30 ms 多 10～20 ms，**在 20% 丢包下依然稳定**，而且发送次数恒为 1000（只是每个包大一点）。代价是它不保证送达：理论残余丢失率约为 p^4，20% 丢包时约 0.16%，1000 条中丢了 1 条，与理论吻合。这就是为什么图 11-1 用"冗余为主、可靠通道兜底"：冗余处理掉绝大部分丢包，极少数漏网之鱼由 KCP 补发。

🎮 **游戏开发视角：几个容易忽略的细节**

① **上行冗余的终止条件**：客户端持续冗余发送某帧输入，直到服务器的包头 ACK 表明已收到——这就是第 8 章 UNA + ACK 位图的直接应用。

② **帧缓冲的大小要自适应**：根据实测抖动动态调整缓冲帧数，网络好时减少输入延迟，网络差时增加平滑度。

③ **包要小**：控制在 MTU 以内，避免 IP 分片（第 1 章）；10 个玩家 × 若干字节的操作，一个帧包通常只有几十到一两百字节。

④ **不要让一个慢玩家拖住所有人**：服务器按固定节奏推进帧号，没收到某玩家的输入就当作"无操作"，这和 TCP 的"必须等"截然不同。

11.3 实战二：状态同步

状态同步（如 UE 的 Actor 复制）由服务器计算权威状态，按需把属性变化发给客户端。它的传输策略可以对照可靠性光谱来理解：

数据	可靠性	机制
属性复制（位置、血量……）	可靠 · 只要最新	不可靠发送；检测到包丢失时，**不重传旧包**，而是把相关属性标记为"脏"，在下一次复制时发送**当前值**
Reliable RPC（开火、技能、交互）	可靠有序（同一 Actor 通道内）	丢失后重传；过多的可靠 RPC 积压会导致通道溢出、连接被断开，所以要节制使用
Unreliable RPC（特效、音效）	不可靠	发了就不管；带宽紧张时可能被丢弃
移动同步	不可靠 + 客户端预测 / 服务器校正	客户端先本地执行移动，服务器验证后纠正；输入用不可靠 RPC 高频发送，并冗余携带最近的移动

另外，状态同步还要做**相关性裁剪**（只同步玩家附近 / 视野内的对象）和**优先级与带宽预算**（每帧按优先级在预算内挑选要发送的对象），本质上是第 7 章架构图中"通道与调度层"的工作。

11.4 实战三：实时音视频（WebRTC）

RTP / RTCP over UDP

RTP 携带媒体数据（序号 + 时间戳），RTCP 反馈接收质量（丢包率、抖动、RTT）。全部通过 SRTP / DTLS 加密。

NACK：有条件的重传

接收方发现缺包时发 NACK 请求重传，但只有在"重传的包能在播放截止前到达"时才有意义——RTT 大时就放弃，改靠 FEC 或丢帧。

FEC + 抖动缓冲

FlexFEC / ULPFEC 用冗余包恢复丢包；抖动缓冲（jitter buffer）动态调整缓冲时长，平滑到达时间的波动。

基于时延的拥塞控制

GCC（Google Congestion Control）监测包到达间隔的变化趋势，在队列开始积压时就降低码率，而不是等到丢包。码率调整直接作用于编码器。

对比游戏和音视频会发现一个共同规律：**越是实时的场景，越倾向于"用带宽和画质换时延"**——FEC、冗余发送、主动降码率、降低快照频率，都是这个思路。

11.5 实战四：网页加载

优化	省掉了什么	对应章节
DNS 预解析（`dns-prefetch`）	DNS 查询的 RTT	第 1 章
预连接（`preconnect`）、HTTP Keep-Alive、连接池	TCP 握手 + TLS 握手；也避免了 TIME_WAIT 堆积	第 5、6 章
TLS 1.3 + 会话恢复	TLS 握手从 2 RTT 降到 1 RTT / 0 RTT	第 10 章
HTTP/2 多路复用	应用层队头阻塞、多连接的开销	第 4 章
HTTP/3（通过 Alt-Svc 升级）	传输层队头阻塞；弱网下的握手和恢复时间；换网断连	第 10 章
CDN 就近接入	物理 RTT	第 1 章
服务端 BBR、增大初始窗口	慢启动阶段的 RTT 数	第 3 章

11.6 弱网测试与质量指标

模拟工具

Windows：**clumsy**（可注入延迟、丢包、乱序、重复）；Linux：`tc qdisc add dev eth0 root netem delay 100ms 20ms loss 5%`；macOS / iOS：Network Link Conditioner；移动端还要实测地铁、电梯、弱信号场景。

关键指标

RTT 均值与 P99、**抖动**（RTT 的波动）、丢包率（上下行分别统计）、**重传占比**、消息端到端时延的 P99 / 最大值、卡顿次数（例如帧缓冲被打空的次数）、断线率与重连耗时。

经验法则

只看平均值会骗人：实验 B 中 T' 的平均时延只有 61 ms，但 P99 是 185 ms。玩家感知到的"卡"几乎都来自尾部时延，所以优化目标应该写成 P99 / P999。

11.7 横向对照：一个问题，多种答案

最后，用一张表把全书串起来。每一行是一个传输层问题，每一列是不同协议给出的答案。面试时如果能从一个点出发，横向联想到其他协议的同类设计，就是"有体系"的表现。

问题	TCP	KCP / 自研 RUDP	QUIC	游戏引擎 / 音视频
如何标识一条连接	四元组	conv 会话 ID	连接 ID（可迁移）	会话 ID / 玩家 ID
如何防伪造握手	随机 ISN、SYN Cookie	Cookie 握手（自行实现）	Retry 令牌、3 倍抗放大	先 TCP 登录拿票据
如何编号	字节序号	包序号 sn	包号（永增）+ 流偏移	包序号（常为 16 位）
如何确认	累计 ACK + SACK	UNA + 逐包 ACK / 位图	ACK 帧（多 Range）	包头捎带 ACK 位图
如何测 RTT	时间戳选项 / Karn	ts 回显	包号唯一 + ACK Delay	心跳 / 包头时间戳
如何发现丢包	RTO、3 dup ACK、RACK、TLP	RTO×1.5、跳过 N 次	包阈值 3、时间阈值 9/8、PTO	位图空洞、NACK
丢了怎么办	重传原数据	重传原包	把帧放进新包重传	发送最新状态 / FEC / 冗余 / 放弃
队头阻塞	有（整条连接）	有（单个 KCP 对象）	只在单个流内	分通道、不可靠数据不排队
流量控制	rwnd、零窗口探测	wnd、WASK / WINS	MAX_STREAM_DATA / MAX_DATA	带宽预算
拥塞控制	CUBIC / BBR	可关闭（nc=1）+ 限速	可插拔	基于时延（GCC）/ 降频降精度
旧报文 / 序号回绕	TIME_WAIT、PAWS	序号算术、窗口 < 空间一半	包号 62 位，实际不回绕	序号算术
如何发现对端	Keepalive（默认 2 h）	心跳、dead_link	空闲超时、PING 帧	应用心跳（秒级）

	消失			
如何关闭	四次挥手 / RST	CLOSE 命令 + 超时	CONNECTION_CLOSE 帧 + 空闲超时	业务层退出协议

本章小结

- 选型先看数据的可靠性需求，再看是否需要多路复用、加密、连接迁移、浏览器兼容。
- 帧同步：UDP 冗余发送为主（仿真中 20% 丢包 P99 仍只有 51 ms），可靠通道兜底，帧缓冲吸收抖动，TCP 处理非实时业务与降级。
- 状态同步：属性复制"只发最新值"、Reliable RPC 节制使用、相关性与优先级调度。
- 音视频：NACK 有条件重传 + FEC + 抖动缓冲 + 基于时延的拥塞控制。
- 网页：减少 RTT 个数是主线（预连接、长连接、TLS 1.3、HTTP/3、CDN）。
- 弱网测试看 P99 和抖动，不要只看平均值。

▶ **思考题 1：帧同步中，如果某个玩家网络特别差，服务器应该等他吗?**

▶ **思考题 2：为什么说"只看平均时延会骗人"？请用本书的仿真数据说明。**

▶ **思考题 3：设计一个手游的网络层，UDP 被运营商封了怎么办?**

CHAPTER 12

面试题库：40 道高频题

先自己口述一遍，再展开答案对照。每题末尾的【→】标出了对应章节，答不上来就回去重读。

使用方法

- 按"基础 → TCP 机制 → 握手 → 挥手 → 可靠 UDP → 场景设计"的顺序，难度递增
- 标 ★ 的是出现频率最高的题，至少要能完整说出 2 分钟
- 答题结构推荐"**结论** → **原因** → **细节 / 数字** → **延伸**"，延伸部分最能体现知识体系

12.1 基础（8 题）

▶ **1. ★ TCP 和 UDP 的区别?**

▶ **2. UDP 头部有哪些字段? 最大能发多大的数据报?**

▶ **3. 校验和为什么要加伪首部?**

▶ **4. 什么是 MTU 和 MSS? 为什么要避免 IP 分片?**

▶ **5. 一台服务器最多能支持多少个 TCP 连接?**

▶ **6. 同一个端口能同时被 TCP 和 UDP 使用吗?**

▶ **7. DNS 为什么用 UDP? 什么时候用 TCP?**

▶ **8. UDP 调用 connect() 有什么作用?**

12.2 TCP 机制（10 题）

▶ 9. ★ TCP 如何保证可靠传输?

▶ 10. ★ 什么是粘包 / 拆包？怎么解决?

▶ 11. RTO 是怎么计算的？Karn 算法是什么?

▶ 12. ★ 快速重传是什么？为什么是 3 个重复 ACK?

▶ 13. ★ 滑动窗口和流量控制是怎么工作的？零窗口怎么办?

▶ 14. ★ 拥塞控制有哪几个阶段?

▶ 15. 流量控制和拥塞控制的区别?

▶ 16. CUBIC 和 BBR 有什么区别?

▶ 17. Nagle 算法和延迟确认是什么？为什么游戏要关闭 Nagle?

▶ 18. TCP 头部有哪些标志位？各自含义?

12.3 三次握手（8 题）

▶ 19. ★ 描述三次握手的过程和状态变化。

▶ 20. ★ 为什么是三次握手，不是两次或四次?

▶ 21. ISN 为什么要随机?

▶ 22. ★ 什么是半连接队列和全连接队列？满了会怎样？

▶ 23. ★ SYN Flood 是什么？怎么防？

▶ 24. 第三次握手的 ACK 丢了会怎样？

▶ 25. 第一次、第二次握手的报文丢了分别会怎样？

▶ 26. connect 到一个没人监听的端口会怎样？

12.4 四次挥手（7 题）

▶ 27. ★ 描述四次挥手的过程和状态变化。

▶ 28. ★ 为什么挥手需要四次？什么时候是三次？

▶ 29. ★ TIME_WAIT 为什么存在？为什么是 2MSL？

▶ 30. ★ TIME_WAIT 过多有什么危害？怎么解决？

▶ 31. 大量 CLOSE_WAIT 说明什么？

▶ 32. close 和 shutdown 的区别？SO_LINGER 有什么用？

▶ 33. 对端机器突然断电，本端能发现吗？

12.5 可靠 UDP（5 题）

▶ 34. ★ 如何基于 UDP 实现可靠传输？

▶ **35. ★ KCP 为什么比 TCP 快？代价是什么？**

▶ **36. 停等、回退 N 步、选择重传的区别？**

▶ **37. ★ QUIC 相比 TCP 有哪些改进？**

▶ **38. 序号回绕怎么处理？**

12.6 场景设计（2 题）

▶ **39. ★ 如果让你设计一个 MOBA 游戏的网络同步方案，传输层怎么选？**

▶ **40. 视频会议为什么不用 TCP？它怎么应对丢包？**

APPENDIX

速记卡：考前 10 分钟

关键数字、默认参数、状态与口述模板。打印出来贴在桌上。

A.1 关键数字

头部与长度

UDP 头 8 字节；TCP 头 20～60 字节；IPv4 头 20 字节；KCP 头 24 字节

以太网 MTU 1500；TCP MSS 1460；UDP 安全负载 ≤ 1472（实践 1200～1400）；QUIC Initial ≥ 1200

UDP 最大负载 65507；TCP 窗口字段 16 位，窗口扩大最多左移 14 位

TCP 定时器（Linux 默认）

初始 RTO 1 s；RTO 下限 200 ms；超时 ×2 退避

tcp_syn_retries = 6（约 127 s）；

tcp_synack_retries = 5（约 63 s）

TIME_WAIT 60 s（TCP_TIMEWAIT_LEN，不可调）；tcp_fin_timeout 60 s（FIN_WAIT_2）

Keepalive：7200 s 空闲 / 75 s 间隔 / 9 次

TCP 其他

快速重传：3 个重复 ACK；初始拥塞窗口 10 MSS

延迟确认：Linux 约 40 ms 起，Windows 默认 200 ms

somaxconn 默认 4096（Linux 5.4+，此前 128）

临时端口 32768～60999；tcp_tw_recycle 已在 4.12 删除

KCP

极速模式 `ikcp_nodelay(kcp, 1, 10, 2, 1)`；普通 `(0, 40, 0, 0)`

窗口默认 32 / 128；MTU 1400；RTO 下限 100（nodelay 30）ms；超时 ×1.5

fastlimit 5；dead_link 20；命令 PUSH 81 / ACK 82 / WASK 83 / WINS 84

QUIC

RFC 9000 / 9001 / 9002（2021）；HTTP/3 RFC 9114（2022）；DATAGRAM RFC 9221

1-RTT 握手，0-RTT 恢复；包阈值 3；时间阈值 9/8 RTT；抗放大 3 倍

本书实测

Windows connect 空端口约 2048 ms 才失败（E3）

UDP 发空端口：sendto 成功、recvfrom 报 10054（E2）

流 ID 62 位；两级流控 MAX_STREAM_DATA / MAX_DATA

仿真 5% 丢包批量传输：SR+快重传 4.0 s，GBN 36 s，停等 172 s，类 KCP 3.6 s（重传占比 30%）

A.2 状态速查

状态	谁会处于	在等什么	大量出现说明
LISTEN	服务器	SYN	—
SYN_SENT	客户端	SYN+ACK	对端不可达 / 被防火墙丢弃
SYN_RCVD	服务器	第三次 ACK	SYN Flood，或全连接队列满
ESTABLISHED	双方	—	（可能有对端已消失的半打开连接）
FIN_WAIT_1	主动关闭方	对 FIN 的 ACK	网络差 / 对端不回应
FIN_WAIT_2	主动关闭方	对方的 FIN	对端应用迟迟不 close
CLOSE_WAIT	被动关闭方	本端应用 close	**本端代码 bug**
LAST_ACK	被动关闭方	最后的 ACK	网络差
CLOSING	双方（同时关闭）	对 FIN 的 ACK	少见
TIME_WAIT	主动关闭方	2MSL 超时	短连接多；客户端侧警惕端口耗尽

A.3 三个口述模板（各约 1 分钟）

三次握手

"客户端发 SYN，带上随机的初始序号 x，进入 SYN_SENT；服务器收到后进入 SYN_RCVD，回复 SYN+ACK，带自己的序号 y 和确认号 x+1；客户端收到后进入 ESTABLISHED，再回 ACK，确认号 y+1，服务器收到后也进入 ESTABLISHED。三次是最少的次数：主要是为了防止历史连接——旧的 SYN 到达时，客户端能在第三步用 RST 拒绝；同时双方的初始序号都需要被确认；还能避免

重复 SYN 建立无用连接。服务器的 ACK 和 SYN 合并了，所以不用四次。另外，内核里有半连接和全连接两个队列，SYN Flood 攻击的就是半连接队列，可以用 SYN Cookie 防御。"

四次挥手

"主动方发 FIN 进入 FIN_WAIT_1；被动方内核回 ACK，进入 CLOSE_WAIT，主动方收到后进入 FIN_WAIT_2，这时是半关闭，被动方还能继续发数据；被动方应用调用 close 后发 FIN，进入 LAST_ACK；主动方收到后回 ACK 进入 TIME_WAIT，等 2MSL 后关闭，被动方收到 ACK 就关闭。之所以四次，是因为 TCP 全双工，两个方向要分别关闭，被动方的 ACK 和 FIN 之间可能还有数据要发；如果没有数据并且有延迟确认，也可能合并成三次。"

TIME_WAIT

"TIME_WAIT 出现在主动关闭方，持续 2MSL，Linux 是固定的 60 秒。它有两个作用：一是保证最后一个 ACK 能到达，如果丢了，对方会重传 FIN，主动方还能重新回 ACK，否则只能回 RST；二是让这个连接在网络中残留的旧报文全部过期，避免被相同四元组的新连接误收。2MSL 就是一个来回的最长时间。TIME_WAIT 太多时，客户端会耗尽本地端口，最好的办法是用长连接或连接池，其次可以开 tcp_tw_reuse；tcp_tw_recycle 在 NAT 环境下有问题，已经从内核删除了。"

A.4 一句话串起全书

> 🧭 **回到起点**
>
> IP 只尽力而为；UDP 把选择权留给应用；TCP 用序号、确认、重传、窗口、拥塞控制和连接管理把网络包装成可靠有序的字节流，并为此付出了握手延迟和队头阻塞的代价；三次握手同步序号、四次挥手分别关闭两个方向、TIME_WAIT 收尾；而 KCP、QUIC 和游戏引擎的网络层，则是在用户态把同一套机制按自己的需求重新组合——**可靠性不是一个开关，而是一组可以挑选、可以调参的机制，每一个都有代价。**